

BACKEND SOFTWARE JOURNEY

The First 90 Days

A practical ASP.NET Core backend build using Clean Architecture, SQL Server, Entity Framework Core, authentication, testing, and continuous integration.

90 focused lessons. One evolving production-style API.

Contents

The journey is organized into three 30-day phases. Review days consolidate the work before the next phase begins.

Month 1

Day 1 - Backend Setup

Day 2 - User Entity

Day 3 - First Task Feature

Day 4 - Cleanup And Validation

Day 5 - Get Task By ID

Day 6 - Update Task Endpoint

Day 7 - Delete Task Endpoint

Day 8 - Clean Task CRUD

Day 9 - Improve API Responses

Day 10 - Complete Task Endpoint

Day 11 - Reopen Task Endpoint

Day 12 - Task Completion Filtering

Day 13 - Task Search By Title

Day 14 - Review Task API

Day 15 - User Service Structure

Day 16 - First User API Endpoints

Day 17 - Get User By ID Endpoint

Day 18 - Update User Endpoint

Day 19 - Delete User Endpoint

Day 20 - Connect Tasks To Users

Day 21 - Review User And Task APIs

Day 22 - Prepare Infrastructure Layer

Day 23 - Add EF Core Database Context

Day 24 - Register SQL Server DbContext

Day 25 - Create First EF Core Migration

Day 26 - Apply EF Core Migration

Day 27 - Move User Storage To SQL Server

Day 28 - Move Task Storage To SQL Server

Day 29 - Test Database Persistence

Day 30 - First Backend Month Review

Month 2

Day 31 - Review Database CRUD

Day 32 - Add Task User Relationship

Day 33 - Get Tasks By User Endpoint

Day 34 - Add UserId Filter To Task List

Day 35 - Add Optional Due Date To Tasks

Day 36 - Add Simple Task Priority

Day 37 - Review Database Task Changes

Day 38 - Add Data Annotation Validation

Day 39 - Review ASP.NET Core Validation Responses

Day 40 - Centralize Common API Messages

Day 41 - Add Basic Global Exception Handling

Day 42 - Review HTTP Status Codes

Day 43 - Review Response DTO Consistency

Day 44 - Weekly Validation Review

Day 45 - Review Application Layer Boundaries

Day 46 - Review Infrastructure Layer

Day 47 - Review Service Interfaces

Day 48 - Add Async Suffix To Service Methods

Day 49 - Remove Dead Code And Starter Files

Day 50 - Architecture Review Day

Day 51 - Plan Authentication Flow

Day 52 - Create Auth DTOs

Day 53 - Add Password Hashing Service

Day 54 - Add Register Use Case

Day 55 - Add Login Use Case

Day 56 - Review Auth Controller Endpoints

Day 57 - Add JWT Configuration

Day 58 - Generate JWT Token On Login

Day 59 - Protect Task Endpoints

Day 60 - Month 2 Review

Month 3

Day 61 - Review Authentication

Day 62 - Add User Id Claim Usage

Day 63 - Make Tasks User-Owned

Day 64 - Restrict Get All Tasks

Day 65 - Restrict Get Task By Id

Day 66 - Restrict Update Task
Day 67 - Restrict Delete Task
Day 68 - Review User Ownership
Day 69 - Add Basic Unit Test Project
Day 70 - Test Task Creation
Day 71 - Test Task Get By Id
Day 72 - Test Task Update
Day 73 - Test Task Delete
Day 74 - Test User Service
Day 75 - Test Auth Service
Day 76 - Test Build And CI Readiness
Day 77 - Add GitHub Actions Build
Day 78 - Add GitHub Actions Tests
Day 79 - Review README For Testing
Day 80 - Add Basic Request Logging
Day 81 - Review Structured Error Responses
Day 82 - Add ProblemDetails Error Responses
Day 83 - Add A Health Check Endpoint
Day 84 - Review Security Basics
Day 85 - Clean Up Application Settings
Day 86 - Document The Local Environment
Day 87 - Manual API Regression Test
Day 88 - Cleanup Before The Month Review
Day 89 - Update README And Progress
Day 90 - Month 3 Review

How to Use This Book

Each day has one focused goal. Read the concept, examine the implementation shape, complete the exercise, and verify the result before moving forward.

The project grows incrementally. Later lessons intentionally build on decisions made earlier, while review days pause feature work to validate the system as a whole.

Code samples are learning snapshots. Adapt configuration, secrets, database settings, and deployment details to your own environment.

MONTH 1

FOUNDATIONS

Build the API, shape task and user features, and introduce database persistence.



Day 1 - Backend Setup

Lesson Goal

Create the first working backend solution skeleton for JobTrackr.

By the end of this lesson, the project has:

- one solution
- one ASP.NET Core Web API project
- three support class libraries
- project references
- a successful API run

What Changed

The backend project starts with a clean solution structure.

Important project folders:

- src/JobTrackr.Api
- src/JobTrackr.Application
- src/JobTrackr.Domain
- src/JobTrackr.Infrastructure

Core Concept

Before adding features, a backend needs a stable project structure.

The Day 1 structure introduces the main layers:

- API layer for HTTP endpoints
- Application layer for use cases
- Domain layer for business models
- Infrastructure layer for database and external systems later

Commands

```
dotnet new sln -n JobTrackr
dotnet new webapi -n JobTrackr.Api -o src/JobTrackr.Api --framework net8.0
dotnet new classlib -n JobTrackr.Application -o src/JobTrackr.Application --framework net8.0
dotnet new classlib -n JobTrackr.Domain -o src/JobTrackr.Domain --framework net8.0
dotnet new classlib -n JobTrackr.Infrastructure -o src/JobTrackr.Infrastructure --framework net8.0
dotnet build
dotnet run --project src/JobTrackr.Api
```

Why This Matters

The first day is not about business features. It is about proving the environment works.

If the solution builds and the API runs, future lessons can focus on backend concepts instead of setup problems.

Lesson Explanation

The first step in building JobTrackr was creating a backend solution that could grow over time.

Instead of putting everything into one project, the solution starts with four projects. This gives each part of the backend a clear responsibility from the beginning.

The API project is the entry point. It receives HTTP requests and returns HTTP responses.

The Application project will hold the use cases. These are the actions the system supports, such as creating a task or updating a task.

The Domain project will hold the core business objects. These objects should describe the problem we are solving, not the technology we are using.

The Infrastructure project is reserved for database and external system code later.

On Day 1, most of these projects are empty. That is intentional. The goal is not to build everything immediately. The goal is to create a structure that is simple enough to understand and strong enough to grow.

Reader Exercise

Create the same solution structure from scratch and run the API once.

Then explain what each project is responsible for in one sentence.

Future Improvement

Later, this structure will support real features, persistence, authentication, testing, and deployment.

Next Lesson

After the backend skeleton runs, the next step is adding the first real domain model: User.

Day 2 - User Entity

Lesson Goal

Create the first real domain model in JobTrackr.

By the end of this lesson, the Domain project has:

- Entities folder
- User.cs
- basic user properties
- a successful build

What Changed

Important file:

- src/JobTrackr.Domain/Entities/User.cs

The first business entity is added to the Domain layer.

Core Concept

A backend starts with business meaning.

The User entity is not an HTTP endpoint and not a database table yet. It is the first model that describes something important in the system.

Code Shape

```
namespace JobTrackr.Domain.Entities;

public class User
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
}
```

Why This Matters

The Domain layer should describe the core objects of the application.

Starting with a simple entity keeps the project understandable while introducing the habit of separating business models from API code.

Lesson Explanation

After the solution structure was created, the next step was to add meaning to the backend.

Day 2 introduced the first domain entity: User.

A domain entity represents something the system cares about. In JobTrackr, users will eventually own tasks, apply for jobs, and interact with the application. Even though the user feature is not complete yet, adding the entity starts shaping the business model.

The User class stays intentionally simple:

- Id identifies the user.
- FullName stores the user's name.
- Email stores contact identity.
- CreatedAtUtc records when the user was created.

At this stage, there is no database and no controller. That is deliberate. The goal is to understand the Domain layer before connecting it to HTTP or persistence.

Reader Exercise

Add one more property to the User entity, such as:

```
public bool IsActive { get; set; } = true;
```

Then explain whether that property belongs in the Domain layer or somewhere else.

Future Improvement

Later, the User entity can be connected to authentication, persistence, and task ownership.

Next Lesson

After creating the first domain entity, the next step is creating the first task feature that connects API, Application, and Domain layers.

Day 3 - First Task Feature

Lesson Goal

Create the first simple task feature.

By the end of this lesson, JobTrackr supports:

```
GET /api/tasks
POST /api/tasks
```

What Changed

Important files:

- src/JobTrackr.Domain/Entities/JobTask.cs
- src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
- src/JobTrackr.Application/Tasks/TaskResponse.cs
- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Api/Controllers/TasksController.cs
- src/JobTrackr.Api/Program.cs

Core Concept

Day 3 connects the layers for the first time:

```
Api -> Application -> Domain
```

The controller receives HTTP requests. The service handles application logic. The domain entity represents the business object.

API Shape

Create task:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Create first task endpoint",
  "description": "Practice controller to service flow"
}
```

Get all tasks:

```
GET /api/tasks
```

Why This Matters

This is the first real backend workflow in the project.

It shows how an API request travels from the controller into the application layer and returns a clean response to the client.

Lesson Explanation

Day 3 is where JobTrackr starts behaving like a real backend API.

The project already had separate layers, but they were not doing much together. The task feature connects them.

The Domain layer gets a new entity called JobTask. The name avoids conflict with C#'s built-in Task type and clearly describes the business object.

The Application layer gets request and response DTOs. CreateTaskRequest describes the data the API user sends when creating a task. TaskResponse describes the data returned by the API.

The Application layer also gets ITaskService and TaskService. The interface defines what the task feature can do. The service implements the logic.

For now, tasks are stored in memory. This is not production persistence, but it is useful for learning the flow without introducing SQL Server too early.

The API layer gets TasksController. The controller exposes GET /api/tasks and POST /api/tasks. It does not manage the task list directly. It calls the service.

This is the first clean architecture habit in practice: the controller should handle HTTP, while application logic belongs in the application layer.

Reader Exercise

Create a task through Swagger, then call GET /api/tasks.

Explain the flow in this order:

HTTP request -> controller -> service -> domain entity -> response DTO -> HTTP response

Future Improvement

The task list is currently in memory. Later, it should be saved in SQL Server through the Infrastructure layer.

Next Lesson

After the first feature works, the next step is cleanup and validation.

Day 4 - Cleanup And Validation

Lesson Goal

Clean up and stabilize the Day 3 task feature.

By the end of this lesson:

- default WeatherForecast code is removed
- Program.cs is cleaner
- task creation rejects an empty title

- existing task endpoints still work

What Changed

Important files:

- src/JobTrackr.Api/Program.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Api/Controllers/TasksController.cs

Core Concept

Backend development is not only adding new features.

A normal workflow is:

```
Build small feature -> test it -> clean it -> continue
```

Day 4 makes the first task workflow more professional before adding more endpoints.

API Shape

Valid create request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Clean Day 3 task API",
  "description": "Remove template code and add validation"
}
```

Invalid create request:

```
{
  "title": "",
  "description": "This should fail"
}
```

Expected result:

```
400 Bad Request
```

Why This Matters

APIs need clear behavior when input is invalid.

Returning 400 Bad Request for an empty task title makes the API easier to use and easier to test.

Lesson Explanation

Day 4 is a cleanup day.

That may sound less exciting than adding a new endpoint, but cleanup is a real part of backend engineering. If a project only grows and never gets cleaned, the code becomes harder to understand.

The first cleanup removes the default WeatherForecast code from the ASP.NET Core template. That code was useful when the project was created, but it does not belong in JobTrackr.

The second improvement is validation.

Before this lesson, the API could create a task with an empty title. That is not useful behavior. A task needs a title because the title is the main thing the user sees.

The validation rule belongs in the application service because it is part of the task use case:

```
if (string.IsNullOrEmpty(request.Title))
{
    throw new ArgumentException("Task title is required.");
}
```

The controller catches that validation failure and converts it into an HTTP response:

```
400 Bad Request
```

This keeps the responsibilities clear. The service protects the business rule. The controller translates the result into HTTP.

Reader Exercise

Add validation for a maximum title length.

Example rule:

```
Task title must be 100 characters or fewer.
```

Then return 400 Bad Request when the title is too long.

Future Improvement

Later, validation can move to a more structured approach, such as validation classes or middleware.

Next Lesson

After create and get-all work cleanly, the next step is reading one specific task by id.

Day 5 - Get Task By ID

Lesson Goal

Add an endpoint to get one task by id.

By the end of this lesson, JobTrackr supports:

```
GET /api/tasks/{id}
```

The endpoint returns a task when it exists and 404 Not Found when it does not.

What Changed

Important files:

- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Api/Controllers/TasksController.cs

Core Concept

Getting one task by id is a search operation.

The service searches the task list for a matching id. The controller turns the result into the correct HTTP response.

API Shape

Create a task first:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Test get by id",
  "description": "Create a task then fetch it by id"
}
```

Get existing task:

```
GET /api/tasks/1
```

Expected:

```
200 OK
```

Get missing task:

```
GET /api/tasks/999
```

Expected:

```
404 Not Found
```

Why This Matters

Most APIs need a way to fetch one specific record.

List endpoints are useful, but real clients often need details for one selected item.

Lesson Explanation

Day 5 adds the next read pattern: get one task by id.

The API already supports creating tasks and returning all tasks. That is useful, but incomplete. A client usually needs to open one task, inspect its details, or prepare it for editing.

The route describes the resource:

```
GET /api/tasks/{id}
```

The `{id}` part comes from the URL. If the client calls `/api/tasks/1`, the controller receives `1` as the task id.

The controller asks the service for that task:

```
var task = await _taskService.GetById(id);
```

The service searches the current task list. If it finds a matching task, it returns the response. If it does not, it returns `null`.

The controller then converts `null` into:

```
404 Not Found
```

This is an important backend pattern. The application service can represent "not found" as `null`, while the API layer decides that `null` means HTTP 404.

Reader Exercise

Write the logic for searching an array of numbers for a target value.

Then compare it with searching the task list by id.

Future Improvement

When SQL Server is added, this search will become a database query instead of an in-memory list search.

Next Lesson

After the API can create tasks, list tasks, and fetch one task, the next CRUD step is updating an existing task.

Day 6 - Update Task Endpoint

Lesson Goal

Add an update endpoint for tasks so an existing task can be modified through the API.

By the end of this lesson, the project supports:

```
PUT /api/tasks/{id}
```

The endpoint updates the task title and description for an existing task.

What Changed

The Day 6 work extends the task feature from read/create behavior into update behavior.

Important files:

- src/JobTrackr.Api/Controllers/TasksController.cs
- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs

Core Concept

An update endpoint usually needs three pieces of information:

- The resource id from the route
- The new data from the request body
- A service method that applies the change

For this project, the route id identifies which task should be updated, while the request body carries the new title and description.

API Shape

Example request:

```
PUT /api/tasks/1
Content-Type: application/json

{
  "title": "Apply for backend developer role",
  "description": "Update resume and send application"
}
```

Expected behavior:

- If the task exists, update it and return the updated task.

- If the task does not exist, return 404 Not Found.
- If the title is empty, return 400 Bad Request.

Why This Matters

Create, read, update, and delete are the foundation of most backend APIs.

This lesson moves JobTrackr closer to a real task management backend because users need to correct, rename, or refine tasks after creating them.

Lesson Explanation

When we added the update task endpoint, we introduced the first real modification workflow in the API.

Creating a task adds a new item to the system. Getting a task reads existing state. Updating is different because it changes an object that already exists. That means the code has to answer two questions before making the change:

1. Does this task exist?
2. Is the new data valid?

The route handles identity:

```
PUT /api/tasks/{id}
```

The request body handles the new values:

```
{
  "title": "Updated title",
  "description": "Updated description"
}
```

This keeps the API clear. The URL says which task we are changing. The body says what we want to change it to.

The controller should stay focused on HTTP behavior. It receives the route id and request body, calls the application service, and converts the result into the right HTTP response. The application service owns the use-case logic, including finding the task and applying validation.

That separation is one of the early clean architecture habits in this project: controllers should not become the place where all business decisions live.

Reader Exercise

After completing this lesson, try adding one more field to the task update request.

Example:

```
{
  "title": "Updated title",
  "description": "Updated description",
  "isCompleted": true
}
```

Then update the service and response so the API can mark a task as completed.

Future Improvement

The current project is still using simple in-memory behavior. Later, this update flow should move toward persistence with SQL Server and Entity Framework Core.

When that happens, the same API concept remains:

- Route id identifies the task
- Request body provides the update data
- Service/application layer validates and applies the change
- Infrastructure layer saves the change

Next Lesson

The next natural feature is deleting a task:

```
DELETE /api/tasks/{id}
```

After create, get, get by id, and update, delete will complete the basic CRUD workflow.

Day 7 - Delete Task Endpoint

Lesson Goal

Add an endpoint to delete an existing task.

By the end of this lesson, JobTrackr supports:

```
DELETE /api/tasks/{id}
```

The endpoint deletes the task when it exists and returns 404 Not Found when it does not.

What Changed

Day 7 completes the basic in-memory CRUD flow for tasks.

Important files:

- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Api/Controllers/TasksController.cs
- README.md

Core Concept

Deleting a resource is usually a find-and-remove operation.

The service searches for the task by id. If the task exists, it removes it from the list. If the task does not exist, the service reports that deletion failed.

The controller converts that result into HTTP:

- 204 No Content when deletion succeeds
- 404 Not Found when the task does not exist

API Shape

Create a task first:

```
POST /api/tasks
Content-Type: application/json

{
```

```
"title": "Task to delete",  
"description": "This task should be deleted"  
}
```

Delete existing task:

```
DELETE /api/tasks/1
```

Expected:

```
204 No Content
```

Try to get the deleted task:

```
GET /api/tasks/1
```

Expected:

```
404 Not Found
```

Delete missing task:

```
DELETE /api/tasks/999
```

Expected:

```
404 Not Found
```

Why This Matters

After Day 7, the task API has the basic CRUD workflow:

```
Create -> Read all -> Read one -> Update -> Delete
```

This is a major early checkpoint. Even though the data is still in memory, the API now has the same basic shape used by many real backend systems.

Lesson Explanation

Day 7 adds the delete task endpoint.

At this point, JobTrackr can create tasks, return all tasks, return one task by id, and update an existing task. The missing CRUD operation is delete.

The route is:

```
DELETE /api/tasks/{id}
```

The id comes from the URL. The controller does not need a request body because deleting a task only requires knowing which task should be removed.

The application service exposes a method such as:

```
Task<bool> DeleteTask(int id);
```

The bool return value keeps the result simple:

- true means the task was found and deleted.
- false means the task was not found.

Inside the service, the logic is straightforward:

```
var task = _tasks.Find(x => x.Id == id);  
  
if (task is null)  
{  
    return false;  
}  
  
_tasks.Remove(task);
```

```
return true;
```

The controller turns that service result into the correct HTTP response:

```
if (!isDeleted)
{
    return NotFound("Task is not found");
}

return NoContent();
```

The important detail is the 204 No Content response. A successful delete does not need to return the deleted task. The client only needs to know the operation succeeded.

This lesson also updates the README so the public repository matches the current API. Documentation should move with the code, especially when the project is being used as a learning archive.

Reader Exercise

Add a second delete-style method that clears all completed tasks.

Example route idea:

```
DELETE /api/tasks/completed
```

Think through what status code should be returned when there are no completed tasks to delete.

Future Improvement

The delete logic currently removes tasks from an in-memory list. Later, when SQL Server and Entity Framework Core are added, this operation will become a database delete.

The API behavior should stay the same even when the storage implementation changes.

Next Lesson

With basic CRUD complete, the next step is to improve the project structure and prepare for persistence.

The future lessons should move toward:

- SQL Server
 - Entity Framework Core
 - repositories
 - better validation
 - tests
-

Day 8 - Clean Task CRUD

Lesson Goal

Clean and stabilize the completed Task CRUD feature before starting the next module.

Day 8 does not add a new endpoint. The goal is to make the existing task code easier to read, remove starter noise, and confirm the project still builds after cleanup.

What Changed

Day 8 turns the first CRUD milestone into cleaner book-quality code.

Important files:

- src/JobTrackr.Api/Controllers/TasksController.cs
- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
- src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
- src/JobTrackr.Application/Tasks/TaskResponse.cs
- src/JobTrackr.Domain/Entities/JobTask.cs
- src/JobTrackr.Domain/Entities/User.cs
- src/JobTrackr.Application/Class1.cs
- src/JobTrackr.Domain/Class1.cs
- src/JobTrackr.Infrastructure/Class1.cs

The default Class1.cs starter files were removed because they were not part of the real project.

Core Concept

Professional backend work is not only adding features.

After a feature reaches a working checkpoint, the next step is cleanup:

```
Build feature -> verify behavior -> clean code -> document -> continue
```

This prevents the project from becoming harder to change with every new lesson.

API Shape

The API shape stays the same after cleanup:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
```

The expected behavior also stays the same:

- 200 OK for successful reads and updates
- 201 Created for task creation
- 204 No Content for successful delete
- 400 Bad Request for invalid task title
- 404 Not Found for missing task id

Why This Matters

Day 8 protects the project from accidental complexity.

At the end of Day 7, the API could create, read, update, and delete tasks. That was enough to prove the feature worked, but working code still needs review. Small cleanup steps make the next feature easier to add.

This is especially important in a learning project that will become a public repository and a book. Readers should see the habit of stopping at checkpoints, removing noise, and verifying the code before moving forward.

Lesson Explanation

Day 8 is a cleanup day.

The task CRUD workflow is now complete, so this lesson pauses feature development and reviews the code that already exists. This is a normal professional workflow. A backend project should not move from feature to feature without checking whether the code is still readable.

The first cleanup target is starter code. New class library projects often include default files such as:

```
Class1.cs
```

These files are useful only as placeholders. Once the real project has domain entities, application services, request models, and controllers, the starter files become noise. Removing them makes the solution easier to scan.

The next cleanup target is naming and formatting. The code should explain itself through clear method names, request model names, and response model names. The task service should expose only the operations the API needs:

```
Task<IEnumerable<TaskResponse>> GetAllTasks();  
Task<TaskResponse> CreateTask(CreateTaskRequest request);  
Task<TaskResponse?> GetTaskById(int id);  
Task<TaskResponse?> UpdateTask(int id, UpdateTaskRequest request);  
Task<bool> DeleteTask(int id);
```

The controller remains responsible for HTTP responses. The service remains responsible for task behavior. This separation matters because it keeps the API layer thin and makes the application logic easier to move later when persistence is added.

The lesson also reinforces a useful rule: cleanup should not secretly become redesign.

Day 8 does not add SQL Server, Entity Framework Core, authentication, or a new User API. Those may come later, but the point of this lesson is to stabilize the current milestone.

After cleanup, the build should still pass. The final result should feel boring in a good way: same behavior, cleaner code, less noise.

Reader Exercise

Open a small project you have worked on before and find three pieces of starter or unused code.

For each one, decide:

- Is this file still used?
- Does it help explain the project?
- Would removing it make the project easier to understand?

Only delete code when you can build and verify the project afterward.

Future Improvement

The task CRUD feature still uses in-memory storage. Later lessons should replace this with real persistence using SQL Server and Entity Framework Core.

When that happens, the external API behavior should stay stable even though the internal storage implementation changes.

Next Lesson

With the task CRUD code cleaned, the project is ready for the next module.

The next lessons can move toward a new feature area or start preparing the architecture for persistence.

Day 9 - Improve API Responses

Lesson Goal

Improve the consistency and correctness of the existing Task API responses without adding a new feature module.

Day 9 keeps the API focused on the completed task CRUD workflow. The work is small, but it teaches an important backend habit: once an API works, the next step is making its responses predictable for clients.

What Changed

Day 9 improves response behavior in the current task endpoints.

Important files:

- src/JobTrackr.Api/Controllers/TasksController.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- README.md

The main changes are:

- Missing tasks now use the same not-found message everywhere.
- Title validation keeps one consistent validation message.
- Task creation returns 201 Created instead of 200 OK.
- The README documents the current API behavior more clearly.

Core Concept

An API should not only return data. It should communicate clearly through status codes and response messages.

For a create endpoint, this distinction matters:

```
200 OK      -> the request succeeded
201 Created -> the request succeeded and created a new resource
```

Both can look similar during manual testing, but 201 Created gives API clients a more accurate contract.

API Shape

The task endpoints stay the same:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
```

```
DELETE /api/tasks/{id}
```

The expected behavior after Day 9:

- GET /api/tasks returns 200 OK
- POST /api/tasks returns 201 Created
- GET /api/tasks/{id} returns 200 OK or 404 Not Found
- PUT /api/tasks/{id} returns 200 OK, 400 Bad Request, or 404 Not Found
- DELETE /api/tasks/{id} returns 204 No Content or 404 Not Found

Response Consistency

Before this cleanup, the controller could drift into slightly different messages for the same problem.

Day 9 standardizes missing task responses:

```
Task not found.
```

It also keeps validation consistent:

```
Task title is required.
```

This may feel minor, but inconsistent messages become a problem when frontends, tests, documentation, and API consumers start depending on responses.

CreatedAtAction

The create endpoint now uses:

```
return CreatedAtAction(nameof(GetById), new { id = response.Id }, response);
```

This does three useful things:

- returns 201 Created
- includes the created task in the response body
- points to the endpoint where the new task can be retrieved

This is a better HTTP response for a successful POST.

Why This Matters

Day 9 is about treating the API like a real contract.

When the project is small, it is easy to accept rough responses because everything is still visible in one controller. But the project is going to grow. It will later include persistence, more modules, authentication, tests, and probably a frontend.

Small consistency decisions now prevent confusion later.

A backend project should not wait until it is large before acting professionally. The habit starts while the codebase is still small enough to clean easily.

Lesson Explanation

At this point in the project, the task API can create, read, update, and delete tasks. That means the first CRUD milestone is working.

But working is not the same as polished.

The next improvement is response consistency. API clients do not only care about whether an endpoint returns something. They care about what the status code means, what message appears when something fails, and whether similar failures behave the same way.

The create endpoint is the first example. A task is a resource. When the API creates a resource, the correct response is not just:

```
200 OK
```

A better response is:

```
201 Created
```

In ASP.NET Core, `CreatedAtAction` is a clean way to express this. It returns the created object and connects the response to the endpoint that can retrieve the new resource.

The second improvement is consistency in error messages. If one endpoint says:

```
Task is not found
```

and another says:

```
Task not found.
```

the user understands both, but the API contract becomes messy. A frontend developer or test writer should not need to handle multiple versions of the same message.

Day 9 fixes this by using one message everywhere:

```
Task not found.
```

The validation message also stays consistent:

```
Task title is required.
```

The lesson is simple: small API response details matter because they become part of the system's public behavior.

Reader Exercise

Pick one endpoint from your own API and write down its expected responses.

Include:

- success status code
- validation failure status code
- not-found status code
- exact error message text

Then check the code and confirm whether the endpoint really behaves that way.

Future Improvement

The API still returns plain string messages for some errors.

Later, the project can introduce a standard error response shape, such as:

```
{  
  "message": "Task not found."  
}
```

or a fuller problem-details response.

That should wait until the project needs a broader error-handling pattern.

Next Lesson

The task CRUD API now has cleaner response behavior.

The next lesson can continue improving backend quality without rushing into a new module too early.

Day 10 - Complete Task Endpoint

Lesson Goal

Add a focused workflow endpoint that marks a task as completed.

Day 10 adds:

```
PATCH /api/tasks/{id}/complete
```

This endpoint does not replace the existing update endpoint. It represents a specific action in the task workflow: completing a task.

What Changed

Day 10 adds completion behavior to the task API.

Important files:

- src/JobTracker.Application/Tasks/ITaskService.cs
- src/JobTracker.Application/Tasks/TaskService.cs
- src/JobTracker.Api/Controllers/TasksController.cs
- README.md

The main changes are:

- ITaskService now exposes a CompleteTask method.
- TaskService can find a task and set IsCompleted to true.
- TasksController exposes PATCH /api/tasks/{id}/complete.
- Missing tasks still return the consistent Task not found. message.
- The README documents the new endpoint.

Core Concept

Not every change to a resource needs to be a full update.

The existing endpoint:

```
PUT /api/tasks/{id}
```

updates editable task fields such as title and description.

The new endpoint:

```
PATCH /api/tasks/{id}/complete
```

changes one specific piece of state:

```
IsCompleted = true
```

This makes the API more expressive because the URL describes the user action directly.

API Shape

After Day 10, the task API includes:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
```

The new endpoint behavior:

- 200 OK when the task exists and is completed
- 404 Not Found when the task does not exist

The not-found response remains:

```
Task not found.
```

Service Method

The application service contract gains:

```
Task<TaskResponse> CompleteTask(int id);
```

The nullable return type matters.

It means the service can return either:

- an updated TaskResponse
- null when the task is missing

The controller can then translate null into 404 Not Found.

Completion Logic

The in-memory implementation is intentionally small:

```
public async Task<TaskResponse?> CompleteTask(int id)
{
    await Task.Yield();

    var task = _tasks.Find(x => x.Id == id);

    if (task is null)
    {
        return null;
    }

    task.IsCompleted = true;

    return task;
}
```

This code follows the current architecture:

- the service handles task behavior
- the controller handles HTTP responses
- storage remains in memory for now

Controller Endpoint

The controller adds:

```
[HttpPatch("{id}/complete")]
```

```
public async Task<IActionResult> Complete(int id)
{
    var response = await _taskService.CompleteTask(id);

    if (response is null)
    {
        return NotFound("Task not found.");
    }

    return Ok(response);
}
```

This keeps the API behavior clear:

- call the service
- return 404 if missing
- return 200 with the updated task if successful

Why This Matters

Day 10 moves the API from generic CRUD into task workflow behavior.

CRUD endpoints are useful, but real applications often need actions that describe what the user is trying to do. Completing a task is a good example. The user is not trying to replace the whole task. The user is performing a domain action.

This is why a focused endpoint can be cleaner than forcing every state change through PUT.

The lesson also reinforces discipline. The endpoint is useful, but the project still avoids adding too much at once. There is no reopen endpoint, no User API, no SQL Server, and no Entity Framework Core today.

Lesson Explanation

The task API already has complete CRUD behavior. A user can create tasks, read tasks, update tasks, and delete tasks. Day 10 adds the first small workflow action on top of that foundation.

The action is completing a task.

A simple way to do this would be to use the existing update endpoint and send a full update request. But that makes the client responsible for changing fields it may not care about. It also hides the meaning of the action inside a generic update call.

A more expressive endpoint is:

```
PATCH /api/tasks/{id}/complete
```

This URL tells the reader exactly what will happen. It will complete the task with the given id.

The service method returns a nullable response:

```
Task<TaskResponse?> CompleteTask(int id);
```

This matches the same pattern already used by read and update operations. If the task exists, the service returns the updated task. If the task does not exist, the service returns null.

The controller converts that service result into HTTP behavior:

```
updated task -> 200 OK
null         -> 404 Not Found
```

The response message also stays consistent with the previous cleanup:

```
Task not found.
```

This is the important part of the lesson: adding a new endpoint should not break the API habits already established. New behavior should fit into the existing style.

Reader Exercise

Think about a resource in one of your own projects.

Find one action that is more specific than a full update.

Examples:

- mark an order as paid
- archive a note
- complete a task
- approve a request
- cancel a booking

Design one focused endpoint for that action and write its expected status codes.

Future Improvement

The API can later add the opposite workflow:

```
PATCH /api/tasks/{id}/reopen
```

That should wait until a future lesson so the project continues to grow in small, clear steps.

Later, persistence will also change the implementation. The external endpoint can stay the same while the internal storage moves from an in-memory list to a database.

Next Lesson

The task API now supports both CRUD and a workflow action.

The next lesson can continue improving the task module or prepare the project for a more durable architecture.

Day 11 - Reopen Task Endpoint

Lesson Goal

Add the opposite workflow to the complete task endpoint.

Day 10 added:

```
PATCH /api/tasks/{id}/complete
```

Day 11 adds:

```
PATCH /api/tasks/{id}/reopen
```

The goal is to let a completed task move back into an active state without using the full update endpoint.

What Changed

Day 11 adds reopen behavior to the task API.

Important files:

- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Api/Controllers/TasksController.cs
- README.md

The main changes are:

- ITaskService now exposes a ReopenTask method.
- TaskService can find a task and set IsCompleted to false.
- TasksController exposes PATCH /api/tasks/{id}/reopen.
- Missing tasks still return the consistent Task not found. message.
- The README documents the new endpoint.

Core Concept

Completion status is now a small workflow instead of a one-way flag.

The API has two explicit actions:

```
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

This keeps the intent clear. A client does not need to send a full task update just to change whether the task is completed.

API Shape

After Day 11, the task API includes:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The task status workflow is now:

```
active -> complete -> reopen
```

Service Method

The application service contract now includes:

```
Task<TaskResponse?> ReopenTask(int id);
```

The nullable return type means the service may not find a task for the supplied id.

Reopen Logic

The reopen operation follows the same pattern as complete:

```
var task = _tasks.FirstOrDefault(task => task.Id == id);

if (task is null)
{
    return null;
}
```

```
task.IsCompleted = false;

return MapToResponse(task);
```

The important part is that reopen changes only one field:

```
IsCompleted = false
```

This keeps the action focused.

Controller Endpoint

The controller exposes the workflow as:

```
[HttpPatch("{id}/reopen")]
public async Task<IActionResult> ReopenTask(int id)
{
    var task = await _taskService.ReopenTask(id);

    if (task is null)
    {
        return NotFound("Task not found.");
    }

    return Ok(task);
}
```

The endpoint returns:

- 200 OK with the updated task when the task exists
- 404 Not Found when the task does not exist

Why This Matters

This is a small feature, but it improves the API design.

Without a reopen endpoint, a client would need to call the general update endpoint and send a request body just to set one boolean value. That works, but it makes the client responsible for more details than necessary.

With a reopen endpoint, the API describes the business action directly.

```
PATCH /api/tasks/5/reopen
```

That is easier to read, easier to test, and easier to explain.

Lesson Explanation

At this point, the task API has moved beyond basic CRUD.

CRUD gives us create, read, update, and delete operations. Real applications also need workflow actions. Completing a task is one workflow action. Reopening a task is another.

The important design choice is to avoid hiding every action inside a generic update endpoint. When an operation has a clear meaning in the product, it can deserve its own endpoint.

The reopen endpoint does not edit the title, description, or any other task field. It only changes the completion state. That makes the endpoint predictable and keeps the controller method small.

Reader Exercise

Try the endpoint in Swagger:

```
PATCH /api/tasks/{id}/reopen
```

Then verify:

- Reopening an existing completed task returns 200 OK.
- The returned task has `isCompleted` set to `false`.
- Reopening a missing task returns 404 Not Found.
- The error message remains Task not found.

Future Improvement

Right now, tasks are still stored in memory.

When persistence is added later, the same service method should update the task in the database instead of only changing the in-memory list.

The API shape should stay stable even when the storage implementation changes.

Next Lesson

Day 11 completes the basic task completion workflow.

The next step should move the project closer to real backend architecture by improving persistence, separating concerns further, or preparing the application layer for database-backed behavior.

Day 12 - Task Completion Filtering

Lesson Goal

Add optional filtering to the task list endpoint.

Before Day 12, this endpoint returned all tasks:

```
GET /api/tasks
```

Day 12 keeps that behavior, but also allows the client to filter by completion status:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

What Changed

Day 12 adds query parameter support to the task API.

Important files:

- `src/JobTrackr.Application/Tasks/ITaskService.cs`
- `src/JobTrackr.Application/Tasks/TaskService.cs`
- `src/JobTrackr.Api/Controllers/TasksController.cs`
- `README.md`

The main changes are:

- `ITaskService.GetAll` now accepts `bool? isCompleted`.
- `TaskService.GetAll` filters tasks when a value is provided.
- `TasksController.GetAll` accepts the optional query parameter.
- Swagger can show and test the optional `isCompleted` query parameter.

- The README documents the new filtering examples.

Core Concept

Query parameters are useful when the endpoint still represents the same resource collection, but the client wants a filtered view.

The resource is still:

```
/api/tasks
```

The filter is:

```
?isCompleted=true
```

This is cleaner than creating separate endpoints such as:

```
GET /api/tasks/completed
GET /api/tasks/incomplete
```

The API keeps one list endpoint and lets the query string describe the requested subset.

API Shape

After Day 12, the task API includes:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The three list behaviors are:

- no query parameter: return all tasks
- isCompleted=true: return completed tasks
- isCompleted=false: return active/incomplete tasks

Nullable Boolean

The service uses:

```
bool? isCompleted
```

The question mark matters.

bool can only represent:

```
true
false
```

bool? can represent:

```
true
false
null
```

That gives the API three states:

- true: completed tasks
- false: incomplete tasks

- null: all tasks

Service Contract

The application service changes from:

```
Task<List<TaskResponse>> GetAll();
```

to:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted);
```

This keeps filtering inside the application service instead of pushing it into the controller.

Filtering Logic

The service method checks whether a filter was provided:

```
if (isCompleted is null)
{
    return _tasks.ToList();
}
```

If no filter exists, all tasks are returned.

When a filter is provided, the service returns only matching tasks:

```
return _tasks
    .Where(task => task.IsCompleted == isCompleted)
    .ToList();
```

The key idea is that the same endpoint can serve both general and filtered list requests.

Controller Endpoint

The controller accepts the optional query parameter:

```
public async Task<IActionResult> GetAll(bool? isCompleted)
{
    var tasks = await _taskService.GetAll(isCompleted);

    return Ok(tasks);
}
```

ASP.NET Core automatically binds `isCompleted` from the query string.

For example:

```
GET /api/tasks?isCompleted=true
```

sets:

```
isCompleted == true
```

Why This Matters

Filtering is one of the first steps from a basic CRUD API toward a useful application API.

Real clients rarely want every record all the time. A task screen may need:

- all tasks
- completed tasks
- incomplete tasks

Instead of making the frontend fetch everything and filter locally, the backend can return the correct subset.

This keeps the API more useful and prepares the project for future database-backed queries.

Lesson Explanation

Day 12 introduces query parameters through a practical example.

The task API already supports completing and reopening tasks. Once tasks can move between active and completed states, users need a way to view those states separately.

The solution is not another command endpoint. Completing and reopening are actions, so they use PATCH. Filtering is a read operation, so it belongs on GET `/api/tasks`.

The optional `isCompleted` parameter gives the list endpoint three behaviors without changing the route. If the parameter is missing, the API returns everything. If it is present, the API returns only matching tasks.

This is a small change in code, but an important API design habit.

Reader Exercise

Try these requests in Swagger:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

Then verify:

- all tasks are returned when the query parameter is missing
- only completed tasks are returned when `isCompleted=true`
- only incomplete tasks are returned when `isCompleted=false`
- completing and reopening a task changes which filtered list it appears in

Future Improvement

The project still uses in-memory storage.

When Entity Framework Core is added, this filtering should move naturally into the database query:

```
query = query.Where(task => task.IsCompleted == isCompleted);
```

The API shape can stay the same while the storage implementation becomes real.

Next Lesson

Day 12 adds the first list filtering behavior.

The next lessons can continue moving toward real backend behavior by adding more query options, persistence, or stronger application-layer boundaries.

Day 13 - Task Search By Title

Lesson Goal

Add basic task search by title to the existing task list endpoint.

Day 12 added completion filtering:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

Day 13 adds title search:

```
GET /api/tasks?search=resume
```

The endpoint should also support filtering and searching together:

```
GET /api/tasks?isCompleted=true&search=resume
```

What Changed

Day 13 extends the existing list endpoint instead of creating a new route.

Important files:

- src/JobTracker.Application/Tasks/ITaskService.cs
- src/JobTracker.Application/Tasks/TaskService.cs
- src/JobTracker.Api/Controllers/TasksController.cs
- README.md

The main changes are:

- ITaskService.GetAll now accepts bool? isCompleted and string? search.
- TaskService.GetAll starts with all tasks and applies optional filters.
- TasksController.GetAll accepts both query parameters.
- Existing completion filtering still works.
- The README documents search examples.

Core Concept

Search is another form of querying a collection.

The resource is still:

```
/api/tasks
```

The query string describes what the client wants:

```
?search=resume
```

This keeps the API focused. The endpoint is not asking for a single task by id. It is asking for a list of tasks that match a title search.

API Shape

After Day 13, the task list endpoint supports:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
```

The full task API now includes:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The route list did not grow, but the list endpoint became more useful.

Service Contract

The service changes from:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted);
```

to:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search);
```

The nullable search value means the client can omit the search parameter.

Query Pipeline

The service starts with all tasks:

```
var query = _tasks.AsEnumerable();
```

Then it applies completion filtering if requested:

```
if (isCompleted is not null)
{
    query = query.Where(task => task.IsCompleted == isCompleted);
}
```

Then it applies title search if requested:

```
if (!string.IsNullOrEmpty(search))
{
    query = query.Where(task =>
        task.Title.Contains(search, StringComparison.OrdinalIgnoreCase));
}
```

Finally, it returns the result:

```
return query.ToList();
```

This creates a simple query pipeline:

```
all tasks -> optional completion filter -> optional title search -> final list
```

Case-Insensitive Search

The search uses:

```
StringComparison.OrdinalIgnoreCase
```

That means a search for:

```
resume
```

can still match:

```
Update Resume
```

This is important because users should not need to match the exact casing of a task title.

Controller Endpoint

The controller accepts both query parameters:

```
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
{
    var tasks = await _taskService.GetAll(isCompleted, search);

    return Ok(tasks);
}
```

ASP.NET Core binds both values from the query string.

Example:

```
GET /api/tasks?isCompleted=true&search=resume
```

sets:

```
isCompleted == true  
search == "resume"
```

Why This Matters

Day 13 teaches how small query options make an API more practical.

A real task list needs more than CRUD. Users need to find tasks quickly. Search by title is a simple first version of that behavior.

This also prepares the project for later database work. In memory, the filtering uses LINQ over a list. With Entity Framework Core later, the same idea can become a database query.

Lesson Explanation

Day 13 builds directly on Day 12.

Day 12 introduced filtering by completion state. Day 13 adds searching by title. Together, these features show how one list endpoint can support multiple query options without becoming messy.

The important implementation detail is to avoid hard-coding every combination. Instead of writing separate branches for every possible query, the service starts with all tasks and then applies each optional condition only when it exists.

This pattern scales better. Later, if the API needs more query options such as priority, due date, or assigned user, the same pipeline style can be extended.

Reader Exercise

Try these requests in Swagger:

```
GET /api/tasks  
GET /api/tasks?search=resume  
GET /api/tasks?isCompleted=true  
GET /api/tasks?isCompleted=false  
GET /api/tasks?isCompleted=true&search=resume
```

Then verify:

- search returns matching titles
- search is case-insensitive
- empty or missing search returns normal list behavior
- completion filtering still works after adding search
- filtering and search can work together

Future Improvement

The current search checks whether the title contains the search text.

Future improvements could include:

- trimming the search value before filtering

- searching description as well as title
- adding pagination
- adding sorting
- moving the query into Entity Framework Core when persistence is added

Next Lesson

Day 13 adds the first search behavior.

The next useful step is to continue shaping the list endpoint into a real production-style query API with pagination, sorting, or persistence-backed querying.

Day 14 - Review Task API

Lesson Goal

Review, test, and stabilize the completed task API before starting the next backend area.

Day 14 does not add a new endpoint.

The goal is to confirm that the current task API works as expected:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Why This Day Matters

Not every productive development day needs a new feature.

After several days of adding endpoints and query behavior, the project needs a checkpoint. Review days prevent the codebase from becoming a pile of partially tested features.

Day 14 closes the current task API phase before moving into the next module.

What To Review

Important files:

- src/JobTrackr.Api/Controllers/TasksController.cs
- src/JobTrackr.Application/Tasks/ITaskService.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- README.md
- Day/PROGRESS.md

The review should confirm:

- the project builds successfully

- Swagger shows the expected endpoints
- the README matches the current API
- all task endpoints behave consistently
- GitHub has the latest clean code

Current Task API

The task API now supports the full in-memory task workflow:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The list endpoint also supports optional query behavior:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
```

This is no longer only a basic CRUD endpoint. It now has simple workflow and query behavior.

Build Check

Run the build from the repository folder:

```
dotnet build
```

Expected result:

```
0 warnings
0 errors
```

The build check matters because it gives a fast signal that the solution still compiles after the recent endpoint changes.

Create Task Test

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Review task API",
  "description": "Day 14 endpoint testing"
}
```

Expected result:

```
201 Created
```

The response should include the created task data, and the endpoint should still behave consistently with the earlier API response improvements.

Get All Tasks Test

Request:

```
GET /api/tasks
```

Expected result:

200 OK

This should return the full task list.

Get Task By Id Test

Request:

```
GET /api/tasks/1
```

Expected result:

200 OK

Missing task request:

```
GET /api/tasks/999
```

Expected result:

404 Not Found

Expected message:

Task not found.

Update Task Test

Request:

```
PUT /api/tasks/1
Content-Type: application/json

{
  "title": "Updated review task",
  "description": "Updated from Day 14"
}
```

Expected result:

200 OK

Invalid title request:

```
PUT /api/tasks/1
Content-Type: application/json

{
  "title": "",
  "description": "Invalid"
}
```

Expected result:

400 Bad Request

Expected message:

Task title is required.

Complete And Reopen Tests

Complete task:

```
PATCH /api/tasks/1/complete
```

Expected result:

200 OK

The returned task should have:


```
{
  "isCompleted": true
}
```

Reopen task:

```
PATCH /api/tasks/1/reopen
```

Expected result:

```
200 OK
```

The returned task should have:

```
{
  "isCompleted": false
}
```

Filtering Tests

Completed tasks:

```
GET /api/tasks?isCompleted=true
```

Incomplete tasks:

```
GET /api/tasks?isCompleted=false
```

Expected result:

```
200 OK
```

The completed filter should return only completed tasks. The incomplete filter should return only active tasks.

Search Tests

Search by title:

```
GET /api/tasks?search=review
```

Search with completion filter:

```
GET /api/tasks?isCompleted=false&search=review
```

Expected result:

```
200 OK
```

Search should be case-insensitive and should continue working with completion filtering.

Lesson Explanation

Day 14 is a checkpoint day.

This is an important habit in backend development. After adding several features, it is easy to keep moving forward without checking whether the earlier pieces still work together. A review day slows the project down just enough to protect quality.

The task API now includes CRUD operations, workflow endpoints, filtering, and search. Before adding persistence or a new module, the current behavior should be tested from the outside through Swagger or HTTP requests.

This day also teaches that documentation is part of the product. If the README does not match the API, future readers and users will be confused even if the code works.

Reader Exercise

Use Swagger or an HTTP client to test:

- create task
- get all tasks
- get task by id
- update task
- delete task
- complete task
- reopen task
- filter completed tasks
- filter incomplete tasks
- search tasks by title
- combine search and completion filtering

For each request, write down:

- request URL
- expected status code
- actual status code
- response body
- any mismatch found

Future Improvement

This kind of manual review is useful, but it should not stay manual forever.

Future improvements should include automated tests for:

- task creation
- validation failures
- missing task behavior
- complete and reopen workflow
- filtering
- search

Automated tests will make later refactoring safer when the project moves from in-memory storage to a real database.

Next Lesson

Day 14 closes the first task API phase.

The next phase can move toward more realistic backend work: persistence, repositories, database queries, or automated tests.

Day 15 - User Service Structure

Lesson Goal

Start the next backend phase by preparing the User API application layer.

Day 15 does not add a database and does not need to add a controller.

The goal is to create the first user service structure:

```
request DTO -> response DTO -> service interface -> service implementation
```

Why This Is Day 15

The task API is stable enough for the current in-memory phase.

It already supports:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

After reviewing that work on Day 14, Day 15 starts a new module: users.

What Changed

Day 15 adds the user application-layer files.

Important files:

- src/JobTracker.Application/Users/CreateUserRequest.cs
- src/JobTracker.Application/Users/UserResponse.cs
- src/JobTracker.Application/Users/IUserService.cs
- src/JobTracker.Application/Users/UserService.cs

The main changes are:

- A Users folder now exists in the Application project.
- CreateUserRequest defines the input for creating a user.
- UserResponse defines the output returned from user operations.
- IUserService defines the user service contract.
- UserService stores users in memory and validates required fields.

Core Concept

Before exposing an API endpoint, build the application-layer shape.

That means creating:

- the input model

- the output model
- the service contract
- the service implementation

This keeps the controller simple when it is added later.

Folder Structure

The new user files live here:

```
src/JobTrackr.Application/Users
```

This keeps user behavior separate from task behavior.

The project now has a clearer application layout:

```
Application/  
  Tasks/  
  Users/
```

That separation matters as the project grows.

CreateUserRequest

The request DTO receives the data needed to create a user:

```
public class CreateUserRequest  
{  
    public string FullName { get; set; } = string.Empty;  
  
    public string Email { get; set; } = string.Empty;  
}
```

This model does not include Id.

The client should send the user details, but the application should generate the identifier.

UserResponse

The response DTO returns user data from the application:

```
public class UserResponse  
{  
    public int Id { get; set; }  
  
    public string FullName { get; set; } = string.Empty;  
  
    public string Email { get; set; } = string.Empty;  
  
    public DateTime CreatedAtUtc { get; set; }  
}
```

The response includes:

- generated Id
- FullName
- Email
- creation timestamp

This separates what the client sends from what the API returns.

IUserService

The service interface defines the first user actions:

```
public interface IUserService
{
    Task<UserResponse> CreateUser(CreateUserRequest request);

    Task<List<UserResponse>> GetAll();
}
```

This follows the same application service style already used for tasks.

The interface gives the future controller a clear dependency.

UserService

The service implementation stores users in memory for now:

```
private readonly List<UserResponse> _users = [];
private int _nextId = 1;
```

This is not permanent persistence. It is a temporary implementation while the project continues to build structure.

The create method validates required fields:

```
if (string.IsNullOrEmpty(request.FullName))
{
    throw new ArgumentException("User full name is required.");
}

if (string.IsNullOrEmpty(request.Email))
{
    throw new ArgumentException("User email is required.");
}
```

Then it creates a response:

```
UserResponse response = new UserResponse()
{
    Id = _nextId,
    FullName = request.FullName,
    Email = request.Email,
    CreatedAtUtc = DateTime.UtcNow
};
```

Finally, it stores and returns the user:

```
_nextId += 1;
_users.Add(response);

return response;
```

Why No Controller Yet

Day 15 intentionally stops at the application layer.

That is a useful discipline. The controller should come after the request, response, interface, and service are ready.

This avoids building an endpoint first and then forcing the rest of the structure around it.

Lesson Explanation

Day 15 starts the user module without adding an HTTP endpoint.

This is an important shift. Earlier days focused heavily on the task API surface. Today focuses on application structure. Before users can be exposed through a controller, the application needs to know what a user creation request looks like, what a user response looks like, and what user operations exist.

The request DTO and response DTO keep input and output separate. The service interface defines the behavior. The service implementation gives the project a working in-memory version that can later be replaced or extended when persistence is added.

This is how a backend grows without everything collapsing into controllers.

Reader Exercise

Open the new user application files and answer:

- What data does `CreateUserRequest` accept?
- What extra data does `UserResponse` return?
- Why does the request model not include `Id`?
- What methods does `IUserService` expose?
- What validation does `UserService` perform?
- Why is user data still stored in memory?

Then run:

```
dotnet build
```

Expected:

```
0 warnings
0 errors
```

Future Improvement

The next steps can add:

- dependency injection registration
- `UsersController`
- `POST /api/users`
- `GET /api/users`
- better validation
- email format checks
- database persistence

The service structure created today makes those steps easier.

Next Lesson

Day 15 creates the User application layer.

The next lesson can expose that work through API endpoints by registering `UserService` and adding a `UsersController`.

Day 16 - First User API Endpoints

Lesson Goal

Expose the User application layer through the first User API endpoints.

Day 15 created the user service structure. Day 16 connects that structure to HTTP.

The new endpoints are:

```
GET /api/users  
POST /api/users
```

Why This Is Day 16

The user module already has:

- CreateUserRequest
- UserResponse
- IUserService
- UserService

Those files live in the Application layer. Day 16 adds the API layer entry point.

This follows the same direction used by the task feature:

```
Controller -> Service -> Response DTO
```

What Changed

Day 16 adds the first user controller and registers the user service.

Important files:

- src/JobTrackr.Api/Controllers/UsersController.cs
- src/JobTrackr.Api/Program.cs
- README.md

The main changes are:

- UserService is registered with dependency injection.
- UsersController is created.
- GET /api/users returns all users.
- POST /api/users creates a user.
- Invalid full name returns 400 Bad Request.
- Invalid email returns 400 Bad Request.
- README documents the new user endpoints.

Dependency Injection

The API project needs to know how to provide IUserService.

In Program.cs, the user service is registered:

```
builder.Services.AddSingleton<IUserService, UserService>();
```

This allows ASP.NET Core to inject IUserService into UsersController.

The application now has service registrations for both task behavior and user behavior.

UsersController

The controller is created in:

```
src/JobTrackr.Api/Controllers/UsersController.cs
```

It uses the normal controller route pattern:

```
[Route("api/[controller]")]
[ApiController]
public class UsersController : ControllerBase
```

Because the controller is named UsersController, the route becomes:

```
/api/users
```

Get All Users

The first endpoint returns all users:

```
[HttpGet]
public async Task<IActionResult> GetAll()
{
    var users = await _userService.GetAll();

    return Ok(users);
}
```

Request:

```
GET /api/users
```

Expected response:

```
200 OK
```

This matches the simple list behavior already used in the task API.

Create User

The second endpoint creates a user:

```
[HttpPost]
public async Task<IActionResult> Create(CreateUserRequest request)
{
    try
    {
        var response = await _userService.CreateUser(request);

        return Ok(response);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Request:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Expected response:

```
200 OK
```


For now, the endpoint returns 200 OK. A future improvement can change this to 201 Created after adding GET /api/users/{id}.

Validation Behavior

The service already validates required user fields.

Invalid full name:

```
{
  "fullName": "",
  "email": "test@example.com"
}
```

Expected response:

```
400 Bad Request
```

Invalid email:

```
{
  "fullName": "Test User",
  "email": ""
}
```

Expected response:

```
400 Bad Request
```

The controller catches `ArgumentException` and returns the message as a bad request.

API Shape

After Day 16, the project has task endpoints and user endpoints.

Task endpoints:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

User endpoints:

```
GET /api/users
POST /api/users
```

This is the first step toward a broader backend, not only a task feature.

Lesson Explanation

Day 16 turns the user application service into real HTTP endpoints.

This is where the separation between layers becomes visible. The controller does not store users itself. It does not decide how users are created. It receives HTTP requests and delegates the work to `IUserService`.

The service remains responsible for user creation and validation. The controller is responsible for translating that result into HTTP responses.

This is a small example of Clean Architecture direction: the API layer depends on the Application layer, while the Application layer does not depend on the API layer.

Reader Exercise

Open Swagger and test:

```
GET /api/users
POST /api/users
```

Create a valid user:

```
{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Then test invalid requests:

```
{
  "fullName": "",
  "email": "test@example.com"
}

{
  "fullName": "Test User",
  "email": ""
}
```

Verify:

- valid create returns 200 OK
- invalid full name returns 400 Bad Request
- invalid email returns 400 Bad Request
- GET /api/users returns created users

Future Improvement

The current create endpoint returns 200 OK.

Later, the API can improve this by adding:

```
GET /api/users/{id}
```

Then POST /api/users can return:

```
201 Created
```

with a location header pointing to the created user.

Other future improvements:

- email format validation
- duplicate email checks
- user update endpoint
- user delete endpoint
- database persistence
- automated tests

Next Lesson

Day 16 exposes the first user endpoints.

The next lesson can improve the user API by adding GET /api/users/{id} or improving create behavior to return 201 Created.

Day 17 - Get User By ID Endpoint

Lesson Goal

Add the endpoint to get one user by id.

Day 16 added:

```
GET /api/users
POST /api/users
```

Day 17 adds:

```
GET /api/users/{id}
```

The endpoint should return the user when it exists and 404 Not Found when it does not.

Why This Is Day 17

The User API now has create and list behavior.

The next normal CRUD step is reading one user by identifier. This follows the same pattern already used in the Task API.

What Changed

Day 17 adds single-user lookup behavior.

Important files:

- src/JobTrackr.Application/Users/IUserService.cs
- src/JobTrackr.Application/Users/UserService.cs
- src/JobTrackr.Api/Controllers/UsersController.cs
- README.md

The main changes are:

- IUserService now exposes GetById.
- UserService can search the in-memory user list by id.
- UsersController exposes GET /api/users/{id}.
- Missing users return 404 Not Found.
- README documents the new endpoint.

Service Contract

The user service interface adds:

```
Task<UserResponse?> GetById(int id);
```

The nullable return type matters.

UserResponse? means:

- return UserResponse when the user exists
- return null when no user is found

This lets the controller translate application behavior into HTTP behavior.

Service Implementation

The service searches the in-memory list:

```
public async Task<UserResponse?> GetById(int id)
{
    await Task.Yield();

    return _users.Find(user => user.Id == id);
}
```

This is simple for now because users are still stored in memory.

Later, when persistence is added, this same method can query the database instead.

Controller Endpoint

The controller adds:

```
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
    var response = await _userService.GetById(id);

    if (response is null)
    {
        return NotFound("User not found.");
    }

    return Ok(response);
}
```

The route is:

```
GET /api/users/{id}
```

Example:

```
GET /api/users/1
```

API Behavior

When the user exists:

```
200 OK
```

When the user does not exist:

```
404 Not Found
```

Expected missing-user message:

```
User not found.
```

This keeps the behavior clear and consistent with the task API pattern.

API Shape

After Day 17, the User API includes:

```
GET /api/users
POST /api/users
GET /api/users/{id}
```

The project now has:

- Task CRUD and workflow endpoints

- User create endpoint
- User list endpoint
- User get-by-id endpoint

Lesson Explanation

Day 17 adds the next basic read operation for users.

Creating users and listing all users is useful, but most APIs also need to fetch one item by id. The service method returns a nullable response because finding a user is not guaranteed. The controller then decides how to represent that result in HTTP.

If the user exists, the controller returns 200 OK. If the service returns null, the controller returns 404 Not Found.

This separation is important. The service describes application behavior. The controller describes HTTP behavior.

Reader Exercise

Create a user:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Then fetch the user:

```
GET /api/users/1
```

Then test a missing user:

```
GET /api/users/999
```

Verify:

- existing user returns 200 OK
- missing user returns 404 Not Found
- missing-user message is User not found.
- GET /api/users still returns the user list
- POST /api/users still creates users

Future Improvement

Now that GET /api/users/{id} exists, POST /api/users can later be improved to return:

```
201 Created
```

with a location header pointing to:

```
GET /api/users/{id}
```

Other future improvements:

- update user endpoint
- delete user endpoint

- duplicate email validation
- email format validation
- database persistence
- automated user API tests

Next Lesson

Day 17 adds single-user lookup.

The next lesson can improve user creation behavior or continue building the user CRUD flow with update and delete endpoints.

Day 18 - Update User Endpoint

Lesson Goal

Add the endpoint to update an existing user.

Day 18 adds:

```
PUT /api/users/{id}
```

This endpoint should:

- update user full name
- update user email
- return 200 OK when the user exists
- return 404 Not Found when the user does not exist
- return 400 Bad Request when full name or email is empty

Why This Is Day 18

The User API already supports:

```
GET /api/users
POST /api/users
GET /api/users/{id}
```

The next normal CRUD step is updating a user.

This continues the same pattern already used by the Task API: add the request model, update the service contract, implement the service logic, then expose it through the controller.

What Changed

Important files:

- src/JobTracker.Application/Users/UpdateUserRequest.cs
- src/JobTracker.Application/Users/IUserService.cs
- src/JobTracker.Application/Users/UserService.cs
- src/JobTracker.Api/Controllers/UsersController.cs
- README.md

The main changes are:

- UpdateUserRequest was added.
- IUserService now exposes UpdateUser.
- UserService can update an existing user.
- UsersController exposes PUT /api/users/{id}.
- Missing users return User not found.
- Invalid full name or email returns 400 Bad Request.

UpdateUserRequest

The update request contains the editable user fields:

```
public class UpdateUserRequest
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;
}
```

The request does not include Id.

The id comes from the URL:

```
PUT /api/users/{id}
```

This keeps route identity separate from request body data.

Service Contract

The user service interface adds:

```
Task<UserResponse?> UpdateUser(int id, UpdateUserRequest request);
```

The nullable response means:

- return the updated user when found
- return null when the user does not exist

Validation errors are represented with ArgumentException, matching the current user service style.

Service Implementation

The service first finds the user:

```
var user = _users.Find(user => user.Id == id);

if (user is null)
{
    return null;
}
```

Then it validates input:

```
if (string.IsNullOrWhiteSpace(request.FullName))
{
    throw new ArgumentException("User full name is required.");
}

if (string.IsNullOrWhiteSpace(request.Email))
{
    throw new ArgumentException("User email is required.");
}
```

Then it updates the user:

```
user.FullName = request.FullName;
user.Email = request.Email;

return user;
```

This keeps update behavior in the Application layer.

Controller Endpoint

The controller exposes:

```
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, UpdateUserRequest request)
{
    try
    {
        var response = await _userService.UpdateUser(id, request);

        if (response is null)
        {
            return NotFound("User not found.");
        }

        return Ok(response);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(ex.Message);
    }
}
```

The controller handles three outcomes:

- updated user -> 200 OK
- missing user -> 404 Not Found
- invalid request -> 400 Bad Request

API Behavior

Valid update:

```
PUT /api/users/1
Content-Type: application/json

{
  "fullName": "Updated User",
  "email": "updated@example.com"
}
```

Expected:

200 OK

Missing user:

```
PUT /api/users/999
```

Expected:

404 Not Found

Invalid input:

```
{
  "fullName": "",
  "email": "updated@example.com"
}
```

Expected:

400 Bad Request

API Shape

After Day 18, the User API includes:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
```

The project now has a partial User CRUD flow:

- create user
- list users
- get user by id
- update user

Lesson Explanation

Day 18 adds the update operation for users.

This is another standard CRUD step, but it also reinforces an important API design rule: the resource id belongs in the route, while the editable data belongs in the request body.

The service is responsible for finding and updating the user. The controller is responsible for translating service outcomes into HTTP responses. When the service returns `null`, the controller returns `404 Not Found`. When validation fails, the controller returns `400 Bad Request`.

This keeps the responsibilities clear and prevents controller code from becoming the place where all application behavior lives.

Reader Exercise

Create a user:

```
POST /api/users
```

Then update it:

```
PUT /api/users/1
```

Verify:

- valid update returns `200 OK`
- updated full name is returned
- updated email is returned
- missing user returns `404 Not Found`
- empty full name returns `400 Bad Request`
- empty email returns `400 Bad Request`
- `GET /api/users/1` returns the updated user

Future Improvement

The current validation checks only empty values.

Future improvements can add:

- email format validation
- duplicate email validation
- better error response objects
- automated tests for user updates
- persistence-backed user updates

Next Lesson

Day 18 adds update behavior to the User API.

The next logical step is to add delete behavior or improve create/update responses before moving toward persistence.

Day 19 - Delete User Endpoint

Lesson Goal

Add the endpoint to delete an existing user.

Day 19 adds:

```
DELETE /api/users/{id}
```

This endpoint should:

- delete the user when it exists
- return 204 No Content when deletion succeeds
- return 404 Not Found when the user does not exist

Why This Is Day 19

The User API already supports:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
```

The next normal CRUD step is deleting a user.

After Day 19, the basic in-memory User CRUD flow is complete.

What Changed

Important files:

- src/JobTracker.Application/Users/IUserService.cs
- src/JobTracker.Application/Users/UserService.cs
- src/JobTracker.Api/Controllers/UsersController.cs
- README.md

The main changes are:

- IUserService now exposes DeleteUser.

- UserService can remove a user from the in-memory list.
- UsersController exposes DELETE /api/users/{id}.
- Successful deletion returns 204 No Content.
- Missing users return 404 Not Found.
- README documents the new endpoint.

Service Contract

The user service interface adds:

```
Task<bool> DeleteUser(int id);
```

The boolean return value gives the controller a simple result:

- true: user existed and was deleted
- false: user was not found

This is different from GetById and UpdateUser, which return nullable user responses.

Service Implementation

The service finds the user first:

```
var user = _users.Find(user => user.Id == id);
```

If the user does not exist, it returns false:

```
if (user is null)
{
    return false;
}
```

If the user exists, it removes the user:

```
_users.Remove(user);

return true;
```

This keeps deletion behavior inside the Application layer.

Controller Endpoint

The controller exposes:

```
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var isDeleted = await _userService.DeleteUser(id);

    if (!isDeleted)
    {
        return NotFound("User not found.");
    }

    return NoContent();
}
```

The route is:

```
DELETE /api/users/{id}
```

Example:

```
DELETE /api/users/1
```

API Behavior

When deletion succeeds:

```
204 No Content
```

When the user does not exist:

```
404 Not Found
```

Expected missing-user message:

```
User not found.
```

204 No Content is appropriate because the delete action succeeded and there is no response body to return.

API Shape

After Day 19, the User API includes:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

This completes the basic in-memory User CRUD phase.

The project now has:

- task CRUD and workflow endpoints
- task filtering and search
- user CRUD endpoints

Lesson Explanation

Day 19 completes the basic User CRUD flow by adding delete behavior.

The delete operation has a slightly different response shape from create, get, and update. When a user is deleted successfully, the API returns 204 No Content. That tells the client the operation succeeded, but there is no response body.

If the user does not exist, the service returns false, and the controller translates that into 404 Not Found.

This keeps the same separation used in previous lessons: the service handles application behavior, and the controller translates that behavior into HTTP responses.

Reader Exercise

Create a user:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Delete the user:

```
DELETE /api/users/1
```

Then try to fetch the deleted user:

```
GET /api/users/1
```

Verify:

- deleting an existing user returns 204 No Content
- fetching the deleted user returns 404 Not Found
- deleting a missing user returns 404 Not Found
- GET /api/users no longer includes the deleted user

Future Improvement

The User API is still in-memory.

Future improvements should include:

- persistence with a database
- automated tests for delete behavior
- duplicate email validation
- better error response objects
- 201 Created for user creation
- user-task relationship modeling

Next Lesson

Day 19 completes the basic User CRUD phase.

The next phase can focus on cleanup, review, testing, persistence, or connecting users and tasks in the domain model.

Day 20 - Connect Tasks To Users

Lesson Goal

Connect tasks to users.

Day 20 updates task creation so every new task belongs to an existing user.

The key new field is:

```
UserId
```

This starts the relationship between the Task API and the User API.

Why This Is Day 20

Tasks and users currently exist separately.

A real task tracking system needs this relationship:

```
One user can have many tasks.  
One task belongs to one user.
```

For now, this relationship stays in memory. SQL Server and Entity Framework Core come later.

What Changed

Important files:

- src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
- src/JobTrackr.Application/Tasks/TaskResponse.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Api/Program.cs
- README.md

The main changes are:

- CreateTaskRequest now includes UserId.
- TaskResponse now includes UserId.
- TaskService receives IUserService.
- Task creation validates that the user exists.
- Creating a task for a missing user returns an error.
- README documents the new task creation behavior.

CreateTaskRequest

Task creation now requires a user id:

```
public class CreateTaskRequest
{
    public string Title { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;

    public int UserId { get; set; }
}
```

The request now says:

```
Create this task for this user.
```

TaskResponse

The task response also includes UserId:

```
public int UserId { get; set; }
```

This lets the API response show which user owns the task.

That is important for clients because a task without ownership is not useful in a real task tracking system.

Service Dependency

TaskService needs to confirm that the user exists before creating the task.

It receives IUserService through dependency injection:

```
private readonly IUserService _userService;

public TaskService(IUserService userService)
{
    _userService = userService;
}
```

This lets the task service ask the user service for the user.

User Validation

Task creation now validates UserId:

```
if (request.UserId <= 0)
{
    throw new ArgumentException("UserId is required.");
}
```

Then it checks that the user exists:

```
var user = await _userService.GetById(request.UserId);

if (user is null)
{
    throw new ArgumentException("User not found.");
}
```

Only after that should the task be created.

API Behavior

Valid task creation now looks like this:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Prepare resume",
  "description": "Update backend project section",
  "userId": 1
}
```

Expected result:

```
201 Created
```

Missing or invalid user:

```
{
  "title": "Prepare resume",
  "description": "Update backend project section",
  "userId": 999
}
```

Expected result:

```
400 Bad Request
```

Expected message:

```
User not found.
```

Service Registration

Both services must be registered:

```
builder.Services.AddSingleton<IUserService, UserService>();
builder.Services.AddSingleton<ITaskService, TaskService>();
```

TaskService depends on IUserService, so the User service registration must exist.

Keeping user registration before task registration is clearer for readers.

API Shape

After Day 20, task creation is no longer isolated from users.

User API:

```
GET /api/users
```

```
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Task API:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

The important Day 20 change is that `POST /api/tasks` now requires a valid user.

Lesson Explanation

Day 20 is the first relationship day.

Until now, tasks and users were separate pieces of the backend. That is useful for learning CRUD, but it is not realistic for a task tracking system. A task should belong to a user.

The project is still in memory, but the design starts moving toward the real database relationship that will exist later. `CreateTaskRequest` now includes `UserId`, and `TaskService` validates that the referenced user exists before creating the task.

This is a small change, but it introduces an important backend idea: one feature may need to depend on another feature's application service to enforce a business rule.

Reader Exercise

Create a user:

```
POST /api/users
Content-Type: application/json

{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Then create a task for that user:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Prepare resume",
  "description": "Update backend project section",
  "userId": 1
}
```

Then try creating a task for a missing user:

```
{
  "title": "Invalid task",
  "description": "This user does not exist",
  "userId": 999
}
```

Verify:

- creating a task for an existing user succeeds
- the response includes `userId`
- creating a task with `userId` less than or equal to zero returns an error

- creating a task for a missing user returns `User not found`.

Future Improvement

This relationship is still in memory.

Future improvements should include:

- database-backed users and tasks
- real foreign key relationship
- user-specific task queries
- GET `/api/users/{id}/tasks`
- stronger validation
- automated tests for task-user ownership

Next Lesson

Day 20 connects tasks to users.

The next phase can continue improving relationships, add user-specific task endpoints, or start preparing for persistence with Entity Framework Core.

Day 21 - Review User And Task APIs

Lesson Goal

Review and stabilize the User and Task APIs after connecting tasks to users.

Day 21 does not add a new feature.

The goal is to confirm:

- users can be created
- tasks can be created for existing users
- tasks cannot be created for missing users
- Task CRUD still works
- User CRUD still works
- the project is ready before moving toward Infrastructure, EF Core, and SQL Server

Why This Is Day 21

Day 20 connected tasks to users with `UserId`.

Before adding database work, the in-memory behavior should be correct. This review day checks that the relationship works and that older endpoints were not broken by the new user-task connection.

What To Review

Important files:

- `src/JobTracker.Api/Controllers/TasksController.cs`

- src/JobTrackr.Api/Controllers/UsersController.cs
- src/JobTrackr.Application/Tasks/TaskService.cs
- src/JobTrackr.Application/Users/UserService.cs
- src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
- src/JobTrackr.Application/Tasks/TaskResponse.cs
- README.md

The review should confirm:

- the project builds successfully
- Swagger shows the expected endpoints
- task creation requires a valid user
- the README matches the current API behavior
- GitHub has the latest clean code

Current API Shape

Task endpoints:

```
GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Task query behavior:

```
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true&search=resume
```

User endpoints:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 warnings
0 errors
```

This confirms the project still compiles after the relationship work from Day 20.

User Create Test

Request:

```
POST /api/users
Content-Type: application/json
```

```
{
  "fullName": "Task Owner",
  "email": "owner@example.com"
}
```

Expected:

200 OK

Check that the response includes:

- id
- fullName
- email

Get User Tests

Get all users:

GET /api/users

Expected:

200 OK

Get one user:

GET /api/users/1

Expected:

200 OK

Missing user:

GET /api/users/999

Expected:

404 Not Found

Expected message:

User not found.

Create Task For Existing User

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "User-owned task",
  "description": "Task belongs to user 1",
  "userId": 1
}
```

Expected:

201 Created

Check:

- response includes userId
- userId is 1

Create Task For Missing User

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Invalid task",
  "description": "User does not exist",
  "userId": 999
}
```

Expected:

```
400 Bad Request
```

Expected message:

```
User not found.
```

Invalid Task UserId

Request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Invalid user id",
  "description": "UserId is invalid",
  "userId": 0
}
```

Expected:

```
400 Bad Request
```

Expected message:

```
UserId is required.
```

Task Workflow Tests

After creating a valid task, verify:

```
GET /api/tasks
GET /api/tasks/1
PATCH /api/tasks/1/complete
PATCH /api/tasks/1/reopen
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=owned
```

These tests confirm that connecting tasks to users did not break the existing task workflow.

User CRUD Tests

Verify:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

These tests confirm that User CRUD still works after task creation began depending on users.

Lesson Explanation

Day 21 is a checkpoint day.

The project now has two connected areas: users and tasks. That makes review more important. A task can no longer be created in isolation; it must belong to a valid user. This relationship is still in memory, but it introduces the same kind of rule that will later be enforced by a database relationship.

The purpose of the review is to confirm that the new rule works without breaking existing behavior. Users should still be creatable, readable, updateable, and deleteable. Tasks should still support CRUD, complete/reopen, filtering, and search. Task creation should now reject invalid users.

This is the right point to stabilize before moving toward Infrastructure and persistence.

Reader Exercise

Test the API in this order:

1. Create a user.
2. Get all users.
3. Get the created user by id.
4. Create a task for that user.
5. Create a task for a missing user and confirm it fails.
6. Complete and reopen the valid task.
7. Search and filter tasks.
8. Update the user.
9. Delete the user.

For each request, write down:

- request URL
- request body if any
- expected status code
- actual status code
- response body

Future Improvement

Manual review is useful, but this flow should eventually become automated tests.

Future improvements should include:

- API tests for user CRUD
- API tests for task CRUD
- tests for task-user ownership
- tests for invalid UserId
- persistence with SQL Server
- EF Core relationships between users and tasks

Next Lesson

Day 21 stabilizes the in-memory User and Task APIs.

The next phase can begin moving toward Infrastructure, EF Core, SQL Server, and real persistence.

Day 22 - Prepare Infrastructure Layer

Lesson Goal

Prepare the Infrastructure layer for future database work.

Day 22 does not add EF Core.

Day 22 does not add SQL Server.

The goal is to review the project structure and make sure the solution is ready before persistence is introduced.

Why This Is Day 22

So far, data is stored in memory inside services.

The next major backend phase is persistence:

Infrastructure -> EF Core -> SQL Server

Before adding packages and database code, the project structure should be understandable and clean.

Current Architecture

The solution currently has these layers:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
```

Expected responsibilities:

```
Api           -> controllers, Program.cs, Swagger, HTTP behavior
Application   -> DTOs, service interfaces, use-case logic
Domain        -> business entities
Infrastructure -> database and external system code later
```

What To Review

Important files and folders:

- src/JobTrackr.Infrastructure/JobTrackr.Infrastructure.csproj
- src/JobTrackr.Infrastructure
- src/JobTrackr.Api/JobTrackr.Api.csproj
- src/JobTrackr.Application/JobTrackr.Application.csproj
- README.md

The review should confirm:

- Infrastructure project exists

- project references are understandable
- unused starter files are removed if present
- README explains Infrastructure responsibility clearly
- project builds successfully

Infrastructure Responsibility

The Infrastructure layer is reserved for code that talks to outside systems.

In this project, that will mainly mean database code later:

- EF Core DbContext
- database configuration
- persistence logic
- repository implementation if needed
- SQL Server integration

The important point is that database concerns should not be placed directly in controllers.

What Not To Do Today

Day 22 has strict boundaries.

Do not add:

- EF Core packages
- SQL Server configuration
- ApplicationDbContext
- migrations
- repositories
- database connection strings

This day is about preparation, not persistence implementation.

Project References

The Infrastructure project should be reviewed for references.

Infrastructure will later need to understand:

- Application contracts
- Domain entities

The API currently uses the Application layer for controllers and service registration.

Later, the API may also reference Infrastructure when database-backed services are registered.

For today, the goal is to understand the references and avoid unnecessary changes.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 warnings
0 errors
```

This confirms the solution is still healthy before persistence work begins.

Git Check

Run:

```
git status
```

If files changed, commit them.

If no files changed:

```
No commit needed.
```

Review days may or may not create commits. The important thing is that the working tree ends clean.

Lesson Explanation

Day 22 starts the persistence preparation phase.

The project already has working in-memory User and Task APIs. Tasks are connected to users through `UserId`, and the current APIs have been reviewed. That means the next major step is moving from in-memory storage to real persistence.

Before adding EF Core or SQL Server, the Infrastructure layer should be reviewed. In Clean Architecture, Infrastructure is the place for database and external system code. Controllers should not directly own database logic, and Application services should not be tightly coupled to SQL Server details.

This day is intentionally conservative. It protects the project from jumping into database work before the structure is ready.

Reader Exercise

Open the solution and answer:

- Does `JobTracker.Infrastructure` exist?
- Does it contain unused starter files?
- What project references does Infrastructure currently have?
- What project references does API currently have?
- Does README explain the role of Infrastructure?
- Does the project build successfully?
- Is git status clean?

Then explain the layer responsibility map in your own words:

```
Api
Application
Domain
Infrastructure
```

Future Improvement

The next phase can add persistence gradually:

- install EF Core packages
- create ApplicationDbContext
- configure SQL Server
- add database entities or mappings
- add migrations
- move in-memory data to database-backed storage

The key is to do this step by step without mixing database code into controllers.

Next Lesson

Day 22 prepares the Infrastructure layer.

The next lesson can begin the actual persistence work by adding EF Core packages or creating the first database context.

Day 23 - Add EF Core Database Context

Lesson Goal

Add the first Entity Framework Core foundation to the Infrastructure layer.

Day 23 does not move the application fully to SQL Server.

The goal is to prepare the project for persistence by adding EF Core packages, creating ApplicationDbContext, and confirming the solution still builds.

Why This Is Day 23

Until now, JobTrackr stores users and tasks in memory.

That is useful for learning API behavior, but it is not permanent storage. When the API stops, the data disappears.

The next backend phase is persistence:

In-memory services -> EF Core DbContext -> SQL Server

Before replacing service logic, the project needs a database context inside the Infrastructure layer.

Files Changed

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Infrastructure/JobTrackr.Infrastructure.csproj
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
```

What Was Added

The Infrastructure project received EF Core package references:

```
Microsoft.EntityFrameworkCore
Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Design
```

The API project now references Infrastructure so the API can later register database services from that layer.

A new persistence folder was created:

```
src/JobTracker.Infrastructure/Persistence
```

Inside that folder, the project now has:

```
AppDbContext.cs
```

AppDbContext

AppDbContext is the EF Core class that represents the database session for the application.

It exposes tables through DbSet properties:

```
public DbSet<User> Users { get; set; }  
public DbSet<JobTask> Tasks { get; set; }
```

This does not create the database by itself.

It defines the shape EF Core will use when the project later connects to SQL Server and adds migrations.

Architecture Decision

Database code belongs in Infrastructure.

The API project should not own EF Core models or database setup directly. The API can call Infrastructure during startup, but the database implementation should stay behind the Infrastructure boundary.

This keeps the Clean Architecture direction clear:

```
API -> Application -> Domain  
API -> Infrastructure  
Infrastructure -> Domain
```

The Domain layer still contains entities. Infrastructure uses those entities to map database tables.

What Not To Change Yet

Do not remove the in-memory services on Day 23.

Do not rewrite all controllers.

Do not add migrations yet unless the lesson specifically reaches that step.

Do not configure every database setting before the persistence flow is ready.

This lesson is only the database foundation.

Build Check

After adding EF Core and AppDbContext, the project should build successfully:

```
dotnet build
```

Expected result:

```
0 errors
```

Existing Swagger endpoints should still work the same as before because the runtime behavior has not been moved to the database yet.

Git Check

The related commit is:

```
8811257 Day 23: Add EF Core database context
```

Files changed in that commit:

```
src/JobTrackr.Api/JobTrackr.Api.csproj  
src/JobTrackr.Infrastructure/JobTrackr.Infrastructure.csproj  
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
```

Lesson Explanation

Day 23 is the first real step toward permanent storage.

The application already has user and task APIs, but the data still lives in memory. That means the API can demonstrate behavior, but it cannot yet behave like a real backend system.

Entity Framework Core gives the project a bridge between C# entities and SQL Server tables.

The important part of this lesson is restraint. The project does not need to replace every service in one step. A cleaner approach is to first install the correct EF Core packages, create the database context, confirm entity names, and make sure the solution still builds.

This keeps the learning path controlled. One lesson adds the foundation. Later lessons can add SQL Server configuration, migrations, repositories, and then replace in-memory behavior one feature at a time.

Reader Exercise

Open `AppDbContext.cs` and identify which domain entities are being prepared for database storage.

Then answer:

- Why should `AppDbContext` live in `Infrastructure`?
- Why is it better not to remove the in-memory services in the same lesson?
- What will EF Core need next before it can create SQL Server tables?

Future Improvement

The next persistence steps should include:

- registering `AppDbContext` in `Program.cs`
- adding a SQL Server connection string
- creating the first EF Core migration
- applying the migration to the database
- replacing in-memory services with database-backed behavior

Next Lesson

Day 23 prepares the database context.

The next lesson can connect the context to SQL Server and start turning the persistence foundation into a working database-backed backend.

Day 24 - Register SQL Server DbContext

Lesson Goal

Register `AppDbContext` in the API startup configuration and add a SQL Server connection string.

Day 23 created the EF Core database context.

Day 24 connects that context to the ASP.NET Core application so EF Core can be used by the API later.

Why This Is Day 24

Creating `AppDbContext` is not enough by itself.

ASP.NET Core needs to know how to create it through dependency injection.

That setup belongs in `Program.cs`, because `Program.cs` is where the application registers services before the API starts.

The progression is:

```
Day 23 -> create AppDbContext
Day 24 -> register AppDbContext with SQL Server
Later   -> add migrations and database-backed services
```

Files Changed

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/appsettings.json
```

Connection String

`appsettings.json` now contains the SQL Server connection string:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

This tells EF Core where the database will be created and used.

If SQL Server uses a named instance, the server value can be changed later, for example:

```
localhost\SQLEXPRESS
```

Registering AppDbContext

`Program.cs` now imports the Infrastructure persistence namespace and EF Core:

```
using JobTrackr.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
```

Then it registers `AppDbContext`:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

This line tells ASP.NET Core:

- create `AppDbContext` through dependency injection
- use SQL Server as the EF Core provider
- read the connection string from configuration

Important Architecture Point

The application is still using the existing in-memory services.

That is correct for Day 24.

Registering `AppDbContext` does not automatically move reads and writes to SQL Server.

The API will continue to behave the same until the services are changed to use EF Core.

What Not To Do Today

Do not rewrite TaskService.

Do not rewrite UserService.

Do not create migrations yet unless the lesson has reached that point.

Do not try to move all data access in one step.

The purpose of this lesson is configuration only.

Build Check

Run the build from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

Existing Swagger endpoints should still appear:

```
GET /api/tasks  
GET /api/users
```

Git Check

The related commit is:

```
eff9c2a Day 24: Register SQL Server DbContext
```

Files changed in that commit:

```
src/JobTracker.Api/Program.cs  
src/JobTracker.Api/appsettings.json
```

Lesson Explanation

Day 24 connects the EF Core foundation to the running API.

The previous lesson created AppDbContext, but the API still had no startup configuration for it. In ASP.NET Core, services are registered in Program.cs, so this lesson adds the database context to the dependency injection container.

The connection string is stored in appsettings.json under ConnectionStrings. Program.cs reads that value and passes it to EF Core with UseSqlServer.

This is a small change, but it is an important backend step. It separates setup from behavior. The project is now configured for SQL Server, but the actual user and task services still use in-memory storage. That keeps the migration toward persistence controlled and easier to debug.

Reader Exercise

Open Program.cs and find the AddDbContext registration.

Then answer:

- Where does the SQL Server connection string come from?
- Why is AppDbContext registered in the API project?

- Why do the existing services still behave the same after this lesson?

Future Improvement

The next database steps should include:

- adding the first EF Core migration
- creating the database
- checking generated tables
- moving one service from in-memory storage to EF Core
- testing API behavior after persistence is introduced

Next Lesson

Day 24 makes EF Core available to the API.

The next lesson can start using that setup to create migrations and move closer to real SQL Server persistence.

Day 25 - Create First EF Core Migration

Lesson Goal

Create the first Entity Framework Core migration for the JobTrackr backend.

Day 23 created ApplicationDbContext.

Day 24 registered ApplicationDbContext with SQL Server.

Day 25 creates migration files that describe the first database structure.

Why This Is Day 25

EF Core migrations track database structure over time.

A migration is a C# description of what tables, columns, keys, and constraints should exist in the database.

The project is moving through persistence in controlled steps:

Entities -> ApplicationDbContext -> Migration files -> Database update -> EF-backed services

Day 25 reaches the migration files step.

Files Changed

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

EF Core Tooling

Before creating migrations, the EF Core CLI tool must be available:

```
dotnet ef --version
```

If the tool is missing, install the matching .NET 8 version:

```
dotnet tool install --global dotnet-ef --version 8.0.27
```

If it already exists but uses the wrong version, update it:

```
dotnet tool update --global dotnet-ef --version 8.0.27
```

Startup Project Design Package

The API project may need `Microsoft.EntityFrameworkCore.Design` because it is the startup project.

EF Core tooling loads `Program.cs` and configuration from the startup project when creating migrations.

Command:

```
dotnet add src/JobTrackr.Api package Microsoft.EntityFrameworkCore.Design --version 8.0.27
```

This is why `JobTrackr.Api.csproj` is part of the Day 25 changes.

Migration Command

The migration is created from the repository root:

```
dotnet ef migrations add InitialCreate --project src/JobTrackr.Infrastructure --startup-project src/JobTrackr.Api --output-dir Persistence/Migrations
```

Meaning:

```
InitialCreate
```

The migration name.

```
--project src/JobTrackr.Infrastructure
```

Migration files belong to the Infrastructure project.

```
--startup-project src/JobTrackr.Api
```

The API project contains `Program.cs` and `appsettings.json`.

```
--output-dir Persistence/Migrations
```

Migration files are stored under the persistence area of Infrastructure.

Expected Migration Output

The migration folder should contain files like:

```
20260607060440_InitialCreate.cs
20260607060440_InitialCreate.Designer.cs
AppDbContextModelSnapshot.cs
```

The important file to review first is:

```
InitialCreate.cs
```

It should contain `migrationBuilder.CreateTable` calls for the current entities.

Expected Tables

Based on the current project, EF Core should prepare tables for:

```
Users
Tasks
```

Expected Users fields:

```
Id
FullName
Email
CreatedAtUtc
```

Expected Tasks fields:

```
Id
Title
Description
IsCompleted
CreatedAtUtc
UserId
```

Use the actual entity properties in the codebase. Do not add random columns just to make the database look bigger.

What Not To Do Today

Do not rewrite TaskService.

Do not rewrite UserService.

Do not move the API from memory to SQL Server yet.

Do not update the database unless the lesson plan explicitly reaches that step.

Day 25 creates migration files only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

If the project builds, the migration files are valid C# and the solution remains consistent.

Git Check

The related commit is:

```
cc3b907 Day 25: Add initial database migration
```

Files changed in that commit:

```
src/JobTrackr.Api/JobTrackr.Api.csproj
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260607060440_InitialCreate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

Lesson Explanation

Day 25 creates the first database migration.

At this point, the application has domain entities, an EF Core AppDbContext, and SQL Server registration in Program.cs. The next step is to ask EF Core to inspect the model and generate a migration.

The migration is not the database itself. It is a versioned set of instructions that tells EF Core what database structure should be created.

For JobTrackr, the first migration prepares the users and tasks tables based on the current domain entities. This is why reviewing the generated migration matters. It shows whether EF Core understood the current model correctly.

The key discipline in this lesson is to stop after creating and reviewing the migration. The application services still use memory. Database-backed behavior will come later after the database structure is confirmed.

Reader Exercise

Open the generated `InitialCreate.cs` migration file.

Find the `CreateTable` calls and answer:

- Which tables will EF Core create?
- Which columns belong to Users?
- Which columns belong to Tasks?
- Is `UserId` present on tasks?
- Why are migrations stored in Infrastructure instead of Domain?

Future Improvement

The next persistence steps should include:

- running `dotnet ef database update`
- confirming the database exists in SQL Server
- checking generated tables
- deciding which service should move to EF Core first
- testing API behavior after persistence is introduced

Next Lesson

Day 25 creates the migration plan.

The next lesson can apply that plan to SQL Server and create the actual database tables.

Day 26 - Apply EF Core Migration

Lesson Goal

Apply the first EF Core migration to SQL Server.

Day 25 created the migration files.

Day 26 uses those migration files to create the real database and tables.

Why This Is Day 26

A migration file is only a database change plan.

It does not create tables until it is applied to a database.

The persistence flow is:

Entities -> AppDbContext -> Migration files -> SQL Server database

Day 26 moves from migration files to an actual SQL Server database.

Expected Result

After applying the migration, SQL Server should contain:

```
JobTrackrDb
```

Inside that database, the expected tables are:

```
dbo.Users
dbo.Tasks
dbo.__EFMigrationsHistory
```

__EFMigrationsHistory is created by EF Core. It records which migrations have already been applied to the database.

Files To Check

```
src/JobTrackr.Api/appsettings.json
src/JobTrackr.Infrastructure/Persistence/Migrations
```

The connection string in appsettings.json should point to the SQL Server instance:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

If SQL Server uses a named instance, use that server name instead:

```
localhost\SQLEXPRESS
```

Migration Command

Run the database update command from the repository root:

```
dotnet ef database update --project src/JobTrackr.Infrastructure --startup-project src/
JobTrackr.Api
```

Meaning:

```
dotnet ef database update
```

Applies pending migrations to the database.

```
--project src/JobTrackr.Infrastructure
```

Tells EF Core where the migration files are stored.

```
--startup-project src/JobTrackr.Api
```

Tells EF Core where to load Program.cs, dependency injection setup, and appsettings.json.

SQL Server Verification

After the command succeeds, open SQL Server Management Studio and connect to the same server from the connection string.

Check:

```
Databases
  JobTrackrDb
    Tables
      dbo.Users
```

```
dbo.Tasks
dbo.__EFMigrationsHistory
```

Optional SQL check:

```
SELECT name
FROM sys.tables;
```

Expected result:

```
Users
Tasks
__EFMigrationsHistory
```

Migration history check:

```
SELECT *
FROM __EFMigrationsHistory;
```

Expected migration:

```
InitialCreate
```

Important Architecture Point

The API still uses the existing in-memory services.

That is correct for Day 26.

Applying the migration creates the database structure, but it does not automatically move API reads and writes to SQL Server.

The service logic must be changed separately in a later lesson.

What Not To Do Today

Do not rewrite TaskService.

Do not rewrite UserService.

Do not move API data storage to SQL Server yet.

Do not add extra tables manually in SQL Server.

Day 26 only applies the migration and verifies the database.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

If no code files changed during this lesson, no new commit is required.

Lesson Explanation

Day 26 is the point where the database stops being theoretical.

Day 25 generated migration files, but those files were only C# instructions. The database still did not exist until EF Core applied the migration.

The `dotnet ef database update` command reads the migrations from Infrastructure, loads application configuration from the API project, connects to SQL Server, and creates the database objects.

After this step, JobTrackrDb should exist with the first Users and Tasks tables. EF Core also creates the `__EFMigrationsHistory` table so it can track which migrations have already been applied.

The important discipline is not to confuse database creation with application persistence. The API can still use in-memory services even after the database exists. Creating the database is one step. Updating the services to use EF Core is a later step.

Reader Exercise

Open SQL Server Management Studio and confirm:

- JobTrackrDb exists
- `dbo.Users` exists
- `dbo.Tasks` exists
- `dbo.__EFMigrationsHistory` exists
- `InitialCreate` appears in the migration history table

Then answer:

- Why does EF Core need the startup project?
- Why is `__EFMigrationsHistory` important?
- Why does the API still use memory after the database is created?

Future Improvement

The next persistence steps should include:

- choosing which service to convert first
- replacing one in-memory service with EF Core queries
- saving new records to SQL Server
- reading records from SQL Server
- testing API behavior after database-backed persistence is introduced

Next Lesson

Day 26 creates the real SQL Server database.

The next lesson can begin connecting application behavior to that database.

Day 27 - Move User Storage To SQL Server

Lesson Goal

Move user storage from memory to SQL Server using EF Core.

Day 26 created the real database.

Day 27 makes the User API use that database for user records.

Why This Is Day 27

Until now, users were stored in an in-memory list inside UserService.

That allowed the API to work during early development, but data disappeared when the API restarted.

Now that SQL Server exists, the user workflow can move to permanent storage.

Before Day 27:

```
UserController -> IUserService -> in-memory list
```

After Day 27:

```
UserController -> IUserService -> ApplicationDbContext -> SQL Server
```

Files Changed

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Users/UserService.cs
src/JobTrackr.Infrastructure/Users/UserService.cs
```

The important change is that the active UserService implementation now belongs in Infrastructure because it uses ApplicationDbContext.

Architecture Decision

IUserService, request DTOs, and response DTOs stay in Application.

The database-backed implementation moves to Infrastructure.

This keeps the dependency direction clear:

```
Application -> service contracts and DTOs
Infrastructure -> database-backed implementation
API -> dependency injection registration
```

Application should not reference Infrastructure.

Infrastructure can reference Application contracts and Domain entities.

Database-Backed UserService

The new UserService receives ApplicationDbContext through dependency injection:

```
private readonly ApplicationDbContext _dbContext;

public UserService(ApplicationDbContext dbContext)
{
    _dbContext = dbContext;
}
```

This allows the service to read and write users through EF Core.

User Operations

After Day 27, the User API should use SQL Server for:

```
POST /api/users
GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Expected behavior:

- creating a user saves it in SQL Server

- listing users reads from SQL Server
- getting one user reads from SQL Server
- updating a user changes the database record
- deleting a user removes the database record

The API routes and response shapes should stay the same.

Program.cs Change

The API dependency injection setup must register the Infrastructure implementation of IUserService.

That means Program.cs should use the database-backed service, not the old in-memory service.

Task storage should remain unchanged for now.

What Not To Do Today

Do not move task storage to SQL Server.

Do not create repositories yet.

Do not redesign the full architecture.

Do not change controller routes.

Day 27 only moves user storage to the database.

Testing Checklist

Use Swagger or HTTP requests to test:

```
POST /api/users
GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Then confirm in SQL Server that the Users table reflects the changes.

Useful SQL check:

```
SELECT *
FROM Users;
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
30f7f97 Day 27: Move user storage to database
```

Files changed in that commit:

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Users/UserService.cs
```

```
src/JobTrackr.Infrastructure/Users/UserService.cs
```

Lesson Explanation

Day 27 is the first lesson where the API starts using SQL Server for real application behavior.

Earlier lessons created the infrastructure for persistence: `AppDbContext`, the connection string, the first migration, and the database. But the API still used in-memory services.

In this lesson, the user workflow moves to EF Core. The `IUserService` contract remains in `Application`, but the implementation that talks to the database belongs in `Infrastructure`.

This is an important Clean Architecture step. The controller does not need to know whether users are stored in memory or SQL Server. It only depends on `IUserService`. The implementation can change behind that interface.

The result is a more realistic backend: user data can survive API restarts because it is stored in SQL Server.

Reader Exercise

Create a user through Swagger.

Then:

- stop the API
- start the API again
- call `GET /api/users`
- confirm the user still exists

That proves the user is no longer stored only in memory.

Future Improvement

The next backend steps can include:

- moving task storage to SQL Server
- connecting tasks to database users
- adding repository classes if the service grows too large
- improving validation
- adding tests around database-backed behavior

Next Lesson

Day 27 moves users to SQL Server.

The next logical step is to move task storage to SQL Server while preserving the existing task API behavior.

Day 28 - Move Task Storage To SQL Server

Lesson Goal

Move task storage from memory to SQL Server using EF Core.

Day 27 moved users to SQL Server.

Day 28 moves tasks to SQL Server so both main resources are now persistent.

Why This Is Day 28

Until now, tasks were stored in an in-memory list inside TaskService.

That made the task API usable, but task data disappeared when the API restarted.

After users moved to SQL Server, tasks need the same persistence treatment.

Before Day 28:

```
TasksController -> ITaskService -> in-memory list
```

After Day 28:

```
TasksController -> ITaskService -> ApplicationDbContext -> SQL Server
```

Files Changed

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Application/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

The active TaskService implementation now belongs in Infrastructure because it uses ApplicationDbContext.

Architecture Decision

ITaskService, request DTOs, and response DTOs stay in Application.

The EF Core implementation belongs in Infrastructure.

This keeps the dependency direction clear:

```
Application -> service contracts and DTOs
Infrastructure -> database-backed implementation
API -> dependency injection registration
```

Application should not depend on Infrastructure.

Infrastructure can depend on Application contracts and Domain entities.

Database-Backed TaskService

The new TaskService receives ApplicationDbContext through dependency injection:

```
private readonly ApplicationDbContext _dbContext;

public TaskService(ApplicationDbContext dbContext)
{
    _dbContext = dbContext;
}
```

The service uses EF Core to read and write JobTask records.

Task Operations

After Day 28, the Task API should use SQL Server for:

```
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```


Existing behavior should remain the same from the API user's perspective.

The storage layer changes, not the public routes.

Search And Filtering

Task search and completion filtering should continue to work after moving to SQL Server.

Examples:

```
GET /api/tasks?search=resume
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
```

The database-backed service builds an EF Core query and applies filters before returning results.

User Validation

When creating a task, the service should validate that UserId belongs to an existing user:

```
Create task -> check Users table -> save task
```

This keeps tasks connected to real users.

Program.cs Change

The API dependency injection setup must register the Infrastructure implementation of `ITaskService`.

That means `Program.cs` should use the database-backed task service instead of the old in-memory implementation.

What Not To Do Today

Do not add repositories.

Do not add authentication.

Do not redesign the full architecture.

Do not change controller routes.

Day 28 only moves task storage to SQL Server.

Testing Checklist

Use Swagger or HTTP requests to test:

```
POST /api/users
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
DELETE /api/tasks/{id}
GET /api/tasks?search=example
GET /api/tasks?isCompleted=true
```

Then confirm in SQL Server that the `Tasks` table reflects the changes.

Useful SQL check:

```
SELECT *
FROM Tasks;
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
7cdd1ba Day 28: Move task storage to database
```

Files changed in that commit:

```
src/JobTrackr.Api/Program.cs  
src/JobTrackr.Application/Tasks/TaskService.cs  
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 28 completes the first major persistence migration for the core JobTrackr resources.

Day 27 moved users from memory to SQL Server. Day 28 applies the same idea to tasks. The controller still depends on `ITaskService`, but the implementation behind that interface now uses `AppDbContext`.

This is an important architecture lesson. The API route design does not need to change just because the storage mechanism changes. The external behavior can remain stable while the internal implementation becomes more realistic.

The task service now saves, reads, updates, completes, reopens, searches, filters, and deletes tasks through SQL Server. It also checks that a user exists before creating a task for that user.

At this point, both users and tasks are backed by SQL Server. The project is no longer just an in-memory API demo. It has crossed into real backend persistence.

Reader Exercise

Create a user.

Create a task for that user.

Then:

- stop the API
- start the API again
- call `GET /api/tasks`
- confirm the task still exists
- complete the task
- confirm the database value changes

That proves task storage has moved from memory to SQL Server.

Future Improvement

The next backend steps can include:

- improving validation
- adding database constraints
- adding repository classes if services become too large
- adding automated tests around EF Core behavior
- introducing authentication and ownership rules later

Next Lesson

Day 28 moves tasks to SQL Server.

The next lessons can review the database-backed API, clean up persistence behavior, and prepare the project for more realistic backend features.

Day 29 - Test Database Persistence

Lesson Goal

Confirm that user and task data is persisted in SQL Server.

Day 27 moved users to SQL Server.

Day 28 moved tasks to SQL Server.

Day 29 proves that the data survives after the API restarts.

Why This Is Day 29

Earlier in the project, data was stored in memory.

Memory storage disappears when the API stops.

Now users and tasks are stored in SQL Server, so the correct test is:

```
Create data -> restart API -> data should still exist
```

If the same user and task still exist after restarting the API, persistence is working.

Files To Check

No new file is required for Day 29.

Check these files only if something fails:

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/appsettings.json
src/JobTrackr.Infrastructure/Users/UserService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger using the URL shown in the terminal.

Create A User

Use:

```
POST /api/users
```

Request:

```
{
  "fullName": "Day 29 User",
  "email": "day29@example.com"
}
```

Expected result:

```
200 OK
```

Save the returned user id.

Confirm The User From The API

Use:

```
GET /api/users
GET /api/users/{id}
```

The created user should appear in both checks.

Create A Task For The User

Use:

```
POST /api/tasks
```

Request:

```
{
  "title": "Day 29 persistence test",
  "description": "Confirm task remains after API restart",
  "userId": 1
}
```

Use the real user id returned when the user was created.

Expected result:

```
201 Created
```

Save the returned task id.

Confirm The Task From The API

Use:

```
GET /api/tasks
GET /api/tasks/{id}
```

The created task should appear in both checks.

Confirm Rows In SQL Server

Open SQL Server Management Studio and select:

```
JobTrackrDb
```

Check users:

```
SELECT *  
FROM Users;
```

Check tasks:

```
SELECT *  
FROM Tasks;
```

The Day 29 user and task should both appear.

Restart Test

Stop the API.

Start it again:

```
dotnet run --project src/JobTrackr.Api
```

Then call:

```
GET /api/users  
GET /api/tasks
```

Expected result:

The same user and task should still exist.

That confirms SQL Server persistence is working.

What Not To Do Today

Do not add new features.

Do not change service logic unless a real bug appears.

Do not add repositories.

Do not add authentication.

Day 29 is a testing and verification day.

Lesson Explanation

Day 29 is where the database-backed API is proven.

Moving services to SQL Server is only useful if data actually survives beyond the running API process. Earlier in the project, users and tasks lived in memory. That meant every restart cleared the data.

The test for persistence is simple and important: create a user, create a task, confirm both records exist in SQL Server, restart the API, and confirm the same records are still available.

This lesson does not need new features. Its value is confidence. It verifies that the work from Day 27 and Day 28 changed the application from a temporary in-memory API into a backend that can store data permanently.

Reader Exercise

Create a user and task through Swagger.

Then:

- check the rows in SQL Server
- stop the API
- restart the API
- call GET /api/users
- call GET /api/tasks

If both records still appear, write down why this proves persistence is working.

Future Improvement

The next backend steps can include:

- cleaning up database-backed service code
- adding stronger validation
- adding automated integration tests
- adding repository classes only if the service code becomes too large
- preparing authentication and ownership rules later

Next Lesson

Day 29 verifies persistence.

The next lesson can improve the database-backed API now that the main user and task data flows are stored in SQL Server.

Day 30 - First Backend Month Review

Lesson Goal

Review the first 30 backend days and confirm the JobTrackr foundation is stable.

Day 30 does not add a new feature.

The goal is review, verification, documentation, and cleanup.

Why This Is Day 30

The first month built the backend foundation.

At this point, JobTrackr has:

- ASP.NET Core Web API
- Application layer DTOs and interfaces
- Domain entities
- Infrastructure EF Core services
- SQL Server persistence
- User CRUD
- Task CRUD
- Task filtering and search
- Task complete and reopen endpoints

Before starting the next 30-day phase, the foundation should be checked.

Files To Review

```
README.md
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Infrastructure/Users/UserService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

The first monthly review should not continue if the project does not build.

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger and confirm the API is available.

User Endpoint Review

Test:

```
GET /api/users
POST /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Expected behavior:

- create user works
- get all users works
- get user by id works
- update user works
- delete user works
- missing user returns 404 Not Found
- invalid user input returns 400 Bad Request

Task Endpoint Review

Test:

```
GET /api/tasks
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?search=value
GET /api/tasks?isCompleted=false&search=value
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
PATCH /api/tasks/{id}/complete
```

```
PATCH /api/tasks/{id}/reopen
DELETE /api/tasks/{id}
```

Expected behavior:

- create task works when a valid userId exists
- create task fails when the user does not exist
- get all tasks works
- get task by id works
- update task works
- delete task works
- complete task works
- reopen task works
- search works
- completion filter works
- missing task returns 404 Not Found
- empty title returns 400 Bad Request

SQL Server Persistence Review

Open SQL Server Management Studio and select:

```
JobTrackrDb
```

Check users:

```
SELECT *
FROM Users;
```

Check tasks:

```
SELECT *
FROM Tasks;
```

Expected behavior:

- users created from the API appear in Users
- tasks created from the API appear in Tasks
- data still exists after the API restarts

README Review

The README should clearly mention:

```
ASP.NET Core Web API
.NET 8
SQL Server
Entity Framework Core
Layered structure
User CRUD
Task CRUD
Task search and filtering
Task complete and reopen
Database persistence
```

Keep the README simple and current.

The goal is clarity, not a large document.

What Not To Do Today

Do not add a new feature.

Do not redesign the architecture.

Do not add authentication.

Do not add repositories.

Day 30 is a review checkpoint.

Lesson Explanation

Day 30 is the first monthly checkpoint for the JobTrackr backend.

The first month moved the project from an empty backend idea into a working ASP.NET Core Web API with users, tasks, Clean Architecture style layering, EF Core, SQL Server, and persistent storage.

This kind of review day matters because long projects can drift. A stable foundation should be checked before adding the next layer of complexity.

The review confirms that the project builds, the user API works, the task API works, SQL Server persistence works, and the documentation still matches the current state of the code.

No new feature is required. The value of Day 30 is confidence.

Reader Exercise

Run the API and test the main user and task flows.

Then answer:

- Which endpoints are now complete?
- Which endpoints use SQL Server persistence?
- What behavior should be tested after restarting the API?
- What should the next 30-day phase focus on?

Future Improvement

The next backend phase can focus on:

- stronger validation
- cleaner error handling
- integration tests
- authentication
- authorization
- ownership rules
- better project documentation

Next Lesson

Day 30 closes the first backend month.

The next lesson can begin the second phase with a stable, database-backed API foundation.

MONTH 2

ARCHITECTURE AND AUTHENTICATION

Strengthen boundaries, validation, error handling, and JWT authentication.

Day 31 - Review Database CRUD

Lesson Goal

Start the second backend phase by reviewing the database-backed CRUD system.

Day 31 does not add a new feature.

The goal is to confirm that users, tasks, search, filtering, complete/reopen behavior, and SQL Server persistence are stable before changing the database model again.

Why This Is Day 31

The first 30 days built the foundation.

Days 27 and 28 moved users and tasks from memory to SQL Server.

Day 29 tested basic persistence.

Day 30 reviewed the first month.

Before adding relationship features or changing the model again, the current database-backed system should be verified.

Current state:

```
Controllers -> Application interfaces -> Infrastructure services -> AppDbContext -> SQL Server
```

Files To Review

```
src/JobTrackr.Api/Program.cs
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Users
src/JobTrackr.Application/Tasks
src/JobTrackr.Infrastructure/Users/UserService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

Run The API

Run:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger and confirm the API starts correctly.

User CRUD Review

Test:

```
POST /api/users
GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}
```

Create request:

```
{
  "fullName": "Day 31 User",
  "email": "day31@example.com"
}
```

Update request:

```
{
  "fullName": "Day 31 Updated User",
  "email": "day31.updated@example.com"
}
```

Expected behavior:

- create returns 200 OK
- get all returns users
- get by id returns the user
- update changes full name and email
- delete returns 204 No Content
- missing user returns 404 Not Found

Task CRUD Review

Create or use an existing user first.

Then test:

```
POST /api/tasks
GET /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
```

Create request:

```
{
  "title": "Day 31 task",
  "description": "Review database-backed task CRUD",
  "userId": 1
}
```

Update request:

```
{
  "title": "Day 31 updated task",
  "description": "Updated task description"
}
```

Expected behavior:

- create returns 201 Created
- create fails if userId does not exist
- get all returns tasks
- get by id returns the task
- update changes title and description

- delete returns 204 No Content
- missing task returns 404 Not Found

Complete And Reopen Review

Create a task or use an existing task.

Complete:

```
PATCH /api/tasks/{id}/complete
```

Expected:

```
isCompleted = true
```

Reopen:

```
PATCH /api/tasks/{id}/reopen
```

Expected:

```
isCompleted = false
```

Search And Filter Review

Test:

```
GET /api/tasks
GET /api/tasks?search=Day
GET /api/tasks?isCompleted=true
GET /api/tasks?isCompleted=false
GET /api/tasks?isCompleted=false&search=Day
```

Expected behavior:

- search returns matching task titles
- completed filter returns completed tasks
- not completed filter returns open tasks
- combined search and filter works

Restart Persistence Test

Create one user and one task.

Stop the API:

```
Ctrl + C
```

Restart:

```
dotnet run --project src/JobTrackr.Api
```

Then call:

```
GET /api/users
GET /api/tasks
```

Expected result:

The same user and task should still exist.

Database Review

Open SQL Server Management Studio and check:

```
SELECT *  
FROM Users;
```

```
SELECT *  
FROM Tasks;
```

```
SELECT *  
FROM __EFMigrationsHistory;
```

Expected result:

- users created through the API appear in Users
- tasks created through the API appear in Tasks
- migrations are visible in __EFMigrationsHistory

What Not To Do Today

Do not add a new feature.

Do not add relationships or navigation properties.

Do not create a migration unless a real bug requires it.

Fix only obvious issues.

Day 31 is review and stability only.

Lesson Explanation

Day 31 starts the second backend phase by slowing down and checking the current system.

The project now has database-backed user and task services. Before adding new database relationships or larger features, the existing CRUD behavior needs to be trusted.

This review confirms that the API still builds, the user endpoints work, the task endpoints work, search and filtering still work, complete and reopen still work, data survives an API restart, and EF Core migrations are applied.

The main lesson is that a long backend project should not only move forward by adding features. It also needs checkpoints where the existing behavior is verified before new complexity is introduced.

Reader Exercise

Run through the full Day 31 checklist:

- create a user
- update the user
- create a task for that user
- complete and reopen the task
- search for the task
- filter tasks by completion status
- restart the API
- confirm the data still exists

Then write down any behavior that does not match the expected result.

Future Improvement

The next backend phase can focus on:

- relationship cleanup
- better validation
- more useful error responses
- integration testing
- authentication and ownership rules later

Next Lesson

Day 31 confirms the database-backed CRUD foundation.

The next lesson can safely start improving the model or API behavior because the current system has been reviewed.

Day 32 - Add Task User Relationship

Lesson Goal

Make the task-user relationship explicit in EF Core.

The API already stores `UserId` on tasks.

Day 32 teaches EF Core that one user can have many tasks, and each task belongs to one user.

Why This Is Day 32

The database already has a `UserId` column in the `Tasks` table.

That connects tasks to users at a basic data level.

But the code should also express the relationship clearly:

```
User -> many Tasks
Task -> one User
```

This prepares the project for future user-specific task queries.

Files Changed

```
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

User Entity Change

User now has a collection of tasks:

```
public List<JobTask> Tasks { get; set; } = [];
```

This means one user can have many tasks.

JobTask Entity Change

JobTask keeps UserId as the foreign key:

```
public int UserId { get; set; }
```

It also adds the navigation property:

```
public User? User { get; set; }
```

This tells EF Core that a task belongs to a user.

DbContext Relationship Configuration

AppDbContext now configures the relationship:

```
modelBuilder.Entity<JobTask>()  
    .HasOne(task => task.User)  
    .WithMany(user => user.Tasks)  
    .HasForeignKey(task => task.UserId)  
    .OnDelete(DeleteBehavior.Cascade);
```

Meaning:

```
HasOne(task => task.User)
```

Each task has one user.

```
WithMany(user => user.Tasks)
```

Each user can have many tasks.

```
HasForeignKey(task => task.UserId)
```

The UserId column connects tasks to users.

```
OnDelete(DeleteBehavior.Cascade)
```

If a user is deleted, that user's tasks are deleted too.

Migration

The related migration is:

```
AddTaskUserRelationship
```

EF Core may add:

```
index on Tasks.UserId  
foreign key from Tasks.UserId to Users.Id
```

This makes the relationship explicit in SQL Server, not only in C# code.

Data Check Before Migration

Before applying the foreign key, existing tasks should have valid users.

SQL check:

```
SELECT t.*  
FROM Tasks t  
LEFT JOIN Users u ON t.UserId = u.Id  
WHERE u.Id IS NULL;
```

Expected result:

```
No rows
```


If rows appear, those tasks have invalid `UserId` values and must be fixed before applying the migration.

What Not To Do Today

Do not add a new API endpoint.

Do not add `GET /api/users/{id}/tasks` yet.

Do not change DTOs unless the build requires it.

Do not return navigation properties from API responses.

Day 32 is database model relationship work only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
813823e Day 32: Add task user relationship
```

Files changed in that commit:

```
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Domain/Entities/User.cs
src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260614072653_AddTaskUserRelationship.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
```

Lesson Explanation

Day 32 makes the relationship between users and tasks explicit.

The project already stored `UserId` on tasks, but EF Core relationships are clearer when the entity model includes navigation properties and the `DbContext` configures the relationship.

The `User` entity now has a `Tasks` collection. The `JobTask` entity now has a `User` navigation property. `AppDbContext` connects both sides with `HasOne`, `WithMany`, and `HasForeignKey`.

This makes the model more accurate: one user can own many tasks, and each task belongs to one user.

The migration then updates SQL Server so the relationship exists at the database level through an index and foreign key.

Reader Exercise

Open `User.cs`, `JobTask.cs`, and `AppDbContext.cs`.

Then answer:

- Which property shows that a user can have many tasks?
- Which property shows that a task belongs to one user?

- Which column is the foreign key?
- What does cascade delete mean in this relationship?

Future Improvement

The next backend step can use this relationship to add user-specific task queries, such as fetching all tasks for one user.

That should be added through a clear API endpoint rather than exposing navigation properties directly in response models.

Next Lesson

Day 32 defines the relationship.

The next lesson can use that relationship to build a user tasks endpoint or improve task query behavior.

Day 33 - Get Tasks By User Endpoint

Lesson Goal

Add an endpoint to fetch all tasks for a specific user.

Day 32 made the task-user relationship explicit in EF Core.

Day 33 uses that relationship in the API.

Why This Is Day 33

The database now understands this relationship:

```
User -> many Tasks
Task -> one User
```

The next useful API behavior is:

```
Show me all tasks for one user.
```

The chosen route is:

```
GET /api/users/{userId}/tasks
```

This route is clear because tasks are being requested in the context of a specific user.

Files Changed

```
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Application/Tasks/ITaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

No new migration is required for Day 33 because the database relationship was already handled on Day 32.

Service Interface Change

ITaskService now includes a method for user-specific task lookup:

```
Task<List<TaskResponse>>> GetByUserId(int userId);
```

The nullable list is intentional:

```
null = user does not exist
```

```
empty list = user exists but has no tasks
```

This lets the API return the correct HTTP behavior.

Infrastructure Service Change

The database-backed TaskService implements the new method.

It first checks whether the user exists:

```
var userExists = await _dbContext.Users.AnyAsync(user => user.Id == userId);
```

If the user does not exist, the service returns null.

If the user exists, the service returns tasks filtered by UserId.

Controller Change

UserController now depends on both:

```
IUserService  
ITaskService
```

The new endpoint is added inside UsersController:

```
GET /api/users/{userId}/tasks
```

Expected behavior:

```
User does not exist -> 404 Not Found  
User exists with no tasks -> 200 OK with []  
User exists with tasks -> 200 OK with task list
```

Why This Endpoint Belongs In UsersController

The endpoint asks for tasks in the context of a user.

That makes this route easy to understand:

```
/api/users/{userId}/tasks
```

It reads as:

```
Get tasks for this user.
```

The existing general task endpoints remain unchanged.

What Not To Do Today

Do not add filtering by userId to GET /api/tasks.

Do not add due dates.

Do not add priority.

Do not create a migration.

Day 33 only adds the get-tasks-by-user endpoint.

Testing Checklist

Run the API and open Swagger.

Confirm this endpoint appears:

```
GET /api/users/{userId}/tasks
```

Test user does not exist:

```
GET /api/users/999999/tasks
```

Expected:

```
404 Not Found
```

Test user exists with no tasks:

```
GET /api/users/{userId}/tasks
```

Expected:

```
200 OK  
[]
```

Test user exists with tasks:

```
POST /api/tasks  
GET /api/users/{userId}/tasks
```

Expected:

```
200 OK  
task list for that user
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
43ac224 Day 33: Add get tasks by user endpoint
```

Files changed in that commit:

```
src/JobTrackr.Api/Controllers/UsersController.cs  
src/JobTrackr.Application/Tasks/ITaskService.cs  
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 33 uses the relationship created on Day 32.

The database already knows that users and tasks are connected. Now the API exposes a practical endpoint for that relationship: getting all tasks for a specific user.

The service method returns null when the user does not exist and an empty list when the user exists but has no tasks. This distinction matters because both cases should not produce the same HTTP response.

The controller returns 404 Not Found for a missing user and 200 OK for an existing user. If the user has no tasks, the response is still successful, but the list is empty.

This is a clean backend step: no schema change, no new feature category, and no route redesign. The API simply starts using the relationship that already exists.

Reader Exercise

Create two users.

Create tasks for only one of them.

Then call:

```
GET /api/users/{firstUserId}/tasks
GET /api/users/{secondUserId}/tasks
GET /api/users/999999/tasks
```

Confirm:

- the first user returns tasks
- the second user returns an empty list
- the missing user returns 404 Not Found

Future Improvement

Later lessons can build on this by adding:

- pagination for user tasks
- filtering user tasks by completion status
- searching user tasks by title
- authentication and ownership checks

Next Lesson

Day 33 adds the user-specific task endpoint.

The next lesson can improve query behavior around users and tasks or begin adding more realistic API rules.

Day 34 - Add UserId Filter To Task List

Lesson Goal

Improve GET /api/tasks by adding userId as a query parameter.

Day 33 added:

```
GET /api/users/{userId}/tasks
```

Day 34 adds:

```
GET /api/tasks?userId=1
```

Why This Is Day 34

The API now supports getting tasks from a user context.

That route is useful:

```
GET /api/users/{userId}/tasks
```

But the main task list endpoint should also support filtering.

This lets clients combine query options:

```
GET /api/tasks?userId=1&isCompleted=false&search=resume
```

That is more flexible for task listing screens.

Files Changed

```
README.md
src/JobTracker.Api/Controllers/TasksController.cs
src/JobTracker.Application/Tasks/ITaskService.cs
src/JobTracker.Infrastructure/Tasks/TaskService.cs
```

No migration is required because this is a query behavior change, not a database schema change.

Service Interface Change

ITaskService.GetAll changed from:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search);
```

to:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search, int? userId);
```

The nullable userId means:

```
null = do not filter by user
value = return tasks for that user only
```

Infrastructure Service Change

The database-backed TaskService now applies the user filter when userId is provided:

```
if (userId is not null)
{
    query = query.Where(task => task.UserId == userId);
}
```

The other filters still run:

```
isCompleted
search
```

The result is one EF Core query that can combine multiple conditions.

Controller Change

TasksController.GetAll now accepts:

```
isCompleted
search
userId
```

The endpoint remains:

```
GET /api/tasks
```

But it now supports these examples:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?isCompleted=true
GET /api/tasks?search=resume
GET /api/tasks?userId=1&isCompleted=false&search=resume
```

Why Keep Both Routes

Both routes are useful:

```
GET /api/users/{userId}/tasks
```

Use this when the client is already working from a user page.

```
GET /api/tasks?userId=1
```

Use this when the client is working from a task list page and wants filtering.

The two routes can coexist because they serve different API usage styles.

Testing Checklist

Run the API and open Swagger.

Check:

```
GET /api/tasks
```

Swagger should show:

```
isCompleted
search
userId
```

Test all tasks:

```
GET /api/tasks
```

Test tasks by user:

```
GET /api/tasks?userId=1
```

Test completed tasks:

```
GET /api/tasks?isCompleted=true
```

Test open tasks:

```
GET /api/tasks?isCompleted=false
```

Test search:

```
GET /api/tasks?search=resume
```

Test combined filters:

```
GET /api/tasks?userId=1&isCompleted=false&search=resume
```

What Not To Do Today

Do not add pagination.

Do not add sorting.

Do not add due dates.

Do not add priority.

Do not create a migration.

Day 34 only improves task query filtering.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
d898eb6 Day 34: Add task user filter
```

Files changed in that commit:

```
README.md
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Tasks/ITaskService.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 34 improves task querying without changing the database schema.

Day 33 added a user-specific route: GET /api/users/{userId}/tasks. That route is useful when the API client is working from a user context.

Day 34 makes the general task list more flexible by adding a userId query parameter. Now GET /api/tasks can return all tasks, tasks for one user, completed tasks, searched tasks, or a combination of those filters.

This is a practical API design step. Not every improvement requires a new endpoint or migration. Sometimes the better change is to make an existing query endpoint more useful while keeping its behavior predictable.

Reader Exercise

Create two users.

Create tasks for both users.

Then test:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?userId=2
GET /api/tasks?userId=1&isCompleted=false
```

Confirm each response only returns the expected tasks.

Future Improvement

Later task query improvements can include:

- pagination
- sorting
- due dates
- priority
- user-specific search
- user-specific completion filtering

Next Lesson

Day 34 improves task filtering.

The next lesson can continue strengthening task query behavior or move into the next backend concern now that user-specific task access is available.

Day 35 - Add Optional Due Date To Tasks

Lesson Goal

Add an optional due date field to tasks.

Day 34 improved task query filtering.

Day 35 adds a simple task deadline field:

```
DueDateUtc
```

The field is optional, so tasks can still exist without a due date.

Why This Is Day 35

Tasks often need deadlines.

The first version should stay simple:

```
null = no due date  
date value = task has a due date
```

The property name uses `Utc` because backend systems should be clear about time storage.

Files Changed

```
README.md  
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs  
src/JobTrackr.Application/Tasks/TaskResponse.cs  
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs  
src/JobTrackr.Domain/Entities/JobTask.cs  
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.cs  
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.Designer.cs  
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs  
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

Domain Entity Change

`JobTask` now has:

```
public DateTime? DueDateUtc { get; set; }
```

The nullable `DateTime?` means the field is not required.

Request Model Changes

`CreateTaskRequest` now accepts:

```
public DateTime? DueDateUtc { get; set; }
```

`UpdateTaskRequest` also accepts:

```
public DateTime? DueDateUtc { get; set; }
```

This allows clients to set or remove the due date during create and update operations.

Response Model Change

`TaskResponse` now returns:

```
public DateTime? DueDateUtc { get; set; }
```

This lets API clients see whether a task has a deadline.

TaskService Changes

When creating a task, the service saves the due date:

```
DueDateUtc = request.DueDateUtc,
```

When updating a task, the service updates the due date:

```
task.DueDateUtc = request.DueDateUtc;
```

When returning task data, the service maps the due date into TaskResponse.

This must be done in:

```
CreateTask  
UpdateTask  
MapToResponse  
GetAll projection  
GetByUserId projection
```

Any response path that returns a task should include DueDateUtc.

Migration

The related migration is:

```
AddTaskDueDate
```

Expected schema change:

```
Tasks table -> nullable DueDateUtc column
```

This is a database schema change, so a migration is required.

Testing Checklist

Create a task without a due date:

```
{  
  "title": "Task without due date",  
  "description": "No deadline yet",  
  "userId": 1  
}
```

Expected:

```
DueDateUtc = null
```

Create a task with a due date:

```
{  
  "title": "Task with due date",  
  "description": "Deadline test",  
  "dueDateUtc": "2026-06-30T00:00:00Z",  
  "userId": 1  
}
```

Expected:

```
DueDateUtc has the supplied value
```

Update a task and change the due date.

Then update it again with DueDateUtc as null to confirm the due date can be removed.

What Not To Do Today

Do not add reminders.

Do not add notifications.

Do not add overdue filtering.

Do not add priority.

Day 35 only adds an optional due date field.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet NU1900 timeout warnings can happen if package vulnerability metadata cannot be reached.

Those are not code errors.

Git Check

The related commit is:

```
90fe90c Day 35: Add task due date
```

Files changed in that commit:

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260617071002_AddTaskDueDate.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Services/TaskService.cs
```

Lesson Explanation

Day 35 adds a realistic task field without making the feature too large.

A task often needs a deadline, but not every task has one. That is why `DueDateUtc` is nullable. The API can create tasks with no due date, create tasks with a due date, update the due date, or remove it later.

The change crosses multiple layers. The Domain entity stores the value. The request models accept it. The response model returns it. The Infrastructure service saves and maps it. The EF Core migration adds the nullable column to SQL Server.

This is a useful example of how one simple API field moves through a layered backend.

Reader Exercise

Create two tasks:

- one without `DueDateUtc`
- one with `DueDateUtc`

Then call:

```
GET /api/tasks
GET /api/tasks/{id}
GET /api/users/{userId}/tasks
```

Confirm that each response returns the expected due date value.

Future Improvement

Later lessons can build on due dates with:

- overdue task filtering
- sorting by due date
- reminders
- notifications
- due date validation rules

Next Lesson

Day 35 adds the due date field.

The next lesson can use that field for filtering, sorting, or more realistic task planning behavior.

Day 36 - Add Simple Task Priority

Lesson Goal

Add simple priority support to tasks.

Day 35 added optional due dates.

Day 36 adds a priority field so tasks can be marked as:

```
Low
Medium
High
```

Why This Is Day 36

Tasks are easier to manage when importance is visible.

For the first version, priority is stored as a string:

```
public string Priority { get; set; } = "Medium";
```

This is beginner-friendly and easy to inspect in Swagger and SQL Server.

Later, the project can improve this with enum mapping or stronger validation.

Files Changed

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

The migration timestamp may be different in another copy of the project. That is normal.

Domain Entity Change

JobTask now has:

```
public string Priority { get; set; } = "Medium";
```

Default priority:

```
Medium
```

Request Model Changes

CreateTaskRequest now accepts:

```
public string Priority { get; set; } = "Medium";
```

UpdateTaskRequest also accepts:

```
public string Priority { get; set; } = "Medium";
```

This allows clients to set task priority during create and update operations.

Response Model Change

TaskResponse now returns:

```
public string Priority { get; set; } = string.Empty;
```

This lets API clients display the priority value.

TaskService Changes

When creating a task, the service validates priority:

```
if (string.IsNullOrWhiteSpace(request.Priority))
{
    throw new ArgumentException("Task priority is required.");
}
```

Then it saves:

```
Priority = request.Priority,
```

When updating a task, the service validates and updates:

```
task.Priority = request.Priority;
```

When returning task data, the service maps priority into TaskResponse.

This should be included in:

```
GetAll projection
GetByUserId projection
MapToResponse
```

Migration

The related migration is:

```
AddTaskPriority
```

Expected schema change:

```
Tasks table -> Priority column
```

This is a database schema change, so a migration is required.

Testing Checklist

Create a task with default priority:

```
{
  "title": "Default priority task",
  "description": "Priority should be Medium",
  "userId": 1
}
```

Create a high priority task:

```
{
  "title": "High priority task",
  "description": "Important work",
  "dueDateUtc": "2026-06-30T00:00:00Z",
  "priority": "High",
  "userId": 1
}
```

Update a task to low priority:

```
{
  "title": "Updated low priority task",
  "description": "Lower importance",
  "dueDateUtc": null,
  "priority": "Low"
}
```

Confirm priority appears in:

```
GET /api/tasks
GET /api/tasks/{id}
GET /api/users/{userId}/tasks
```

What Not To Do Today

Do not add priority filtering.

Do not add sorting.

Do not add enum mapping.

Do not add complex validation.

Day 36 only adds a simple priority field.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet NU1900 timeout warnings can happen if package vulnerability metadata cannot be reached.

Those are not code errors.

Git Check

The related commit is:

```
b1b328d Day 36: Add task priority
```

Files changed in that commit:

```
README.md
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/TaskResponse.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Domain/Entities/JobTask.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.Designer.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/20260618083307_AddTaskPriority.cs
src/JobTrackr.Infrastructure/Persistence/Migrations/AppDbContextModelSnapshot.cs
src/JobTrackr.Infrastructure/Tasks/TaskService.cs
```

Lesson Explanation

Day 36 adds task priority in the simplest useful form.

The project already supports task ownership, filtering, due dates, and SQL Server persistence. Priority is another common task field, but this lesson keeps the implementation small by using a string instead of an enum.

The change still moves through every important backend layer. The Domain entity stores priority. The request models accept it. The response model returns it. The Infrastructure service validates, saves, updates, and maps it. The EF Core migration adds the new database column.

This shows how a single field becomes part of a real API feature across a layered backend.

Reader Exercise

Create three tasks:

- one with Low
- one with Medium
- one with High

Then call:

```
GET /api/tasks
GET /api/users/{userId}/tasks
```

Confirm that each response includes the expected priority value.

Future Improvement

Later lessons can improve priority with:

- enum mapping
- stricter validation
- priority filtering
- priority sorting
- better API error messages for invalid values

Next Lesson

Day 36 adds priority.

The next lesson can use due dates and priority for richer task filtering or start improving validation rules.

Day 37 - Review Database Task Changes

Lesson Goal

Review and stabilize the recent database-backed task changes.

Day 37 is not a new feature day.

It is a checkpoint for relationship, due date, priority, filtering, migrations, SQL Server tables, and README accuracy.

Why This Is Day 37

Days 32 to 36 changed the task system several times:

- Task-user relationship
- Get tasks by user endpoint
- UserId task filtering
- Optional due date
- Simple task priority

Before adding validation and error handling work, the current database-backed task system should be verified.

Recent migrations include:

- AddTaskUserRelationship
- AddTaskDueDate
- AddTaskPriority

Files To Review

- src/JobTrackr.Domain/Entities/User.cs
- src/JobTrackr.Domain/Entities/JobTask.cs
- src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
- src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
- src/JobTrackr.Application/Tasks/TaskResponse.cs
- src/JobTrackr.Infrastructure/Tasks/TaskService.cs
- src/JobTrackr.Infrastructure/Persistence/AppDbContext.cs
- src/JobTrackr.Infrastructure/Persistence/Migrations
- src/JobTrackr.Api/Controllers/TasksController.cs
- src/JobTrackr.Api/Controllers/UsersController.cs
- README.md

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet NU1900 warnings can appear if vulnerability metadata times out. Those are network metadata warnings, not code errors.

Migration Review

Run:

```
dotnet ef migrations list --project src/JobTrackr.Infrastructure --startup-project src/JobTrackr.Api
```

Expected migrations:

```
InitialCreate
AddTaskUserRelationship
AddTaskDueDate
AddTaskPriority
```

All should be applied.

SQL Server Review

Open SQL Server Management Studio and use:

```
JobTrackrDb
```

Check users:

```
SELECT TOP 10 *
FROM Users;
```

Check tasks:

```
SELECT TOP 10 Id, Title, UserId, DueDateUtc, Priority, IsCompleted
FROM Tasks;
```

Expected result:

- Users table exists
- Tasks table exists
- Tasks.UserId exists
- Tasks.DueDateUtc exists
- Tasks.Priority exists
- task rows show priority values
- due date can be NULL

Relationship Review

Check the task-user join:

```
SELECT t.Id, t.Title, t.UserId, u.FullName
FROM Tasks t
INNER JOIN Users u ON t.UserId = u.Id;
```

Expected result:

Tasks should join correctly with users.

If a task appears without a valid user, the relationship or existing data needs review.

API Review

Run the API:

```
dotnet run --project src/JobTrackr.Api
```

Open Swagger.

Test creating a task with due date and priority:

```
{
  "title": "Day 37 review task",
  "description": "Testing relationship, due date, and priority",
  "dueDateUtc": "2026-07-01T00:00:00Z",
  "priority": "High",
  "userId": 1
}
```

Expected:

201 Created

Response should include:

```
"dueDateUtc": "2026-07-01T00:00:00Z",
"priority": "High"
```

Test creating a task without due date:

```
{
  "title": "Day 37 no due date task",
  "description": "Testing nullable due date",
  "priority": "Medium",
  "userId": 1
}
```

Expected response should include:

```
"dueDateUtc": null,
"priority": "Medium"
```

Update Review

Use:

```
PUT /api/tasks/{id}
```

Request:

```
{
  "title": "Day 37 updated task",
  "description": "Updated due date and priority",
  "dueDateUtc": null,
  "priority": "Low"
}
```

Expected:

200 OK

The response should show DueDateUtc as null and Priority as Low.

Query Review

Test:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?isCompleted=false
GET /api/tasks?search=Day
GET /api/tasks?userId=1&isCompleted=false&search=Day
GET /api/users/1/tasks
```

Expected result:

All filters and user-specific task queries should still work after the due date and priority changes.

What Not To Do Today

Do not add a new feature.

Do not add another migration unless something is broken.

Do not add priority filtering.

Do not add due date filtering.

Do not add authentication.

Day 37 is review and cleanup only.

Lesson Explanation

Day 37 is a stability checkpoint after several database changes.

The project recently added a task-user relationship, user-specific task endpoints, task filtering by user, optional due dates, and task priority. Each of those changes is useful, but together they increase the chance of small bugs or missing mappings.

A review day protects the project from moving forward on top of unstable behavior. The build should pass, migrations should be applied, SQL Server should contain the expected columns, API responses should include due date and priority, and task filters should still work.

The main lesson is that backend progress is not only about adding features. A serious project also needs verification days where the current system is checked before more complexity is added.

Reader Exercise

Run the Day 37 checklist:

- build the project
- check migrations
- inspect Users and Tasks tables
- create a task with due date and priority
- create a task without due date
- update due date and priority
- test task filters
- test GET /api/users/{userId}/tasks

Write down any behavior that does not match the expected result.

Future Improvement

The next backend phase can focus on:

- stronger validation
- better error responses
- due date filtering
- priority filtering
- automated tests

- authentication later

Next Lesson

Day 37 verifies the database-backed task system.

The next lesson can start validation and error handling work with a stable foundation.

Day 38 - Add Data Annotation Validation

Lesson Goal

Start the validation and error handling phase by adding data annotation validation to request DTOs.

Day 38 moves basic request shape validation closer to the DTOs.

Examples:

```
Title is required
Email must be valid
FullName has a max length
UserId must be greater than 0
```

Why This Is Day 38

Earlier validation lived mostly inside services.

That is acceptable for business rules, but request shape validation should be closer to the request models.

ASP.NET Core can automatically validate request DTOs when controllers use:

```
[ApiController]
```

The project already uses `[ApiController]`, so invalid request models can return `400 Bad Request` automatically.

Files Changed

```
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Application/Users/CreateUserRequest.cs
src/JobTrackr.Application/Users/UpdateUserRequest.cs
```

No migration is required because this is API request validation only.

DataAnnotations Import

Each request DTO uses:

```
using System.ComponentModel.DataAnnotations;
```

This enables attributes such as:

```
Required
EmailAddress
MaxLength
Range
```

User Request Validation

`CreateUserRequest` and `UpdateUserRequest` should validate:

```
FullName is required
```

```
FullName max length is 100
Email is required
Email must be a valid email address
Email max length is 200
```

Expected attributes:

```
[Required]
[MaxLength(100)]
public string FullName { get; set; } = string.Empty;

[Required]
[EmailAddress]
[MaxLength(200)]
public string Email { get; set; } = string.Empty;
```

Task Request Validation

CreateTaskRequest should validate:

```
Title is required
Title max length is 150
Description max length is 1000
Priority is required
Priority max length is 20
UserId must be greater than 0
```

Expected attributes:

```
[Required]
[MaxLength(150)]
public string Title { get; set; } = string.Empty;

[MaxLength(1000)]
public string Description { get; set; } = string.Empty;

[Required]
[MaxLength(20)]
public string Priority { get; set; } = "Medium";

[Range(1, int.MaxValue)]
public int UserId { get; set; }
```

UpdateTaskRequest should validate title, description, and priority in the same way.

It does not need UserId because updating a task does not move it to a different user in this lesson.

Service Validation Still Matters

Service validation should stay for business rules.

Examples:

```
User not found
Task not found
UserId must refer to a real user
```

Data annotations validate request shape.

Services validate business and data rules.

Both layers can exist together.

Bad User Request Test

Use:

```
POST /api/users
```

Request:

```
{
  "fullName": "",
  "email": "not-an-email"
}
```

Expected:

```
400 Bad Request
```

Validation errors should mention:

```
FullName
Email
```

Bad Task Request Test

Use:

```
POST /api/tasks
```

Request:

```
{
  "title": "",
  "description": "Invalid request",
  "priority": "",
  "userId": 0
}
```

Expected:

```
400 Bad Request
```

Validation errors should mention:

```
Title
Priority
UserId
```

Valid Task Request Test

Use a valid request to confirm normal behavior still works:

```
{
  "title": "Day 38 valid task",
  "description": "Testing valid request after data annotations",
  "dueDateUtc": "2026-07-01T00:00:00Z",
  "priority": "Medium",
  "userId": 1
}
```

Expected:

```
201 Created
```

What Not To Do Today

Do not build custom validation middleware.

Do not change the validation response shape.

Do not remove business validation from services unless it is clearly duplicated and safe.

Do not add FluentValidation.

Day 38 uses built-in data annotations only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet NU1900 warnings can appear if vulnerability metadata times out. Those are network metadata warnings, not code errors.

Git Check

The related commit is:

```
49901e0 Day 38: Add data annotation validation
```

Files changed in that commit:

```
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
src/JobTrackr.Application/Users/CreateUserRequest.cs
src/JobTrackr.Application/Users/UpdateUserRequest.cs
```

Lesson Explanation

Day 38 starts the validation phase by adding built-in data annotations to request DTOs.

The project already has working user and task APIs. Now the next step is to make invalid requests fail earlier and more consistently.

Data annotations are a simple ASP.NET Core feature. They let request models describe basic rules such as required fields, email format, max length, and numeric ranges. Because the controllers use [ApiController], ASP.NET Core can automatically return 400 Bad Request when those rules fail.

This keeps basic request shape validation close to the DTOs. Service methods can still keep business validation such as checking whether a user exists.

Reader Exercise

Send one invalid user request and one invalid task request.

Confirm both return:

```
400 Bad Request
```

Then send valid requests and confirm normal API behavior still works.

Future Improvement

Later validation work can include:

- custom error response shape
- stronger priority validation
- due date validation
- reusable validation helpers
- FluentValidation if the project needs more complex rules

Next Lesson

Day 38 adds basic request validation.

The next lesson can continue improving error handling and make API responses more consistent.

Day 39 - Review ASP.NET Core Validation Responses

Lesson Goal

Review the default ASP.NET Core validation response shape.

Day 38 added data annotation validation to request DTOs.

Day 39 checks what ASP.NET Core returns by default when validation fails.

Why This Is Day 39

The controllers use:

```
[ApiController]
```

Because of that, ASP.NET Core can automatically validate request DTOs and return:

```
400 Bad Request
```

Before building custom validation middleware or custom response classes, the default framework behavior should be understood.

For this project stage, the default response is likely good enough if:

```
it returns 400
it shows which fields failed
it gives readable error messages
it does not expose sensitive data
```

Files To Check

No file change is required for Day 39 unless something is broken.

Files to review if validation behaves incorrectly:

```
src/JobTrackr.Api/Controllers/UsersController.cs
src/JobTrackr.Api/Controllers/TasksController.cs
src/JobTrackr.Application/Users/CreateUserRequest.cs
src/JobTrackr.Application/Users/UpdateUserRequest.cs
src/JobTrackr.Application/Tasks/CreateTaskRequest.cs
src/JobTrackr.Application/Tasks/UpdateTaskRequest.cs
```

Build Check

Run from the repository root:

```
dotnet build
```

Expected result:

```
0 errors
```

NuGet NU1900 warnings can appear if package vulnerability metadata times out. Those are network metadata warnings, not code errors.

Run The API

Run:


```
dotnet run --project src/JobTracker.Api
```

Open Swagger and test invalid requests.

Invalid Create User Test

Use:

```
POST /api/users
```

Request:

```
{
  "fullName": "",
  "email": "not-an-email"
}
```

Expected:

```
400 Bad Request
```

The response body should include validation errors for:

```
FullName
Email
```

Invalid Update User Test

Use:

```
PUT /api/users/{id}
```

Request:

```
{
  "fullName": "",
  "email": "wrong-email"
}
```

Expected:

```
400 Bad Request
```

The response should show validation errors for the invalid fields.

Invalid Create Task Test

Use:

```
POST /api/tasks
```

Request:

```
{
  "title": "",
  "description": "Invalid task request",
  "priority": "",
  "userId": 0
}
```

Expected:

```
400 Bad Request
```

The response should include validation errors for:

```
Title
Priority
UserId
```

Invalid Update Task Test

Use:

```
PUT /api/tasks/{id}
```

Request:

```
{
  "title": "",
  "description": "Invalid update",
  "priority": ""
}
```

Expected:

```
400 Bad Request
```

The response should include validation errors for:

```
Title
Priority
```

Valid Request Test

Use a valid task request to confirm normal behavior still works:

```
{
  "title": "Day 39 valid task",
  "description": "Validation response review",
  "dueDateUtc": "2026-07-01T00:00:00Z",
  "priority": "Medium",
  "userId": 1
}
```

Expected:

```
201 Created
```

Decision

Keep the default ASP.NET Core validation response for now.

Reason:

```
It is clear enough.
It is built into ASP.NET Core.
It avoids unnecessary custom middleware.
It lets the project move forward without overengineering.
```

What Not To Do Today

Do not build custom middleware.

Do not create a custom validation response class.

Do not add FluentValidation.

Do not change global exception handling.

Day 39 is review and decision only unless something is clearly broken.

Lesson Explanation

Day 39 reviews what ASP.NET Core already provides for validation errors.

After Day 38, request DTOs contain data annotations such as `Required`, `EmailAddress`, `MaxLength`, and `Range`. Since the controllers use `[ApiController]`, ASP.NET Core automatically checks those rules before controller action logic runs.

When validation fails, the framework returns `400 Bad Request` with a response body that identifies the failed fields and their messages.

The important lesson is not to customize too early. The default response is readable enough for this stage of the project. Custom validation responses can be added later when the API has a stronger reason for them.

Reader Exercise

Send invalid requests for:

- create user
- update user
- create task
- update task

Confirm each returns:

```
400 Bad Request
```

Then inspect the response body and identify which fields failed validation.

Future Improvement

Later validation and error handling work can include:

- custom error response shape
- consistent business error responses
- due date validation
- priority validation
- global exception handling
- `FluentValidation` if rules become complex

Next Lesson

Day 39 confirms the default validation response shape.

The next lesson can improve error handling or continue toward more consistent API responses.

Day 40 - Centralize Common API Messages

Lesson Goal

Centralize obvious repeated API message strings into one shared file.

Day 40 is a small cleanup day.

It does not add global exception handling, custom middleware, or a new error response model.

Why This Is Day 40

The API now has repeated messages in controllers and services.

Examples:

```
User not found.  
Task not found.  
Task title is required.  
UserId is required.
```

Repeated strings are easy to mistype and harder to keep consistent.

The goal is to move common message text into one place without changing API behavior.

Files Changed

```
src/JobTrackr.Application/Common/ErrorMessage.cs  
src/JobTrackr.Api/Controllers/TasksController.cs  
src/JobTrackr.Api/Controllers/UsersController.cs  
src/JobTrackr.Infrastructure/Tasks/TaskService.cs  
src/JobTrackr.Infrastructure/Users/UserService.cs
```

ErrorMessage Class

The new file is:

```
src/JobTrackr.Application/Common/ErrorMessage.cs
```

It stores common message constants:

```
namespace JobTrackr.Application.Common;  
  
public static class ErrorMessage  
{  
    public const string UserNotFound = "User not found.";  
    public const string TaskNotFound = "Task not found.";  
    public const string TaskTitleRequired = "Task title is required.";  
    public const string TaskPriorityRequired = "Task priority is required.";  
    public const string UserIdRequired = "UserId is required.";  
    public const string UserFullNameRequired = "User full name is required.";  
    public const string UserEmailRequired = "User email is required.";  
}
```

Controller Changes

Controllers now use shared messages instead of inline string literals.

Example:

```
return NotFound(ErrorMessage.TaskNotFound);
```

instead of:

```
return NotFound("Task not found.");
```

The controller behavior should stay the same.

Only the source of the message text changes.

Service Changes

Services now use shared validation messages.

Examples:

```
throw new ArgumentException(ErrorMessage.UserIdRequired);  
throw new ArgumentException(ErrorMessage.UserNotFound);  
throw new ArgumentException(ErrorMessage.TaskTitleRequired);
```

```
throw new ArgumentException(ErrorMessages.TaskPriorityRequired);
```

User service validation can also use:

```
throw new ArgumentException(ErrorMessages.UserFullNameRequired);  
throw new ArgumentException(ErrorMessages.UserEmailRequired);
```

What Stays The Same

Day 40 should not change API behavior.

Expected results should remain:

```
missing task -> 404 Not Found -> Task not found.  
missing user -> 404 Not Found -> User not found.  
invalid task title -> 400 Bad Request -> Task title is required.  
invalid user id -> 400 Bad Request -> UserId is required.
```

This is a cleanup, not a feature redesign.

What Not To Do Today

Do not create custom middleware.

Do not add global exception handling.

Do not create a custom error response model.

Do not build a large message abstraction.

Do not redesign the project.

Day 40 only centralizes obvious repeated messages.

Test Examples

Test task not found:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found  
Task not found.
```

Test user not found:

```
GET /api/users/999999
```

Expected:

```
404 Not Found  
User not found.
```

Test create task with missing user:

```
{  
  "title": "Prepare resume",  
  "description": "Update backend project details",  
  "userId": 999999,  
  "dueDateUtc": null,  
  "priority": "Medium"  
}
```

Expected:

```
400 Bad Request  
User not found.
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
6567f1d Day 40: Centralize common messages
```

Lesson Explanation

Day 40 is a small but useful cleanup lesson.

As APIs grow, repeated strings start appearing in controllers and services. At first this is harmless, but over time it becomes harder to keep messages consistent.

Instead of writing "User not found." or "Task not found." in many places, the project now has one `ErrorMessages` class in the Application layer. Controllers and services can reference the same constants.

This keeps message text consistent without changing API behavior. It also avoids overengineering. The project does not need custom middleware or global exception handling yet. It only needs a simple shared place for obvious repeated messages.

Reader Exercise

Find every place where the API returns:

```
User not found.  
Task not found.
```

Confirm those messages now come from `ErrorMessages`.

Then test the endpoints and confirm the HTTP behavior is unchanged.

Future Improvement

Later error handling work can include:

- consistent error response models
- global exception handling
- problem details customization
- business error codes
- localization if the project ever needs it

Next Lesson

Day 40 centralizes common messages.

The next lesson can continue improving error handling without losing the simple behavior already working in the API.

Day 41 - Add Basic Global Exception Handling

Lesson Goal

Add basic global exception handling middleware to the API.

Day 41 prevents unexpected exceptions from leaking technical details to API users.

Validation errors and normal not-found responses should keep their existing behavior.

Why This Is Day 41

The API already has request validation and common error messages.

The next error handling step is to catch unexpected server errors in one place.

Instead of exposing raw exception details, the API should return:

```
500 Internal Server Error
An unexpected error occurred.
```

This is for unexpected server errors only.

Normal validation errors should still return:

```
400 Bad Request
```

Normal missing records should still return:

```
404 Not Found
```

Files Changed

```
src/JobTrackr.Api/Middleware/ExceptionHandlingMiddleware.cs
src/JobTrackr.Api/Program.cs
```

Middleware File

The new middleware file is:

```
src/JobTrackr.Api/Middleware/ExceptionHandlingMiddleware.cs
```

It wraps the request pipeline in a try/catch.

If an unexpected exception happens, it sets:

```
StatusCode = 500
ContentType = text/plain
```

Then it returns:

```
An unexpected error occurred.
```

Middleware Code Shape

The middleware follows this structure:

```
public async Task InvokeAsync(HttpContext context)
{
    try
    {
        await _next(context);
    }
    catch (Exception)
```

```
{
    context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;
    context.Response.ContentType = "text/plain";

    await context.Response.WriteAsync("An unexpected error occurred.");
}
```

Program.cs Registration

Program.cs imports the middleware namespace:

```
using JobTrackr.Api.Middleware;
```

Then it registers the middleware before controller mapping:

```
app.UseMiddleware<ExceptionHandlerMiddleware>();

app.MapControllers();
```

This makes every API request pass through the exception handler.

What Should Stay The Same

Task not found should still return:

```
404 Not Found
Task not found.
```

User not found should still return:

```
404 Not Found
User not found.
```

Invalid request DTOs should still return:

```
400 Bad Request
```

ASP.NET Core validation should continue to handle those validation failures.

What Not To Do Today

Do not create a custom error response model.

Do not add a logging service.

Do not change validation responses.

Do not create custom exception classes.

Do not build a large middleware system.

Day 41 only adds basic global exception handling.

Test Examples

Test normal not found:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found
Task not found.
```

Test validation:

```
POST /api/tasks
```


Invalid body:

```
{
  "title": "",
  "description": "Test task",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

```
400 Bad Request
```

Test unexpected exception only with temporary code.

If a controller action temporarily throws an exception, expected response:

```
500 Internal Server Error
An unexpected error occurred.
```

Remove the temporary throw after testing.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Git Check

The related commit is:

```
e2fc4b4 Day 41: Add basic exception handling middleware
```

Files changed in that commit:

```
src/JobTrackr.Api/Middleware/ExceptionHandlingMiddleware.cs
src/JobTrackr.Api/Program.cs
```

Lesson Explanation

Day 41 adds the first global exception handling middleware.

Validation and not-found cases are already handled by the API. But unexpected exceptions need a safe fallback so technical details are not leaked to users.

The middleware wraps the request pipeline in a try/catch. If the request completes normally, nothing changes. If an unexpected exception is thrown, the middleware returns a simple `500 Internal Server Error` response.

This is intentionally small. The project does not need a full error response system yet. It only needs a safe default for unexpected server failures.

Reader Exercise

Test three cases:

- task not found
- invalid task request
- temporary unexpected exception

Confirm:

```
not found -> 404
validation -> 400
unexpected exception -> 500
```

Then remove any temporary exception test code.

Future Improvement

Later error handling work can include:

- structured error response models
- request logging
- exception logging
- custom exception types
- problem details responses
- environment-specific error details

Next Lesson

Day 41 adds a safe fallback for unexpected errors.

The next lesson can continue improving API error handling while keeping validation and normal not-found behavior stable.

Day 42 - Review HTTP Status Codes

Lesson Goal

Review the API response status codes and fix user creation so it returns the correct status code.

Day 42 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 42

The API now has users, tasks, validation, shared messages, and basic global exception handling.

Before adding more behavior, the existing endpoints should use consistent HTTP status codes.

The expected rules are:

```
successful GET -> 200 OK
successful POST create -> 201 Created
successful PUT update -> 200 OK
successful PATCH -> 200 OK
successful DELETE -> 204 No Content
missing resource -> 404 Not Found
invalid input -> 400 Bad Request
unexpected server error -> 500 Internal Server Error
```

Endpoints Reviewed

Task endpoints:

```
GET /api/tasks
```

```
GET /api/tasks/{id}
POST /api/tasks
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

User endpoints:

```
GET /api/users
GET /api/users/{id}
GET /api/users/{userId}/tasks
POST /api/users
PUT /api/users/{id}
DELETE /api/users/{id}
```

Main Fix

If user creation returns 200 OK, it should be changed to 201 Created.

Expected create response:

```
return CreatedAtAction(nameof(GetById), new { id = response.Id }, response);
```

This makes user creation consistent with task creation.

GET Rules

Successful list request:

```
return Ok(items);
```

Expected:

```
200 OK
```

Successful single item request:

```
return Ok(response);
```

Expected:

```
200 OK
```

Missing item:

```
return NotFound(ErrorMessages.TaskNotFound);
```

Expected:

```
404 Not Found
```

POST Rules

Successful create should return:

```
201 Created
```

This applies to:

```
POST /api/users
POST /api/tasks
```

The response should include the created resource and a location reference when possible.

PUT Rules

Successful update:

```
return Ok(response);
```

Expected:

200 OK

Missing item:

```
return NotFound(ErrorMessages.UserNotFound);
```

Expected:

404 Not Found

Invalid input:

```
return BadRequest(ex.Message);
```

Expected:

400 Bad Request

PATCH Rules

Successful partial update:

```
return Ok(response);
```

Expected:

200 OK

Missing task:

```
return NotFound(ErrorMessages.TaskNotFound);
```

Expected:

404 Not Found

DELETE Rules

Successful delete:

```
return NoContent();
```

Expected:

204 No Content

Missing item:

```
return NotFound(ErrorMessages.UserNotFound);
```

Expected:

404 Not Found

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not add new middleware.

Do not create custom response wrappers.

Do not make database changes.

Day 42 only checks and fixes HTTP status code consistency.

Testing Checklist

Create user:

```
POST /api/users
```

Expected:

```
201 Created
```

Create task:

```
POST /api/tasks
```

Expected:

```
201 Created
```

Get missing user:

```
GET /api/users/999999
```

Expected:

```
404 Not Found
User not found.
```

Get missing task:

```
GET /api/tasks/999999
```

Expected:

```
404 Not Found
Task not found.
```

Delete existing user:

```
DELETE /api/users/{id}
```

Expected:

```
204 No Content
```

Invalid request:

```
POST /api/tasks
```

Expected:

```
400 Bad Request
```

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
5056756 Day 42: Fix user create status code
```

Lesson Explanation

Day 42 reviews one of the most practical parts of API design: HTTP status codes.

Status codes are part of the API contract. A client should be able to understand what happened from the status code before reading the response body.

For create operations, 201 Created is more accurate than 200 OK because the request created a new resource. For deletes, 204 No Content is appropriate because the operation succeeded and there is no response body. Missing resources should return 404 Not Found, validation problems should return 400 Bad Request, and unexpected failures should be handled by the global exception middleware as 500 Internal Server Error.

The main fix in this lesson is user creation. If it returns 200 OK, it should be updated to 201 Created so it matches task creation and follows normal HTTP API behavior.

Reader Exercise

Open the users and tasks controllers.

For each action, write down:

- what happens when the request succeeds
- what happens when the resource is missing
- what happens when the request is invalid

Then confirm the status code matches the expected rule.

Future Improvement

Later API response improvements can include:

- consistent problem details responses
- response documentation
- integration tests for status codes
- better error response models
- automated API contract checks

Next Lesson

Day 42 makes response status codes more consistent.

The next lesson can continue improving API reliability through tests or cleaner error response behavior.

Day 43 - Review Response DTO Consistency

Lesson Goal

Review API response DTO consistency across the users and tasks endpoints.

Day 43 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 43

The API should return response DTOs, not database entities directly.

Expected response models:

```
TaskResponse
UserResponse
```

Types that should not be returned directly from the public API:

```
JobTask
User
EF Core entity objects
```

DTOs protect the API contract. The internal database model can change later without forcing API clients to change.

Files Reviewed

Main areas to review:

```
TasksController
UsersController
TaskService
UserService
ITaskService
IUserService
TaskResponse
UserResponse
```

The goal is to confirm that controllers return service responses and services return DTOs.

Controller Pattern

Controllers should receive response models from services:

```
var response = await _taskService.GetById(id);

if (response is null)
{
    return NotFound(ErrorMessages.TaskNotFound);
}

return Ok(response);
```

The controller should not query EF Core directly and return database entities.

Service Pattern

Services can use EF Core entities internally.

But public service methods should return DTOs:

```
TaskResponse
TaskResponse?
List<TaskResponse>
List<TaskResponse>?
UserResponse
UserResponse?
List<UserResponse>
```

Example mapping:

```
return new TaskResponse
{
    Id = task.Id,
```

```
Title = task.Title,
Description = task.Description,
CreatedAtUtc = task.CreatedAtUtc,
DueDateUtc = task.DueDateUtc,
Priority = task.Priority,
IsCompleted = task.IsCompleted,
UserId = task.UserId
};
```

User Response Mapping

User responses should expose only the fields intended for API clients:

```
return new UserResponse
{
    Id = user.Id,
    FullName = user.FullName,
    Email = user.Email,
    CreatedAtUtc = user.CreatedAtUtc
};
```

This avoids returning the full User entity directly.

Why DTOs Matter

DTOs keep the public API separate from the database model.

This gives the project flexibility:

```
database entity can change
API response can stay stable
internal fields can stay hidden
EF Core navigation properties do not leak accidentally
```

This is especially important now that the project has relationships between users and tasks.

What Not To Do Today

Do not add new endpoints.

Do not add database columns.

Do not create migrations.

Do not add authentication.

Do not add a response wrapper model.

Do not add AutoMapper.

Do not create a large abstraction.

Day 43 only checks response DTO consistency.

Testing Checklist

Get all tasks:

```
GET /api/tasks
```

Expected:

```
200 OK
```

Response should show task response fields only.

Get all users:

```
GET /api/users
```


Expected:

```
200 OK
```

Response should show user response fields only.

Get tasks for a user:

```
GET /api/users/{userId}/tasks
```

Expected when the user exists:

```
200 OK
```

Response should return task DTOs, not full user entities.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 43 reviews the boundary between the internal model and the public API contract.

The project uses EF Core entities such as `User` and `JobTask` to represent database records. Those entities are useful inside the application, but they should not be exposed directly from API endpoints.

Instead, the API should return response DTOs such as `UserResponse` and `TaskResponse`. These models define exactly what the API client is allowed to see.

This keeps the API stable. If the database entity gains a new property, relationship, or internal field later, it does not automatically leak into the public response.

The lesson is simple but important: controllers should return response models, services should map entities to DTOs, and EF Core entities should remain internal implementation details.

Reader Exercise

Open the users and tasks API flow.

For each endpoint, check:

- what the controller returns
- what the service method returns
- whether any EF Core entity is returned directly
- whether the response DTO contains only public fields

Then test the API responses in Swagger and confirm the output shape is clean.

Future Improvement

Later response design improvements can include:

- response documentation
- integration tests for response shape

- consistent error response DTOs
- pagination response DTOs
- separate summary and detail response models

Next Lesson

Day 43 confirms response DTO consistency.

The next lesson can continue strengthening the API contract through tests, documentation, or cleaner response behavior.

Day 44 - Weekly Validation Review

Lesson Goal

Review and stabilize validation and response behavior across the API.

Day 44 is a review and testing day.

No new feature is added.

Why This Is Day 44

The project recently added validation, common API messages, global exception handling, HTTP status code cleanup, and response DTO consistency checks.

Before moving into more backend features, the current API behavior should be tested as a complete flow.

The review focuses on:

```
invalid create requests
invalid update requests
missing user records
missing task records
unexpected server errors
response status codes
response messages
```

Areas Reviewed

Request DTOs:

```
CreateTaskRequest
UpdateTaskRequest
CreateUserRequest
UpdateUserRequest
```

Controllers:

```
TasksController
UsersController
```

Shared messages:

```
ErrorMessages
```

Documentation:

```
README
```

Expected Validation Behavior

Invalid request body should return:

```
400 Bad Request
```

Examples:

```
empty task title
missing task title
invalid user id
empty user full name
invalid user email
missing priority
```

This confirms data annotation validation is still working.

Expected Missing Record Behavior

Missing task:

```
404 Not Found
Task not found.
```

Missing user:

```
404 Not Found
User not found.
```

This confirms shared messages and controller responses are consistent.

Expected Success Behavior

Create user:

```
201 Created
```

Create task:

```
201 Created
```

Update user:

```
200 OK
```

Update task:

```
200 OK
```

Delete user:

```
204 No Content
```

Delete task:

```
204 No Content
```

Expected Unexpected Error Behavior

Unexpected server error:

```
500 Internal Server Error
An unexpected error occurred.
```

This confirms the basic global exception handling middleware is still active.

Test: Invalid Create User

Endpoint:

POST /api/users

Example body:

```
{
  "fullName": "",
  "email": "not-valid-email"
}
```

Expected:

400 Bad Request

Test: Missing User

Endpoint:

GET /api/users/999999

Expected:

404 Not Found
User not found.

Test: Invalid Create Task

Endpoint:

POST /api/tasks

Example body:

```
{
  "title": "",
  "description": "Test validation",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

400 Bad Request

Test: Missing Task

Endpoint:

GET /api/tasks/999999

Expected:

404 Not Found
Task not found.

Test: Create User

Endpoint:

POST /api/users

Example body:

```
{
  "fullName": "Test User",
  "email": "test@example.com"
}
```

Expected:

201 Created

Test: Create Task

Endpoint:

```
POST /api/tasks
```

Example body:

```
{
  "title": "Review validation",
  "description": "Weekly validation review",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Expected:

```
201 Created
```

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add database changes.

Do not create migrations.

Do not add authentication.

Do not add a custom response wrapper.

Do not redesign architecture.

Day 44 only tests and stabilizes current validation behavior.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 44 is a weekly validation review.

The API now has several layers of response behavior: data annotations for invalid requests, shared messages for repeated errors, consistent status codes, global exception handling for unexpected failures, and DTO-based responses.

A review day confirms those pieces work together.

The value of this lesson is stability. Instead of adding a new feature, the API is tested against expected behavior: invalid requests return 400, missing records return 404, successful creates return 201, successful updates return 200, successful deletes return 204, and unexpected errors return a safe 500.

This keeps the backend reliable before more features are added.

Reader Exercise

Run the validation review checklist:

- send an invalid create user request
- request a missing user
- send an invalid create task request
- request a missing task
- create a valid user
- create a valid task
- update an existing record
- delete an existing record

Confirm every response uses the expected status code and message.

Future Improvement

Later improvements can include:

- automated integration tests
- documented response examples
- custom problem details responses
- stricter priority validation
- due date validation
- consistent business error response models

Next Lesson

Day 44 confirms validation and response behavior.

The next lesson can move forward with stronger confidence that the API contract is stable.

Day 45 - Review Application Layer Boundaries

Lesson Goal

Review the application layer boundaries and confirm the project structure still makes sense.

Day 45 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 45

The project now has controllers, service interfaces, request DTOs, response DTOs, domain entities, EF Core services, validation, common messages, and basic exception handling.

Before adding more backend complexity, the current boundaries should be reviewed.

The main question is:

Is each part of the project doing the right job?

Current Project Layers

The project is organized around these layers:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
```

Expected responsibilities:

```
Api           -> controllers, HTTP requests, HTTP responses
Application    -> service interfaces, request DTOs, response DTOs
Domain         -> core entities
Infrastructure  -> EF Core database work and service implementations
```

Controller Review

Controllers should stay thin.

They should:

```
receive HTTP requests
call service interfaces
return HTTP responses
handle simple status code decisions
```

Controllers should not:

```
use ApplicationDbContext directly
create or update entities directly
contain main business workflow logic
return EF Core entities directly
```

Acceptable controller pattern:

```
var response = await _taskService.GetById(id);

if (response is null)
{
    return NotFound(ErrorMessages.TaskNotFound);
}

return Ok(response);
```

This is acceptable because the controller delegates the use case to the service and only decides the HTTP response.

Service Interface Review

ITaskService should describe task use cases clearly:

```
get tasks
get task by id
create task
update task
delete task
complete task
reopen task
get tasks by user id
```

IUserService should describe user use cases clearly:

```
get users
get user by id
create user
update user
delete user
```

The API should depend on these interfaces instead of directly depending on database code.

DTO Review

Request DTOs should be used for API input:

```
CreateUserRequest
UpdateUserRequest
CreateTaskRequest
UpdateTaskRequest
```

Response DTOs should be used for API output:

```
UserResponse
TaskResponse
```

Controllers should not return domain entities like:

```
User
JobTask
```

DTOs keep the public API contract separate from the internal database model.

Infrastructure Review

The current service implementations live in Infrastructure because they use EF Core.

That is acceptable at this stage.

Example:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

This is acceptable inside TaskService because the service implementation is part of Infrastructure.

It would not be acceptable inside a controller.

What Is Acceptable Right Now

It is acceptable that:

```
controllers call services
services contain the main workflow logic
Infrastructure services use AppDbContext
Application defines interfaces and DTOs
Domain contains entities
controllers return DTO responses
```

The project does not need a larger architecture yet.

What Is Not Acceptable

Do not put this inside a controller:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Reason:

```
Controllers should not talk directly to the database.
```

Do not return this from a controller:

```
return Ok(taskEntity);
```

Reason:

```
Controllers should return DTOs through service responses.
```


What Not To Add Today

Do not add repositories.

Do not add MediatR.

Do not add CQRS.

Do not add AutoMapper.

Do not redesign the architecture.

Do not add database changes.

Do not add new endpoints.

Day 45 is only about reviewing boundaries and fixing obvious misplaced logic.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 45 reviews the boundaries of the backend architecture.

Clean Architecture is not about adding as many patterns as possible. It is about keeping responsibilities clear. Controllers should handle HTTP behavior. Application should describe use cases and DTOs. Domain should hold core entities. Infrastructure should handle database-specific work.

At this stage, the project does not need repositories, CQRS, MediatR, or AutoMapper. Adding those too early would increase complexity without solving a real problem.

The useful review is simpler: controllers should stay thin, services should contain workflow logic, database access should stay out of controllers, and API responses should use DTOs instead of EF Core entities.

This confirms that the project can continue growing without turning controllers into database scripts or exposing internal models directly to API clients.

Reader Exercise

Open the users and tasks flow.

For each endpoint, check:

- does the controller call a service?
- does the controller avoid direct database access?
- does the service return a response DTO?
- does Infrastructure handle EF Core work?
- are domain entities kept out of public API responses?

If the answer is yes, the current boundaries are healthy.

Future Improvement

Later architecture improvements can include:

- repositories if service code becomes too large
- separate application use-case classes
- query objects for complex filtering
- AutoMapper if manual mapping becomes repetitive
- integration tests for boundary behavior

Those should be added when the project needs them, not before.

Next Lesson

Day 45 confirms the application layer boundaries.

The next lesson can continue strengthening the architecture while keeping the current simple structure intact.

Day 46 - Review Infrastructure Layer

Lesson Goal

Review the Infrastructure layer and confirm database logic is in the right place.

Day 46 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 46

Day 45 reviewed the Application layer boundaries.

Day 46 reviews the Infrastructure layer because the project now uses EF Core, SQL Server, migrations, and database-backed services.

The main question is:

Is database-specific code staying in Infrastructure?

Infrastructure Responsibilities

Infrastructure is responsible for database and external system details.

In this project, that currently means:

```
AppDbContext
EF Core setup
SQL Server persistence
migrations
database-backed TaskService
database-backed UserService
```

Infrastructure is allowed to use EF Core.

Controllers and Domain should not directly depend on EF Core.

Files Reviewed

Main areas to review:

```
AppDbContext
Program.cs
appsettings.json
TaskService
UserService
EF Core migrations
project references
```

AppDbContext Review

AppDbContext should live in Infrastructure.

It should contain:

```
DbSet<User>
DbSet<JobTask>
task-user relationship configuration
```

Expected idea:

```
public DbSet<User> Users { get; set; }
public DbSet<JobTask> Tasks { get; set; }
```

The relationship between users and tasks should also be configured in the context.

Program.cs Review

The API project starts the application and wires dependencies together.

It is acceptable for Program.cs to register AppDbContext:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

It should also register services with scoped lifetime:

```
builder.Services.AddScoped<ITaskService, TaskService>();
builder.Services.AddScoped<IUserService, UserService>();
```

This keeps EF Core services aligned with request-based database context usage.

Connection String Review

The SQL Server connection string belongs in application configuration.

Do not paste passwords or secrets into notes.

For this project stage, the important check is that the API can read the connection string and create AppDbContext.

TaskService Review

Task database operations should live in the Infrastructure task service.

Examples:

```
add task
find task by id
remove task
save changes
query task filters
```

This is acceptable inside Infrastructure because Infrastructure owns database access.

It is not acceptable inside controllers.

UserService Review

User database operations should live in the Infrastructure user service.

Examples:

```
add user
find user by id
remove user
save changes
query users
```

Controllers should call IUserService.

They should not use ApplicationDbContext directly.

Migrations Review

EF Core migrations should stay in Infrastructure.

Migration files represent database structure changes, so they belong beside persistence code.

Do not edit migration files manually during this review.

Project Reference Review

Expected dependency direction:

```
Api -> Application
Api -> Infrastructure
Infrastructure -> Application
Infrastructure -> Domain
Application -> Domain if needed
```

Controllers should not need direct EF Core package usage.

Domain should not reference EF Core.

What Is Acceptable Right Now

This is acceptable inside an Infrastructure service:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Reason:

```
Infrastructure owns database access.
```

This is acceptable inside Program.cs:

```
builder.Services.AddDbContext<AppDbContext>(...);
```

Reason:

```
The API project starts the app and wires dependencies together.
```

What Is Not Acceptable

Do not put this inside controllers:

```
private readonly ApplicationDbContext _dbContext;
```

Reason:

```
Controllers should not directly access the database.
```

Do not move EF Core code into Domain:

```
using Microsoft.EntityFrameworkCore;
```

Reason:

Domain should stay independent from EF Core.

What Not To Add Today

Do not add repositories.

Do not add Unit of Work.

Do not add CQRS.

Do not add MediatR.

Do not create new migrations.

Do not add database tables.

Do not add endpoints.

Do not redesign the architecture.

Day 46 only reviews Infrastructure boundaries and fixes obvious misplaced database logic.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 46 reviews the database side of the project architecture.

The API now uses SQL Server through EF Core. That means the project needs a clear place for database-specific logic. In this structure, that place is Infrastructure.

AppDbContext, migrations, and EF Core-backed service implementations belong in Infrastructure. Controllers should not query the database directly. Domain entities should not know about EF Core. Application should describe use cases and DTOs without owning database details.

This review keeps the project from drifting into a common backend problem: controllers becoming database scripts. By keeping database access in Infrastructure, the API layer stays focused on HTTP behavior and the rest of the project remains easier to reason about.

Reader Exercise

Review the project references and answer:

- Does the API reference Application and Infrastructure?
- Does Infrastructure reference Application and Domain?
- Does Domain avoid EF Core references?
- Do controllers avoid AppDbContext?

- Are migrations stored in Infrastructure?

If the answer is yes, the Infrastructure boundary is healthy.

Future Improvement

Later infrastructure improvements can include:

- repository classes if services become too large
- better EF Core configuration files
- seed data
- integration tests
- production-safe connection string handling
- logging database errors

Those should be added when the project needs them.

Next Lesson

Day 46 confirms Infrastructure boundaries.

The next lesson can continue strengthening the project structure without adding unnecessary architecture too early.

Day 47 - Review Service Interfaces

Lesson Goal

Review service interfaces and confirm method names are clean, useful, and consistent.

Day 47 is mostly a review and cleanup day.

No new feature is added.

Why This Is Day 47

Day 46 reviewed the Infrastructure layer.

Day 47 reviews the service interfaces in the Application layer.

The main question is:

Do the interfaces clearly describe what the application can do?

Service interfaces should:

- describe clear use cases
- use consistent method names
- return DTOs instead of database entities
- avoid EF Core details
- be easy to explain in an interview

Interfaces Reviewed

Task service interface:

```
ITaskService
```

User service interface:

```
IUserService
```

Related implementations and callers:

```
TaskService  
UserService  
TasksController  
UsersController
```

Task Service Review

The task interface should describe task use cases clearly.

Examples:

```
Task<List<TaskResponse>> GetAll(bool? isCompleted, string? search, int? userId);  
Task<TaskResponse?> GetById(int id);  
Task<TaskResponse> CreateTask(CreateTaskRequest request);  
Task<TaskResponse?> UpdateTask(int id, UpdateTaskRequest request);  
Task<bool> DeleteTask(int id);  
Task<TaskResponse?> CompleteTask(int id);  
Task<TaskResponse?> ReopenTask(int id);  
Task<List<TaskResponse?>> GetByUserId(int userId);
```

These method names are acceptable because they describe real application actions.

User Service Review

The user interface should describe user use cases clearly.

Examples:

```
Task<UserResponse> CreateUser(CreateUserRequest request);  
Task<List<UserResponse>> GetAll();  
Task<UserResponse?> GetById(int id);  
Task<UserResponse?> UpdateUser(int id, UpdateUserRequest request);  
Task<bool> DeleteUser(int id);
```

These names are clear enough for the current project stage.

Naming Rules

Use simple names.

Good examples:

```
GetAll  
GetById  
CreateTask  
UpdateTask  
DeleteTask  
CompleteTask  
ReopenTask  
GetByUserId
```

Avoid vague names:

```
Process  
Handle  
DoTask  
Manage  
Execute
```

Avoid names that expose database details:

```
InsertIntoDatabase  
SaveToSqlServer  
QueryDbSet
```

What Is Acceptable Right Now

It is acceptable if task methods include Task in the name:

```
CreateTask
UpdateTask
DeleteTask
CompleteTask
ReopenTask
```

Reason:

The service is task-focused and the names are beginner-friendly.

It is also acceptable if read methods are generic:

```
GetAll
GetById
```

Reason:

Inside `ITaskService` or `IUserService`, the context is already clear.

Possible Cleanup

More explicit names can also work:

```
GetAllTasks
GetTaskById
GetTasksByUserId
GetAllUsers
GetUserById
```

But renaming only for style is not necessary if the current names are already clear.

Consistency matters more than personal preference.

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not add repositories.

Do not make database changes.

Do not create migrations.

Do not do async suffix cleanup today.

Day 47 is only about service interface naming and clarity.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Interview Explanation

A simple explanation:

The Application project contains service interfaces because it defines what the application can do.
The Infrastructure project implements those interfaces because it contains the database-backed code.
Controllers depend on interfaces, not directly on database classes.

Lesson Explanation

Day 47 reviews service interface clarity.

Interfaces are important because they describe the use cases of the application. Controllers call these interfaces, and Infrastructure provides the database-backed implementations.

The service interface should not expose EF Core details. It should not mention SQL Server, DbSet, or database-specific operations. It should describe application behavior such as creating a task, updating a user, completing a task, or getting tasks for a user.

This keeps the API layer separated from the database implementation and makes the code easier to explain.

The lesson is also about restraint. If names are already clear, there is no need to rename everything for style. A good interface is understandable, consistent, and focused on application use cases.

Reader Exercise

Open the task and user service interfaces.

For each method, answer:

- does the name describe a real use case?
- does the return type use a response DTO?
- does the method avoid EF Core details?
- would the method name make sense in an interview explanation?

If the answer is yes, the interface is healthy.

Future Improvement

Later cleanup can include:

- async naming review
- splitting large services if they grow too much
- use-case classes if workflow logic becomes complex
- tests for interface behavior
- repository abstraction only if the project needs it

Next Lesson

Day 47 reviews service interfaces.

The next lesson can focus on async naming cleanup or another small architecture review step.

Day 48 - Add Async Suffix To Service Methods

Lesson Goal

Improve C# async method naming consistency across the service layer.

Day 48 is a naming cleanup day.

No new feature is added.

Why This Is Day 48

Day 47 reviewed the service interfaces.

Day 48 applies a common C# convention:

methods that return Task or Task<T> should use the Async suffix

Example:

```
GetAll()
```

becomes:

```
GetAllAsync()
```

This makes asynchronous code easier for other .NET developers to recognize.

Files Updated

Main areas updated:

```
ITaskService
IUserService
TaskService
UserService
TasksController
UsersController
```

The service interfaces define the method names.

The Infrastructure implementations must match those names.

The controllers must call the renamed methods.

Task Service Rename

Task service methods are renamed with the Async suffix:

```
GetAll -> GetAllAsync
GetById -> GetByIdAsync
CreateTask -> CreateTaskAsync
UpdateTask -> UpdateTaskAsync
DeleteTask -> DeleteTaskAsync
CompleteTask -> CompleteTaskAsync
ReopenTask -> ReopenTaskAsync
GetByUserId -> GetByUserIdAsync
```

Expected shape:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int? userId);
Task<TaskResponse?> GetByIdAsync(int id);
Task<TaskResponse> CreateTaskAsync(CreateTaskRequest request);
Task<TaskResponse?> UpdateTaskAsync(int id, UpdateTaskRequest request);
```

```
Task<bool> DeleteTaskAsync(int id);
Task<TaskResponse?> CompleteTaskAsync(int id);
Task<TaskResponse?> ReopenTaskAsync(int id);
Task<List<TaskResponse?>> GetByUserIdAsync(int userId);
```

User Service Rename

User service methods are also renamed:

```
CreateUser -> CreateUserAsync
GetAll -> GetAllAsync
GetById -> GetByIdAsync
UpdateUser -> UpdateUserAsync
DeleteUser -> DeleteUserAsync
```

Expected shape:

```
Task<UserResponse> CreateUserAsync(CreateUserRequest request);
Task<List<UserResponse>> GetAllAsync();
Task<UserResponse?> GetByIdAsync(int id);
Task<UserResponse?> UpdateUserAsync(int id, UpdateUserRequest request);
Task<bool> DeleteUserAsync(int id);
```

Controller Updates

Controllers should call the renamed service methods.

Example:

```
var response = await _taskService.GetByIdAsync(id);
```

instead of:

```
var response = await _taskService.GetById(id);
```

The same applies for user service calls.

Controller Action Names

Do not rename controller action methods today unless required.

This is acceptable:

```
public async Task<IActionResult> GetById(int id)
{
    var response = await _taskService.GetByIdAsync(id);
}
```

Reason:

Controller action names are also used by routing and CreatedAtAction.

Day 48 only renames service methods.

Why This Matters

The Async suffix tells C# developers that a method is asynchronous.

It improves readability in:

```
interfaces
service implementations
controller calls
code reviews
interview explanations
```

It also follows common .NET naming conventions.

What Not To Do Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not make database changes.

Do not create migrations.

Do not add repositories.

Do not redesign architecture.

Day 48 only cleans up async method names.

Search Check

After renaming, search for old service method calls:

```
GetAll(  
  GetById(  
    CreateTask(  
      UpdateTask(  
        DeleteTask(  
          CompleteTask(  
            ReopenTask(  
              GetByIdUserId(  
                CreateUser(  
                  UpdateUser(  
                    DeleteUser(  
                      
```

It is acceptable if controller action method names still use some of these names.

The important part is that service interface and implementation methods use Async.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
5d25572 Day 48: Add async suffix to service methods
```

Lesson Explanation

Day 48 is a naming cleanup lesson.

The project already uses asynchronous service methods because database access through EF Core is asynchronous. The method signatures return Task or Task<T>, but the method names did not clearly show that.

Adding the Async suffix makes the code more idiomatic for C# developers. It also makes interfaces easier to scan. When a developer sees GetByIdAsync, they immediately know the method should be awaited.

This is a small change, but it improves consistency across the Application interfaces, Infrastructure implementations, and API controllers.

The key discipline is to keep the cleanup focused. No endpoints, DTOs, database changes, or architecture changes are needed.

Reader Exercise

Open the service interfaces and check each method.

For every method that returns:

```
Task  
Task<T>
```

Confirm the method name ends with:

```
Async
```

Then check controller calls and confirm they call the renamed service methods.

Future Improvement

Later cleanup can include:

- integration tests after renames
- consistent naming for query helpers
- reviewing private helper method names
- separating large services if needed

Next Lesson

Day 48 improves async naming consistency.

The next lesson can continue with small architecture cleanup or move toward testing the service layer.

Day 49 - Remove Dead Code And Starter Files

Lesson Goal

Review the repository and remove safe starter code or dead code.

Day 49 is a cleanup day.

No new feature is added.

Why This Is Day 49

Day 48 cleaned up async service method names.

Day 49 focuses on keeping the repository clean before adding more backend features.

As a project grows, leftover starter files and unused code make the codebase harder to understand.

The goal is to remove only code that is clearly safe to remove.

What To Search For

Review the project for:

```
unused starter files
unused classes
unused methods
unused using statements
old WeatherForecast code
old Class1.cs files
unused template comments
```

Cleanup should be careful.

Do not delete code just because it looks unfamiliar.

Projects Reviewed

API project:

```
check for WeatherForecast files
check for unused controllers
check for template comments
check for unused using statements
```

Application project:

```
check for Class1.cs
check for unused DTO files
check for unused interface files
check for unused using statements
```

Domain project:

```
check for Class1.cs
check for unused entity files
check for unused using statements
```

Infrastructure project:

```
check for Class1.cs
check for unused service files
check for unused persistence files
check for unused using statements
```

Search Commands

Search for starter files:

```
Get-ChildItem -Recurse -Filter Class1.cs
Get-ChildItem -Recurse -Filter *Weather*
```

Search for old template text:

```
rg "WeatherForecast|Class1|TODO|template|sample|example" src
```

Search for public methods that may need manual review:

```
rg "public .*\\(" src
```

Use judgment before deleting anything.

Safe Cleanup Examples

Safe examples:

```
Class1.cs
WeatherForecast.cs
WeatherForecastController.cs
unused template comments
unused using statements
```

Only remove them if the project builds after removal.

What Not To Remove

Do not remove:

```
DTOs currently used by controllers or services
service interfaces
service implementations
entity classes
migrations
AppDbContext
Program.cs registrations
appsettings.json
```

Do not remove migration files during this cleanup.

Migrations are part of the database history.

What Not To Add Today

Do not add new endpoints.

Do not add new DTOs.

Do not add new services.

Do not make database changes.

Do not create migrations.

Do not add repositories.

Do not redesign architecture.

Day 49 is only about safe cleanup.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

The build is the final safety check after removing dead code.

Lesson Explanation

Day 49 is about keeping the repository clean.

A backend project can become confusing if starter files, unused classes, and old template code remain in the solution. These files distract from the real application and make it harder for another developer to understand what matters.

Cleanup should be conservative. Remove obvious starter code such as old WeatherForecast files or empty Class1.cs files, but do not remove DTOs, services, entities, migrations, AppDbContext, or application startup code.

The important rule is simple: delete only what is clearly safe, then build the project.

Reader Exercise

Search the project for:

```
WeatherForecast
Class1
TODO
sample
template
```

For each result, decide:

- is this still used?
- is it starter/template code?
- will the project build without it?

Remove only the safe items and run the build.

Future Improvement

Later cleanup can include:

- organizing folders
- adding code analyzers
- enabling stricter compiler warnings
- reviewing duplicated mapping code
- adding tests to protect cleanup work

Next Lesson

Day 49 removes safe dead code.

The next lesson can continue improving code quality or begin the next backend feature on a cleaner project.

Day 50 - Architecture Review Day

Lesson Goal

Review the current JobTrackr architecture and confirm the four-project structure still makes sense.

Day 50 is a review day.

No new feature is added.

Why This Is Day 50

Day 49 cleaned up dead code and starter files.

Day 50 reviews the full project architecture before moving into the next stage of backend development.

The main question is:

```
Are the project responsibilities still clear?
```


Projects Reviewed

The solution is split into four projects:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
```

Each project should have a clear responsibility.

JobTrackr.Api

The API project should contain:

```
controllers
Program.cs
middleware
app settings
dependency injection setup
Swagger setup
```

It should not contain:

```
domain entities
EF Core migrations
database query logic inside controllers
```

The API project handles HTTP behavior and application startup.

JobTrackr.Application

The Application project should contain:

```
request DTOs
response DTOs
service interfaces
shared application constants like ErrorMessages
```

It should not contain:

```
AppDbContext
EF Core migrations
SQL Server setup
controller classes
```

The Application project describes what the backend can do without owning database details.

JobTrackr.Domain

The Domain project should contain:

```
domain entities
core business objects
```

Current entities:

```
User
JobTask
```

It should not contain:

```
controllers
DTOs
EF Core DbContext
SQL Server setup
```

Domain should stay independent from infrastructure details.

JobTrackr.Infrastructure

The Infrastructure project should contain:

- AppDbContext
- EF Core migrations
- database-backed service implementations
- SQL Server persistence code

It should not contain:

- controllers
- Swagger setup
- API route logic

Infrastructure owns database-specific implementation details.

Project Reference Direction

Expected direction:

- Api -> Application
- Api -> Infrastructure
- Application -> Domain
- Infrastructure -> Application
- Infrastructure -> Domain
- Domain -> no project references

This direction is acceptable for the current stage of the project.

Search Checks

Controllers should not use AppDbContext directly.

Expected:

- no AppDbContext usage inside controllers

Domain should not use EF Core.

Expected:

- no Microsoft.EntityFrameworkCore usage inside Domain

Starter code should be gone.

Expected:

- no WeatherForecast or Class1 starter files

README Review

The README should accurately describe:

- .NET 8 Web API
- SQL Server persistence
- Entity Framework Core
- users API
- tasks API
- task-user relationship
- current project architecture

Update README only if it is outdated.

What Not To Add Today

Do not add repositories.

Do not add Unit of Work.

Do not add CQRS.

Do not add MediatR.

Do not add new endpoints.

Do not add new DTOs.

Do not make database changes.

Do not create migrations.

Do not add authentication.

Day 50 is architecture review only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Interview Explanation

A simple explanation:

```
JobTrackr is split into four projects.  
Api handles HTTP requests and dependency injection.  
Application contains DTOs and service interfaces.  
Domain contains core entities.  
Infrastructure contains EF Core, SQL Server, migrations, and database-backed service  
implementations.  
This keeps controllers thin and keeps database logic out of API controllers.
```

Lesson Explanation

Day 50 is an architecture review checkpoint.

The project has grown from a small API into a database-backed backend with users, tasks, validation, middleware, DTOs, EF Core, SQL Server, and multiple project layers.

At this stage, it is important to stop and confirm that the project structure still makes sense. The API project should handle HTTP behavior. Application should hold DTOs and service interfaces. Domain should hold core entities. Infrastructure should hold EF Core, migrations, and database-backed service implementations.

This keeps responsibilities clear. Controllers stay thin, database logic stays out of controllers, Domain stays independent from EF Core, and the public API does not directly expose database entities.

The lesson is not to add more architecture. The lesson is to confirm the current architecture is still understandable and useful.

Reader Exercise

Review each project and answer:

- what is this project responsible for?
- what should not be inside this project?

- does the project reference direction make sense?
- are controllers free from direct database access?
- is Domain free from EF Core?
- is README architecture information accurate?

If the answer is yes, the architecture is healthy for the current stage.

Future Improvement

Later architecture improvements can include:

- repositories if service classes grow too large
- use-case classes if workflows become complex
- integration tests
- clearer module organization
- authentication and authorization structure
- deployment configuration

Those should be added when needed, not just for appearance.

Next Lesson

Day 50 confirms the project architecture.

The next lesson can move into the next backend stage with a cleaner, reviewed foundation.

Day 51 - Plan Authentication Flow

Lesson Goal

Plan the authentication flow before writing code.

Day 51 starts the authentication phase, but it does not implement authentication yet.

Why This Is Day 51

Day 50 reviewed the current project architecture.

Before adding authentication code, the project needs a clear plan for:

```
registration
login
password hashing
JWT tokens
future auth endpoints
what should not be stored in the database
```

Authentication is sensitive enough that planning first is better than rushing into implementation.

First Authentication Scope

The first version should be simple.

Users should be able to:

```
register
login
receive a JWT token
use the token later for protected APIs
```

Do not add these yet:

```
roles
refresh tokens
email verification
password reset
```

Those features can come later.

Planned Auth Endpoints

The first authentication endpoints will be:

```
POST /api/auth/register
POST /api/auth/login
```

Do not create them during the planning day.

Day 51 only defines the direction.

Register Request

Planned registration request:

```
public class RegisterRequest
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;
}
```

Purpose:

```
FullName identifies the user
Email will be used for login
Password will be hashed before storage
```

Login Request

Planned login request:

```
public class LoginRequest
{
    public string Email { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;
}
```

Purpose:

```
email identifies the user
password proves the user owns the account
```

Auth Response

Planned authentication response:

```
public class AuthResponse
{
    public int UserId { get; set; }

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;
}
```

```
    public string Token { get; set; } = string.Empty;
}
```

Purpose:

```
client receives basic user details
client receives JWT token for protected requests
```

Password Storage Rule

Never store plain text passwords.

Bad:

```
Password = "mypassword123"
```

Good:

```
PasswordHash = hashed password value
```

The database should store:

```
public string PasswordHash { get; set; } = string.Empty;
```

Do not return PasswordHash from API responses.

Password Hashing Plan

Use ASP.NET Core password hashing:

```
PasswordHasher<User>
```

Reasons:

```
built into ASP.NET Core
safer than custom hashing
beginner-friendly
production-style enough for this stage
```

Do not create a custom hashing algorithm.

JWT Plan

Login should return a JWT token.

The JWT can contain simple claims:

```
user id
email
full name if needed
```

The client will later send the token like this:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

JWT setup will be implemented later.

Do not implement JWT during Day 51.

Future User Entity Change

Later, the User entity may need:

```
public string PasswordHash { get; set; } = string.Empty;
```

That change will require:

```
entity update
EF Core migration
database update
```

Do not add it until the implementation day.

Planned File Structure

Possible files for the implementation phase:

```
Application/Auth/RegisterRequest.cs
Application/Auth/LoginRequest.cs
Application/Auth/AuthResponse.cs
Application/Auth/IAuthService.cs
Infrastructure/Auth/AuthService.cs
Api/Controllers/AuthController.cs
```

These files should be created when authentication implementation starts, not during planning.

What Not To Add Today

Do not add an auth controller.

Do not add an auth service.

Do not add auth DTOs.

Do not add JWT packages.

Do not add JWT settings.

Do not add a password hash column.

Do not create a migration.

Do not protect endpoints.

Do not add roles.

Do not add refresh tokens.

Day 51 is planning only.

Interview Explanation

A simple explanation:

```
Before implementing authentication, I planned the register and login flow.
The API will store password hashes, not plain text passwords.
Login will return a JWT token, and later protected endpoints will require that token.
```

Lesson Explanation

Day 51 begins the authentication phase by planning before coding.

Authentication affects database design, API contracts, security, and future endpoint protection. Because of that, it should not be added casually.

The first version stays small: users register, users log in, passwords are hashed, and login returns a JWT token. More advanced features like roles, refresh tokens, email verification, and password reset are deliberately left out.

The most important rule is password safety. The API should never store or return plain text passwords. It should store a password hash and keep that field out of response DTOs.

This planning day gives the next implementation steps a clear direction.

Reader Exercise

Before writing authentication code, answer:

- what fields are needed for registration?
- what fields are needed for login?
- where should password hashes be stored?
- why should plain text passwords never be stored?
- what should the login response return?
- which authentication features should be delayed?

If those answers are clear, implementation will be easier.

Future Improvement

Future authentication work can include:

- register endpoint
- login endpoint
- password hashing
- JWT generation
- JWT validation
- protected endpoints
- user ownership rules
- roles later
- refresh tokens later

Next Lesson

Day 51 plans authentication.

The next lesson can start implementing authentication step by step, beginning with DTOs or the password hash field.

Day 52 - Create Auth DTOs

Lesson Goal

Create request and response DTOs for authentication.

Day 52 only creates DTO classes.

Authentication logic is not implemented yet.

Why This Is Day 52

Day 51 planned the authentication flow.

Day 52 starts implementation in a controlled way by creating the models that will define the auth API contract.

The first authentication flow will need:

```
registration input
login input
authentication output
```

These should be represented by DTOs before creating the service or controller.

Files Created

Authentication DTOs are added under the Application layer:

```
Application/Auth/RegisterRequest.cs
Application/Auth/LoginRequest.cs
Application/Auth/AuthResponse.cs
```

These files belong in Application because they define API request and response shapes.

They do not belong in Domain because they are not core entities.

They do not belong in Infrastructure because they are not database implementation details.

RegisterRequest

RegisterRequest represents the request body for user registration.

It contains:

```
FullName
Email
Password
```

It also includes validation attributes:

```
FullName is required
FullName max length is 100
Email is required
Email must be valid
Email max length is 150
Password is required
Password minimum length is 6
Password max length is 100
```

This keeps basic request validation close to the DTO.

LoginRequest

LoginRequest represents the request body for user login.

It contains:

```
Email
Password
```

It also includes validation attributes:

```
Email is required
Email must be valid
Password is required
```

Login should stay simple at this stage.

The user identifies themselves with email and proves ownership with password.

AuthResponse

AuthResponse represents the response after successful authentication.

It contains:

```
UserId
FullName
Email
Token
```

The Token field is prepared for JWT usage later.

Day 52 does not generate the token yet.

Password Safety

The request DTO accepts a password because the user must send it during registration and login.

But the API should never return the password.

The response DTO does not include:

```
Password
PasswordHash
```

Later implementation should store a password hash, not the plain text password.

What Not To Add Today

Do not add an auth controller.

Do not add an auth service.

Do not add password hashing logic.

Do not add JWT generation.

Do not add JWT packages.

Do not change the User entity.

Do not create a database migration.

Do not protect endpoints.

Day 52 only creates authentication DTOs.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
85052fb Day 52: Create auth DTOs
```

Interview Explanation

A simple explanation:

```
I created authentication DTOs before implementing authentication logic.
RegisterRequest and LoginRequest define the API input.
```

AuthResponse defines the API output after successful authentication.
This keeps request and response models separate from database entities.

Lesson Explanation

Day 52 begins authentication implementation by creating DTOs only.

This is a clean first step because authentication has multiple moving parts: registration, login, password hashing, JWT generation, database changes, and endpoint protection. Instead of doing everything at once, the project first defines the request and response models.

RegisterRequest describes what a client must send to create an account. LoginRequest describes what a client must send to sign in. AuthResponse describes what the API will eventually return after successful authentication.

This keeps the API contract clear before adding authentication behavior.

Reader Exercise

Review the three DTOs and answer:

- what fields are needed to register?
- what fields are needed to login?
- why does AuthResponse include a token?
- why should AuthResponse not include Password or PasswordHash?
- why do these DTOs belong in Application?

Future Improvement

The next authentication steps can include:

- adding PasswordHash to the User entity
- creating an EF Core migration
- adding IAuthService
- implementing registration
- implementing login
- adding JWT generation
- adding an auth controller

Next Lesson

Day 52 creates the auth DTOs.

The next lesson can start adding the database support needed for authentication.

Day 53 - Add Password Hashing Service

Lesson Goal

Add a password hashing service so passwords are never stored as plain text.

Day 53 only creates the hashing service.

Register and login are not implemented yet.

Why This Is Day 53

Day 52 created the authentication DTOs.

The next authentication building block is password hashing.

Before the API can register or log in users, it needs a safe way to:

```
hash a new password
verify a login password against a stored hash
```

The project should never store plain text passwords.

Package Added

The Application project uses ASP.NET Core Identity password hashing:

```
Microsoft.Extensions.Identity.Core
```

For this stage, both the password hashing interface and implementation live in the Application project.

This keeps the implementation simple while the authentication flow is still being built step by step.

Files Created

Two authentication service files are added:

```
Application/Auth/IPasswordHasherService.cs
Application/Auth/PasswordHasherService.cs
```

IPasswordHasherService

The interface defines what the password hashing service can do:

```
public interface IPasswordHasherService
{
    string HashPassword(string password);

    bool VerifyPassword(string password, string passwordHash);
}
```

Other code can depend on this interface instead of directly depending on PasswordHasher.

PasswordHasherService

The implementation uses ASP.NET Core Identity:

```
PasswordHasher<object>
```

It has two responsibilities:

```
create a password hash
verify a plain text password against an existing hash
```

This avoids writing a custom password hashing algorithm.

Verify Password Order

The verification call must use the correct argument order:

```
VerifyHashedPassword(new object(), passwordHash, password)
```

Meaning:

```
stored hash first  
plain password second
```

This order matters.

Dependency Injection

The API registers the password hashing service:

```
builder.Services.AddScoped<IPasswordHasherService, PasswordHasherService>();
```

Later, the auth service can use this dependency during registration and login.

Security Rule

Never store:

```
Password = "mypassword123"
```

Store only:

```
PasswordHash = hashed password value
```

Also do not return password hashes from API responses.

What Not To Add Today

Do not add register endpoint.

Do not add login endpoint.

Do not add auth controller.

Do not add auth service.

Do not add JWT generation.

Do not add PasswordHash to the User entity.

Do not create a database migration.

Do not protect endpoints.

Day 53 only sets up password hashing.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
3f6b1fb Day 53: Add password hashing service
```

Interview Explanation

A simple explanation:

I added a password hashing service so the application never stores plain text passwords.
The service uses ASP.NET Core Identity's PasswordHasher.
It can hash a new password and verify a login password against a stored password hash.

Lesson Explanation

Day 53 adds the password hashing service, which is a core security building block for authentication.

Authentication should never store a user's plain password. During registration, the API should hash the password and store only the hash. During login, the API should verify the submitted password against the stored hash.

Instead of writing a custom hashing algorithm, the project uses ASP.NET Core Identity's built-in PasswordHasher. This is safer and more appropriate than inventing password hashing logic manually.

This lesson keeps the scope controlled. It does not add register or login endpoints yet. It only prepares the hashing service that those features will need.

Reader Exercise

Review the password hashing service and answer:

- why should plain text passwords never be stored?
- what method creates a password hash?
- what method verifies a password?
- why should custom hashing algorithms be avoided?
- why should password hashes not be returned from API responses?

Future Improvement

The next authentication steps can include:

- adding PasswordHash to the User entity
- creating an EF Core migration
- creating IAuthService
- implementing registration
- implementing login
- generating JWT tokens
- protecting endpoints

Next Lesson

Day 53 adds password hashing.

The next lesson can add database support for authentication by storing password hashes safely.

Day 54 - Add Register Use Case

Lesson Goal

Add the first authentication use case: user registration.

Day 54 implements register only.

Login and JWT generation are not implemented yet.

Why This Is Day 54

Day 52 created authentication DTOs.

Day 53 added the password hashing service.

Day 54 uses those pieces to register a new user safely.

Registration should:

```
accept full name, email, and password
check if the email already exists
hash the password
save the user with PasswordHash
return a safe response without password data
```

User Entity Change

The User entity now needs a password hash field:

```
public string PasswordHash { get; set; } = string.Empty;
```

This field stores the hashed password value.

It must never store the plain text password.

Auth Service Interface

The authentication service interface starts with one use case:

```
public interface IAuthService
{
    Task<AuthResponse> RegisterAsync(RegisterRequest request);
}
```

More authentication methods can be added later.

For Day 54, only registration is needed.

AuthService

AuthService handles registration logic.

It uses:

```
AppDbContext
IPasswordHasherService
```

The registration flow is:

```
check duplicate email
hash password
create user
save user
return AuthResponse
```

Duplicate emails should be rejected.

Example error:

```
Email is already registered.
```

Password Hashing

The plain password from `RegisterRequest` is passed to the password hashing service:

```
PasswordHash = _passwordHasherService.HashPassword(request.Password)
```

Only the hash is stored.

The plain password is not stored.

Auth Response

After successful registration, the API returns a safe response:

```
return new AuthResponse
{
    UserId = user.Id,
    FullName = user.FullName,
    Email = user.Email,
    Token = string.Empty
};
```

For now, Token is empty because JWT generation is not implemented yet.

This is acceptable for Day 54.

AuthController

The new controller exposes:

```
POST /api/auth/register
```

The controller calls:

```
RegisterAsync
```

If registration succeeds, it returns the auth response.

If the email already exists, it returns:

```
400 Bad Request
```

Dependency Injection

The API registers the auth service:

```
builder.Services.AddScoped<IAuthService, AuthService>();
```

This allows `AuthController` to depend on `IAuthService`.

Database Migration

Because the `User` entity gets a new `PasswordHash` property, EF Core needs a migration:

```
AddUserPasswordHash
```

The database must be updated after the migration.

API Test: Register User

Endpoint:

```
POST /api/auth/register
```

Example body:

```
{
  "fullName": "Test User",
  "email": "testuser@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Example response:

```
{
  "userId": 1,
  "fullName": "Test User",
  "email": "testuser@example.com",
  "token": ""
}
```

API Test: Duplicate Email

Register with the same email again.

Expected:

```
400 Bad Request
Email is already registered.
```

Security Rule

The API response must not include:

```
plain password
password hash
```

Only safe user details should be returned.

What Not To Add Today

Do not add login endpoint.

Do not add JWT generation.

Do not add protected endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add password reset.

Day 54 is only register.

Build Check

Run:

```
dotnet build
```

Expected result:

0 errors

Code Checkpoint

The related commit is:

5a2f615 Day 54: Add register use case

Interview Explanation

A simple explanation:

```
I added a register use case.  
The API checks if the email already exists, hashes the password, stores only PasswordHash, and  
returns a safe response without password data.  
Login and JWT are separate next steps.
```

Lesson Explanation

Day 54 creates the first working authentication use case.

The project already has auth DTOs and a password hashing service. Registration connects those pieces together. The API accepts full name, email, and password. It checks whether the email is already registered, hashes the password, saves the user, and returns a response that does not expose password data.

This is an important security step. The password is used only to create a hash. The database stores PasswordHash, not the original password. The API response returns safe user information and keeps the token empty until JWT generation is implemented later.

The scope stays controlled: register is implemented, but login and JWT are left for later.

Reader Exercise

Test registration with:

- a new email
- the same email again
- an invalid email
- a short password

Confirm:

```
new email creates a user  
duplicate email returns 400  
response does not include password  
response does not include password hash
```

Future Improvement

The next authentication steps can include:

- implementing login
- verifying passwords
- generating JWT tokens
- protecting endpoints
- adding ownership rules
- improving auth error responses

Next Lesson

Day 54 adds registration.

The next lesson can implement login and start using the password verification service.

Day 55 - Add Login Use Case

Lesson Goal

Add the login use case for authentication.

Day 55 validates user credentials.

JWT generation is not implemented yet.

Why This Is Day 55

Day 54 added registration.

Day 55 adds the matching login flow.

Login should:

```
accept email and password
find the user by email
verify the password against PasswordHash
reject invalid credentials
return a safe response without password data
```

This completes the first basic authentication loop: register, then login.

Auth Service Interface Change

IAuthService now includes login:

```
Task<AuthResponse> LoginAsync(LoginRequest request);
```

The interface now describes two authentication use cases:

```
RegisterAsync
LoginAsync
```

Controllers call this interface instead of directly accessing the database or password hashing logic.

Login Logic

AuthService contains the login workflow.

The login flow is:

```
find user by email
return generic error if user does not exist
verify submitted password against PasswordHash
return generic error if password is invalid
return AuthResponse if credentials are valid
```

Password Verification

The password is verified through the password hashing service:

```
_passwordHasherService.VerifyPassword(request.Password, user.PasswordHash)
```

The API does not compare plain text passwords.

It verifies the submitted password against the stored hash.

Generic Error Message

Wrong email and wrong password should return the same message:

```
Invalid email or password.
```

Reason:

```
The API should not reveal whether an email exists.
```

This avoids giving attackers account discovery information.

Auth Response

Successful login returns:

```
UserId  
FullName  
Email  
Token
```

For now:

```
Token = empty string
```

That is acceptable because JWT generation is not implemented yet.

AuthController Change

The controller exposes:

```
POST /api/auth/login
```

If login succeeds:

```
200 OK
```

If credentials are invalid:

```
400 Bad Request  
Invalid email or password.
```

API Test: Correct Credentials

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{  
  "email": "testuser@example.com",  
  "password": "Password123"  
}
```

Expected:

```
200 OK
```

Example response:

```
{  
  "userId": 1,  
  "fullName": "Test User",  
  "email": "testuser@example.com",  
  "token": ""  
}
```

```
}
```

API Test: Wrong Email

Example body:

```
{
  "email": "wrong@example.com",
  "password": "Password123"
}
```

Expected:

```
400 Bad Request
Invalid email or password.
```

API Test: Wrong Password

Example body:

```
{
  "email": "testuser@example.com",
  "password": "WrongPassword"
}
```

Expected:

```
400 Bad Request
Invalid email or password.
```

Security Rules

Do not return:

```
plain password
password hash
different messages for wrong email vs wrong password
```

Use one generic message:

```
Invalid email or password.
```

What Not To Add Today

Do not add JWT generation.

Do not protect endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add password reset.

Do not create a new database migration.

Day 55 is only login credential validation.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
77789ad Day 55: Add login use case
```

Interview Explanation

A simple explanation:

```
I added the login use case.
The API finds a user by email, verifies the submitted password against the stored password hash,
and returns a safe response.
It uses the same error message for wrong email and wrong password so it does not reveal account
existence.
```

Lesson Explanation

Day 55 adds the login use case.

Registration already stores users with hashed passwords. Login now uses those stored hashes to validate credentials. The API finds the user by email, verifies the submitted password, and returns a safe auth response when the credentials are correct.

The important security detail is the error message. The API should not say "email not found" or "wrong password" separately. Both cases return `Invalid email or password`. so the API does not reveal whether an account exists.

JWT generation is still intentionally delayed. The login response has a token field, but the real token will be added in a later step.

Reader Exercise

Test login with:

- correct email and password
- wrong email
- wrong password

Confirm:

```
correct credentials return 200
wrong email returns the generic error
wrong password returns the same generic error
response does not include password
response does not include password hash
```

Future Improvement

The next authentication steps can include:

- JWT generation
- JWT settings
- token claims
- protected endpoints
- authentication middleware setup
- ownership rules for user-owned data

Next Lesson

Day 55 adds login.

The next lesson can start JWT generation so login returns a real token.

Day 56 - Review Auth Controller Endpoints

Lesson Goal

Review and stabilize the authentication controller endpoints.

Day 56 confirms that register and login are exposed through AuthController.

JWT generation is not added yet.

Why This Is Day 56

Day 54 added registration.

Day 55 added login.

Day 56 checks that the API surface is clear and stable:

```
POST /api/auth/register
POST /api/auth/login
```

Both endpoints should call IAuthService, return safe responses, and avoid exposing password data.

AuthController Review

AuthController should expose:

```
POST /api/auth/register
POST /api/auth/login
```

The controller should depend on:

```
IAuthService
```

It should not directly use database code or password hashing logic.

Those details belong inside the auth service.

Register Endpoint

Register endpoint:

```
POST /api/auth/register
```

Example request:

```
{
  "fullName": "Day 56 User",
  "email": "day56@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

The response should not include:

```
password
passwordHash
```

Login Endpoint

Login endpoint:

```
POST /api/auth/login
```

Example request:

```
{
  "email": "day56@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

For now, this is acceptable:

```
{
  "token": ""
}
```

JWT generation will be added later.

Duplicate Register Test

Register the same email again.

Expected:

```
400 Bad Request
Email is already registered.
```

This confirms duplicate email handling still works.

Invalid Login Test

Use the wrong password.

Expected:

```
400 Bad Request
Invalid email or password.
```

Login should use the same generic message for wrong email and wrong password.

Security Rules

Do not return:

```
plain password
password hash
different messages for wrong email and wrong password
```

Login should use:

```
Invalid email or password.
```

This avoids revealing whether an email exists.

Dependency Injection Review

The API should register:


```
builder.Services.AddScoped<IAuthService, AuthService>();
```

This allows `AuthController` to call the auth service through the interface.

The controller stays thin.

The auth service owns the registration and login workflow.

Swagger Review

Swagger should show:

```
POST /api/auth/register
POST /api/auth/login
```

Both endpoints should accept the expected request bodies and return safe auth responses.

What Not To Add Today

Do not add JWT generation.

Do not add authorization attributes.

Do not protect endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add password reset.

Do not create a new database migration.

Day 56 is only auth controller endpoint review and stabilization.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
6f3405d Day 56: Update README auth endpoints
```

Lesson Explanation

Day 56 reviews the authentication controller after registration and login have been added.

At this stage, the API should expose two authentication endpoints: register and login. The controller should be simple. It receives the request, calls `IAuthService`, and returns the appropriate HTTP response.

The important part is separation of responsibility. The controller should not hash passwords, query the database directly, or generate auth logic itself. Those responsibilities belong in the auth service and supporting services.

The review also checks security behavior. Register and login responses must not expose passwords or password hashes. Invalid login should use a generic message so the API does not reveal whether an email exists.

JWT is intentionally still missing. The token field can remain empty until the token generation lesson.

Reader Exercise

Open Swagger and test:

- register with a new email
- register with the same email again
- login with correct credentials
- login with the wrong password

Confirm:

```
register endpoint appears
login endpoint appears
password is never returned
passwordHash is never returned
invalid login uses a generic message
```

Future Improvement

Next authentication steps can include:

- JWT generation
- token settings
- token claims
- authentication middleware
- protected endpoints
- ownership rules
- roles later

Next Lesson

Day 56 stabilizes the auth controller endpoints.

The next lesson can start generating real JWT tokens for successful login.

Day 57 - Add JWT Configuration

Lesson Goal

Prepare the API for JWT token generation.

Day 57 only adds JWT configuration.

It does not generate tokens yet.

Why This Is Day 57

Day 56 reviewed the authentication controller endpoints.

The next authentication step is preparing the API to understand JWT bearer tokens.

Before login can return a real token, the API needs:

```
JWT package
JWT settings
JWT authentication registration
authentication middleware
```

Day 57 adds that setup.

Package Added

The API project uses:

```
Microsoft.AspNetCore.Authentication.JwtBearer
```

This package belongs in the API project because authentication middleware is configured in `Program.cs`.

JWT Settings

The API configuration now includes JWT settings:

```
{
  "Jwt": {
    "Issuer": "JobTrackr",
    "Audience": "JobTrackr",
    "Key": "THIS_IS_A_DEVELOPMENT_SECRET_KEY_CHANGE_LATER"
  }
}
```

These values are used when validating JWT tokens.

For learning, a development key is acceptable.

For production, secrets should not be committed to Git.

Program.cs Configuration

`Program.cs` reads:

```
Jwt:Issuer
Jwt:Audience
Jwt:Key
```

Then it configures JWT bearer authentication.

Important validation settings:

```
ValidateIssuer
ValidateAudience
ValidateLifetime
ValidateIssuerSigningKey
```

This tells ASP.NET Core how to validate tokens later.

Middleware Order

The authentication middleware should be registered before controller mapping:

```
app.UseHttpsRedirection();
app.UseMiddleware<ExceptionHandlerMiddleware>();
app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapControllers();
```

This prepares the request pipeline for protected endpoints later.

What Each Change Does

JWT package:

```
adds support for validating JWT bearer tokens
```

JWT settings:

```
store issuer, audience, and signing key configuration
```

Program.cs:

```
registers JWT authentication services and middleware
```

API Behavior Today

Register should still work.

Login should still work.

Login still returns:

```
{
  "token": ""
}
```

That is expected.

Day 58 can generate the real token.

What Not To Add Today

Do not add JWT token generation.

Do not return a real token from login yet.

Do not add [Authorize] attributes.

Do not protect endpoints.

Do not add refresh tokens.

Do not add roles.

Do not make database changes.

Do not create migrations.

Day 57 is only JWT configuration.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

3b17ce4 Day 57: Add JWT configuration

Interview Explanation

A simple explanation:

I added JWT authentication configuration before generating tokens.
The API now knows how to validate JWT tokens using issuer, audience, and signing key settings.
Token generation will be added separately so the setup stays easy to test.

Lesson Explanation

Day 57 prepares the API for token-based authentication.

The project already has register and login endpoints, but login still returns an empty token. Before generating real tokens, the API must know how JWT validation will work.

This lesson adds the JWT bearer package, configuration values for issuer, audience, and signing key, and the authentication middleware setup in `Program.cs`.

The scope stays controlled. The API can now be prepared for JWT validation, but it does not generate tokens and does not protect endpoints yet.

This keeps authentication implementation step by step: configure first, generate tokens later, protect endpoints after that.

Reader Exercise

Review the JWT configuration and answer:

- where are issuer and audience configured?
- where is the signing key configured?
- why does the API need JWT bearer authentication?
- why should production secrets not be committed?
- why is login still returning an empty token today?

Future Improvement

The next authentication steps can include:

- creating a JWT token service
- generating tokens during login
- adding claims
- protecting endpoints with `[Authorize]`
- reading the current user id from claims
- adding ownership rules

Next Lesson

Day 57 adds JWT configuration.

The next lesson can generate a real JWT token during login.

Day 58 - Generate JWT Token On Login

Lesson Goal

Return a real JWT token after successful login.

Day 58 generates the token.

Protected endpoints are not added yet.

Why This Is Day 58

Day 57 added JWT configuration.

The API now has issuer, audience, signing key, and JWT bearer authentication setup.

Day 58 uses that setup to create a signed token when login succeeds.

The flow becomes:

```
login request
validate email and password
generate JWT token
return token in AuthResponse
```

Files Created

JWT token generation uses two new files:

```
Application/Auth/IJwtTokenService.cs
Infrastructure/Auth/JwtTokenService.cs
```

The interface belongs in Application because it defines what the application needs.

The implementation belongs in Infrastructure because it uses configuration and token-generation details.

IJwtTokenService

The token service interface exposes:

```
string GenerateToken(User user);
```

It accepts the authenticated user and returns a JWT token string.

JwtTokenService

The implementation reads JWT settings from configuration:

```
Jwt:Issuer
Jwt:Audience
Jwt:Key
```

Then it creates claims, signs the token, and returns it as a string.

Token Claims

The generated token includes simple user claims:

```
user id
email
full name
```

These claims can be used later when protected endpoints need to know who the current user is.

Token Signing

The token is signed with:

```
HmacSha256
```

The signing key comes from JWT configuration.

If the key is missing, the service should fail clearly instead of silently generating an invalid token.

AuthService Update

AuthService now depends on:

```
AppDbContext
IPasswordHasherService
IJwtTokenService
```

Login still validates credentials first.

Only after credentials are valid does it generate a token:

```
var token = _jwtTokenService.GenerateToken(user);
```

The login response now returns:

```
Token = token
```

Register Behavior

Register can still return:

```
Token = empty string
```

That is acceptable for Day 58.

Today only login returns a JWT token.

Dependency Injection

The API registers the token service:

```
builder.Services.AddScoped<IJwtTokenService, JwtTokenService>();
```

It should be registered near the other authentication services:

```
IPasswordHasherService
IJwtTokenService
IAuthService
```

API Test: Login

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "day56@example.com",
  "password": "Password123"
}
```

Expected:

```
200 OK
```

Response should contain a real token:

```
{
  "userId": 1,
  "fullName": "Day 56 User",
  "email": "day56@example.com",
  "token": "eyJhbGciOi..."
}
```

The token value should no longer be empty.

What Not To Add Today

Do not add [Authorize].

Do not protect endpoints.

Do not add user-specific task filtering by token.

Do not add refresh tokens.

Do not add roles.

Do not add password reset.

Do not create a database migration.

Day 58 is only JWT token generation on login.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
ce3a362 Day 58: Generate JWT token on login
```

Interview Explanation

A simple explanation:

```
I added JWT token generation during login.
After credentials are valid, the API creates a signed token containing the user id, email, and
full name.
The token is returned in AuthResponse and will be used later to access protected endpoints.
```

Lesson Explanation

Day 58 makes login return a real JWT token.

The project already had registration, login validation, and JWT configuration. This lesson adds the token generation service and connects it to the login flow.

The token service reads issuer, audience, and signing key values from configuration. It creates claims for the authenticated user, signs the token, and returns the token string.

Login now does more than validate credentials. After the email and password are confirmed, the API generates a signed JWT and returns it in `AuthResponse`.

Protected endpoints are intentionally delayed. This keeps the authentication work clear: configure JWT, generate JWT, then protect endpoints later.

Reader Exercise

Login with valid credentials.

Confirm:

```
response is 200 OK
token is not empty
token starts with eyJ
invalid credentials still fail
register still works
```

Then explain why token generation should only happen after password verification succeeds.

Future Improvement

Next authentication steps can include:

- adding [Authorize]
- protecting task and user endpoints
- reading the current user id from token claims
- adding ownership rules
- adding token expiration handling
- improving JWT secret management

Next Lesson

Day 58 generates JWT tokens on login.

The next lesson can start protecting endpoints with authentication.

Day 59 - Protect Task Endpoints

Lesson Goal

Require authentication for task operations.

Day 59 protects task endpoints with JWT authentication.

User endpoints are not protected yet.

Why This Is Day 59

Day 58 made login return a real JWT token.

Day 59 starts using that token to protect API endpoints.

The first protected area is task operations:

```
GET /api/tasks
GET /api/tasks/{id}
```

```
POST /api/tasks
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Auth endpoints must stay public:

```
POST /api/auth/register
POST /api/auth/login
```

Users need those endpoints to get a token.

Main Change

TasksController now uses:

```
[Authorize]
```

This requires a valid JWT bearer token before task endpoints can be used.

The controller should also import:

```
using Microsoft.AspNetCore.Authorization;
```

Controller Header

Expected controller setup:

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class TasksController : ControllerBase
{
}
```

This protects every action inside TasksController.

What Stays Public

Do not add [Authorize] to AuthController.

These endpoints must remain public:

```
POST /api/auth/register
POST /api/auth/login
```

If login were protected, users would not be able to get a token.

Test Without Token

Request:

```
GET /api/tasks
```

Expected:

```
401 Unauthorized
```

This confirms the task endpoint is protected.

Test With Token

First login:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "day56@example.com",
  "password": "Password123"
}
```

Copy the token from the response.

Then call:

```
GET /api/tasks
```

With header:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Expected:

```
200 OK
```

This confirms valid JWT authentication works.

Curl Test Without Token

```
curl -k https://localhost:7024/api/tasks
```

Expected:

```
401 Unauthorized
```

Curl Test With Token

```
curl -k https://localhost:7024/api/tasks -H "Authorization: Bearer YOUR_TOKEN_HERE"
```

Expected:

```
200 OK
```

Swagger Note

If Swagger does not show an authorize button yet, use another client for testing:

```
Postman
Thunder Client
curl
```

Swagger JWT setup can be improved later.

Do not spend Day 59 redesigning Swagger.

What Not To Add Today

Do not add user-specific task ownership from token.

Do not protect user endpoints.

Do not add roles.

Do not add refresh tokens.

Do not add Swagger JWT button setup unless required.

Do not make database changes.

Do not create migrations.

Day 59 is only protecting task endpoints.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Code Checkpoint

The related commit is:

```
9298b61 Day 59: Protect task endpoints
```

Interview Explanation

A simple explanation:

```
I protected task endpoints with JWT authentication by adding [Authorize] to TasksController.
Unauthenticated requests now return 401 Unauthorized.
Authenticated requests with a valid Bearer token can access task endpoints.
Auth endpoints remain public so users can register and login.
```

Lesson Explanation

Day 59 is the first step where JWT authentication actually controls API access.

The project already has registration, login, JWT configuration, and token generation. Now the token is used to protect task endpoints.

Adding [Authorize] to TasksController means every task action requires authentication. Without a token, the API returns 401 Unauthorized. With a valid bearer token, the request can continue.

The auth endpoints remain public because users need to register and log in before they can receive a token.

This lesson does not yet enforce task ownership. It only checks whether the caller is authenticated.

Reader Exercise

Test the protected task API:

- call GET /api/tasks without a token
- confirm 401 Unauthorized
- login and copy the token
- call GET /api/tasks with the bearer token
- confirm 200 OK

Then explain why AuthController should not be protected.

Future Improvement

Next authentication steps can include:

- protecting user endpoints
- adding Swagger JWT support
- reading current user id from claims

- filtering tasks by the authenticated user
- enforcing task ownership
- adding role-based authorization later

Next Lesson

Day 59 protects task endpoints.

The next lesson can continue securing the API by improving token usage or adding ownership rules.

Day 60 - Month 2 Review

Lesson Goal

Review the second month of backend work.

Day 60 is a checkpoint day.

No new feature is added unless something is clearly broken.

Why This Is Day 60

The second month moved JobTrackr from a database-backed API into an API with authentication foundations.

By Day 60, the project should have:

- SQL Server persistence
- EF Core migrations
- user/task relationship
- task filters
- due date and priority fields
- stronger validation
- consistent API responses
- cleaner application layer
- authentication DTOs
- register/login flow
- basic JWT authentication
- protected task endpoints

This review confirms that the current backend foundation is still working before the next phase begins.

Areas Reviewed

The review covers:

- user endpoints
- task endpoints
- auth endpoints
- protected task endpoints
- database persistence
- README accuracy

README Review

The README should mention:

- SQL Server
- EF Core
- users API
- tasks API
- auth register endpoint
- auth login endpoint
- JWT token generation

protected task endpoints

Update README only if it is outdated.

Auth Review

Auth endpoints should exist:

```
POST /api/auth/register
POST /api/auth/login
```

Registration should:

```
hash passwords
reject duplicate emails
return safe user data
avoid returning password or PasswordHash
```

Login should:

```
verify password
return a JWT token
reject invalid credentials
use a generic invalid credential message
```

JWT Review

The token service should include claims for:

```
user id
email
full name
```

The API pipeline should use:

```
app.UseAuthentication();
app.UseAuthorization();
```

This confirms JWT authentication is wired into the request pipeline.

Protected Task Review

TasksController should require authentication.

Without token:

```
GET /api/tasks
```

Expected:

```
401 Unauthorized
```

With a valid bearer token:

```
GET /api/tasks
```

Expected:

```
200 OK
```

Auth Test: Register

Endpoint:

```
POST /api/auth/register
```

Example body:

```
{
  "fullName": "Month Two User",
```

```
"email": "monthtwo@example.com",
"password": "Password123"
}
```

Expected:

200 OK

Response should not include:

```
password
passwordHash
```

Auth Test: Duplicate Register

Use the same email again.

Expected:

```
400 Bad Request
Email is already registered.
```

Auth Test: Login

Endpoint:

POST /api/auth/login

Example body:

```
{
  "email": "monthtwo@example.com",
  "password": "Password123"
}
```

Expected:

200 OK

Response should include a token:

```
{
  "token": "eyJ..."
}
```

User Endpoint Review

Check:

```
GET /api/users
GET /api/users/{id}
POST /api/users
PUT /api/users/{id}
DELETE /api/users/{id}
```

Expected:

```
successful reads -> 200 OK
create -> 201 Created
delete -> 204 No Content
missing user -> 404 Not Found
```

Task Endpoint Review

With a valid token, check:

```
GET /api/tasks
GET /api/tasks?userId=1
GET /api/tasks?isCompleted=false
GET /api/tasks?search=resume
POST /api/tasks
```

```
PUT /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
DELETE /api/tasks/{id}
```

Expected:

```
successful reads -> 200 OK
create -> 201 Created
update -> 200 OK
delete -> 204 No Content
missing task -> 404 Not Found
```

What Not To Add Today

Do not add new auth features.

Do not add roles.

Do not add refresh tokens.

Do not add ownership rules.

Do not add deployment.

Do not add tests.

Do not create migrations.

Day 60 is review only.

Build Check

Run:

```
dotnet build
```

Expected result:

```
0 errors
```

Lesson Explanation

Day 60 is the second monthly checkpoint for JobTrackr.

The first month built the core API foundation. The second month strengthened the backend with SQL Server persistence, relationships, task fields, validation, cleaner responses, architecture cleanup, and authentication basics.

By this point, the API can register users, log them in, return JWT tokens, and protect task endpoints. That is a major backend milestone.

This review day exists to confirm the pieces work together: users, tasks, auth, JWT login, protected task routes, database persistence, and documentation.

The lesson is that long projects need checkpoints. Before starting the next backend phase, verify that the current foundation is stable.

Reader Exercise

Run the Month 2 review checklist:

- register a user
- try duplicate registration

- login and copy the token
- call task endpoints without token
- call task endpoints with token
- test user endpoints
- test task filters
- build the project
- review the README

Write down anything that does not match the expected behavior.

Next Backend Phase

After Day 60, the project is ready for:

- authorization rules
- user-owned tasks
- better Swagger JWT support
- unit testing
- integration testing
- deployment basics

Next Lesson

Day 60 closes the second month.

The next lesson can start the next backend phase from a reviewed, authenticated API foundation.

MONTH 3

OWNERSHIP, TESTING, AND RELIABILITY

Protect user data, automate verification, add CI, and prepare the project for continued growth.

Day 61 - Review Authentication

Lesson Goal

Review the authentication work completed so far.

Day 61 is a verification day. The goal is not to add a new feature. The goal is to prove that the authentication flow still works from registration to protected API access.

Why This Review Matters

Authentication touches several parts of the backend:

- API controllers
- application services
- password hashing
- JWT token generation
- middleware configuration
- protected endpoints
- HTTP status codes

If one part is wired incorrectly, the API can look finished but still fail in real use. This review checks the full flow instead of only checking individual files.

Current Authentication Features

By this point, JobTrackr has:

- auth request and response DTOs
- password hashing
- register use case
- login use case
- JWT settings
- JWT token generation
- protected task endpoints
- anonymous request rejection

This gives the project a basic but realistic authentication foundation.

Files Reviewed

The review focuses on these areas:

- AuthController
- AuthService
- JwtTokenService
- TasksController
- Program

AuthController should expose:

- POST /api/auth/register
- POST /api/auth/login

AuthService should:

- hash passwords during registration
- reject duplicate email addresses
- validate password during login
- return a token after successful login
- avoid returning password or password hash data

JwtTokenService should create a token with useful user claims:

- user id
- email

```
full name
```

TasksController should require authentication for task endpoints:

```
[Authorize]
```

Program should register the authentication middleware in the correct request pipeline:

```
app.UseAuthentication();
app.UseAuthorization();
```

Test 1 - Register A User

Endpoint:

```
POST /api/auth/register
```

Example body:

```
{
  "fullName": "Day 61 User",
  "email": "day61@example.com",
  "password": "Password123"
}
```

Expected result:

```
200 OK
```

The response should not include:

```
password
passwordHash
```

That confirms sensitive password data is not being exposed through the API response.

Test 2 - Reject Duplicate Registration

Register the same email address again.

Expected result:

```
400 Bad Request
Email is already registered.
```

This confirms the API does not allow duplicate users with the same email.

Test 3 - Login With Valid Credentials

Endpoint:

```
POST /api/auth/login
```

Example body:

```
{
  "email": "day61@example.com",
  "password": "Password123"
}
```

Expected result:

```
200 OK
```

The response should include a JWT token:

```
{
  "token": "eyJ..."
}
```

This proves the login flow can authenticate a registered user and issue a bearer token.

Test 4 - Reject Invalid Login

Use the correct email with the wrong password.

Expected result:

```
400 Bad Request
Invalid email or password.
```

The message should stay generic. It should not reveal whether the email exists or only the password is wrong.

Test 5 - Block Anonymous Task Access

Request:

```
GET /api/tasks
```

Do not send an authorization header.

Expected result:

```
401 Unauthorized
```

This confirms task endpoints are no longer open to anonymous users.

Test 6 - Allow Task Access With Token

Use the token returned from login.

Header:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Request:

```
GET /api/tasks
```

Expected result:

```
200 OK
```

This confirms the protected endpoint accepts a valid JWT token.

Build Check

The project should still build successfully:

```
dotnet build
```

Expected result:

```
Build succeeded.
```

The build check matters because authentication changes often involve package references, configuration, middleware, and service registrations.

What This Day Teaches

This lesson teaches that authentication is not one file or one endpoint.

A working backend authentication flow needs:

```
safe registration
password hashing
duplicate user protection
credential validation
token generation
middleware setup
```

```
endpoint authorization
manual verification
```

Review days make the project stronger because they catch integration issues before the next feature is added.

Reader Exercise

Repeat the full authentication test flow:

```
register user
try duplicate registration
login user
try invalid login
call tasks without token
call tasks with token
run the build
```

Then write down which layer handled each part of the flow.

Next Step

After confirming authentication works, the next step can move toward user-specific behavior and stronger authorization rules.

Day 62 - Add User Id Claim Usage

Lesson Goal

Read the authenticated user id from JWT claims.

Day 62 adds a small helper inside the tasks API controller. The goal is not to change task ownership yet. The goal is to prepare the controller so future task operations can know which user is making the request.

Why This Step Matters

Authentication is useful only when the API can connect the current request to a real user.

After login, the API returns a JWT token. That token contains claims about the authenticated user. One of the most important claims is the user id.

This day connects two earlier pieces:

```
Day 58 generated a JWT token with the user id claim.
Day 62 reads that user id claim from the authenticated request.
```

This prepares the project for user-owned task behavior.

What Was Added

The tasks controller now has helper logic that reads the current user id from the authenticated user's JWT claims.

The helper uses:

```
User.FindFirstValue(ClaimTypes.NameIdentifier)
```

That reads the user id stored in the JWT token.

The value is parsed into an integer:

```
if (int.TryParse(userIdValue, out var userId))
```

```
{  
    return userId;  
}
```

If the claim is missing or invalid, the helper returns:

```
null
```

This keeps the method defensive and avoids throwing an exception for a malformed or missing claim.

Code Shape

The helper method is simple:

```
private int? GetCurrentUserId()  
{  
    var userIdValue = User.FindFirstValue(ClaimTypes.NameIdentifier);  
  
    if (int.TryParse(userIdValue, out var userId))  
    {  
        return userId;  
    }  
  
    return null;  
}
```

The controller also needs:

```
using System.Security.Claims;
```

This import provides access to `ClaimTypes` and the claim helper extension method.

What Did Not Change

Day 62 intentionally does not change task behavior.

No changes were made to:

```
task ownership  
create task request shape  
task service behavior  
database schema  
migrations  
roles  
refresh tokens  
new endpoints
```

This keeps the lesson focused. The project now knows how to read the current user id, but it does not yet use that id to filter or assign tasks.

Why Use `ClaimTypes.NameIdentifier`

The JWT token was created with the user id stored as:

```
ClaimTypes.NameIdentifier
```

Reading the same claim type in the controller keeps both sides consistent.

The token generation side writes the claim. The controller side reads the claim.

That gives the API a clean path from login to authenticated request context.

Verification

The project should still build successfully:

```
dotnet build
```

Expected result:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

This confirms the helper method, namespace import, and controller changes compile correctly.

What This Day Teaches

This lesson teaches an important backend pattern:

```
Do not pass user id from the client when the API can read it from the authenticated token.
```

Once a user is authenticated, the API should trust the token context more than request body fields for identity decisions.

That prevents users from sending another user's id in a request and accidentally or deliberately working with data that is not theirs.

Reader Exercise

Find where the JWT token is created and confirm the user id is added as a `NameIdentifier` claim.

Then find the tasks controller helper and confirm it reads the same claim type.

The important connection is:

```
token creation claim type = token reading claim type
```

Next Step

The next step is to use the current user id when creating or reading tasks, so task data can belong to the authenticated user instead of being controlled by request body values.

Day 63 - Make Tasks User-Owned

Lesson Goal

Make newly created tasks belong to the logged-in user.

Day 63 changes task creation only. The API now reads the user id from the authenticated JWT token instead of trusting a `UserId` value sent in the request body.

Why This Matters

Before authentication, `CreateTaskRequest` accepted `UserId` because the API did not yet know who the current user was.

After adding JWT authentication, the backend can identify the logged-in user from the token. That means the client should not decide which user owns a new task.

This is an important security improvement:

```
The API should use authenticated identity for ownership decisions.  
The request body should not be trusted for user identity.
```

Without this change, a user could send another user's id while creating a task.

What Changed

Task creation now follows this flow:

```
user logs in
API returns JWT token
client sends token when creating a task
TasksController reads current user id from token claims
TaskService creates the task with that authenticated user id
```

The task still has a `UserId`, but that value now comes from the authenticated request context.

Request Model Change

`CreateTaskRequest` no longer needs `UserId`.

Old request idea:

```
{
  "title": "Task title",
  "description": "Task description",
  "userId": 1,
  "dueDateUtc": null,
  "priority": "Medium"
}
```

New request idea:

```
{
  "title": "Task title",
  "description": "Task description",
  "dueDateUtc": null,
  "priority": "Medium"
}
```

This makes the API cleaner and safer. The client sends task information, while the server decides ownership.

Service Contract Change

The task service create method now accepts the authenticated user id separately:

```
Task<TaskResponse> CreateTaskAsync(CreateTaskRequest request, int userId);
```

This makes the method explicit.

The request contains task data. The `userId` parameter contains identity data from the authenticated user.

Controller Flow

The create endpoint reads the current user id first:

```
var currentUserId = GetCurrentUserId();
```

If no valid user id exists, the API returns:

```
401 Unauthorized
```

If the user id exists, the controller passes it to the service:

```
var response = await _taskService.CreateTaskAsync(request, currentUserId.Value);
```

This keeps the controller responsible for authenticated request context and keeps the service focused on task creation rules.

Task Service Flow

The task service uses the `userId` parameter:

```
UserId = userId
```

It should not use:

```
UserId = request.UserId
```

The service can also validate that the user exists before saving the task.

Swagger Result

Swagger should no longer show `userId` in the create task request body.

That is a useful sign that the API contract has improved:

```
Clients create tasks.  
The authenticated backend assigns ownership.
```

Test Flow

First, log in:

```
POST /api/auth/login
```

Then create a task with the bearer token:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

Request:

```
{  
  "title": "User owned task",  
  "description": "Created from authenticated user",  
  "dueDateUtc": null,  
  "priority": "Medium"  
}
```

Expected result:

```
201 Created
```

The response should include a `userId` matching the logged-in user.

If the same request is sent without a token, the API should reject it:

```
401 Unauthorized
```

What Did Not Change

Day 63 does not yet restrict every task read, update, or delete operation by user.

This day only changes task creation.

That keeps the lesson focused and makes it easier to verify one behavior at a time.

What This Day Teaches

This lesson teaches a core backend rule:

```
Authenticated user identity should come from the server-side auth context, not from client-  
submitted fields.
```

That rule becomes more important as the API grows. User-owned data, authorization checks, and secure multi-user behavior all depend on it.

Reader Exercise

Create a task using a logged-in user's token and confirm the returned `userId`.

Then try to send a fake `userId` in the request body. The API should no longer need that field for task creation.

Next Step

The next step is to apply authenticated user ownership to more task operations, such as reading only the current user's tasks or blocking updates to another user's task.

Day 64 - Restrict Get All Tasks

Lesson Goal

Return only tasks owned by the logged-in user.

Day 64 changes only the `GET /api/tasks` endpoint. The endpoint still supports filtering, but task ownership now comes from the authenticated JWT token instead of a public query parameter.

Why This Matters

Before this change, the task list endpoint could accept a `userId` query parameter.

Example:

```
GET /api/tasks?userId=2
```

That is risky after authentication exists. A logged-in user should not be able to request another user's tasks by changing a query string value.

The backend should decide ownership from the authenticated request:

```
JWT token -> current user id -> only that user's tasks
```

This is a key step toward secure multi-user behavior.

What Changed

The public task list endpoint now:

```
reads the current user id from JWT claims
returns only tasks owned by that user
keeps isCompleted filtering
keeps title search
removes userId as a public query parameter
```

The API contract becomes safer because clients can no longer choose which user's task list to request.

Service Contract Change

The task service method changed from an optional user filter:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int? userId);
```

To a required authenticated user id:

```
Task<List<TaskResponse>> GetAllAsync(bool? isCompleted, string? search, int userId);
```

This makes ownership explicit. Getting all tasks now means getting all tasks for the authenticated user.

Query Behavior

The query starts with the ownership filter:

```
var query = _dbContext.Tasks
    .Where(task => task.UserId == userId);
```

Then the optional filters are applied inside that owned task set:

```
if (isCompleted is not null)
{
    query = query.Where(task => task.IsCompleted == isCompleted);
}

if (!string.IsNullOrEmpty(search))
{
    query = query.Where(task => task.Title.Contains(search));
}
```

The order matters conceptually:

```
first restrict by owner
then filter that owner's tasks
```

Controller Flow

The controller no longer accepts `userId` as a query parameter.

Old shape:

```
public async Task<IActionResult> GetAll(
    bool? isCompleted,
    string? search,
    int? userId)
```

New shape:

```
public async Task<IActionResult> GetAll(bool? isCompleted, string? search)
```

The controller reads the current user id from the authenticated request:

```
var currentUserId = GetCurrentUserid();
```

If the user id is missing, the API returns:

```
401 Unauthorized
```

If the user id is valid, it is passed into the task service:

```
var tasks = await _taskService.GetAllAsync(
    isCompleted,
    search,
    currentUserId.Value);
```

Swagger Result

Swagger should still show:

```
isCompleted
search
```

Swagger should no longer show:

```
userId
```

That is an important API design improvement. The client can filter tasks, but it cannot choose the owner.

Test Flow

Create tasks for two different users.

Then call the task list endpoint as user one:

```
GET /api/tasks
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

The response should contain only user one's tasks.

Call the same endpoint as user two:

```
GET /api/tasks
Authorization: Bearer USER_TWO_TOKEN
```

Expected:

```
200 OK
```

The response should contain only user two's tasks.

Filter Test

Filtering should still work inside the logged-in user's own task list.

Example:

```
GET /api/tasks?isCompleted=false&search=resume
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

The response should include only user one's incomplete tasks that match the search term.

Anonymous Request Test

Request:

```
GET /api/tasks
```

Expected:

```
401 Unauthorized
```

This confirms the endpoint is protected and depends on authenticated identity.

What Did Not Change

Day 64 does not change ownership rules for:

```
get task by id
update task
delete task
complete task
reopen task
```

Those endpoints need their own focused authorization checks.

What This Day Teaches

This lesson teaches a practical authorization rule:

```
Filtering by owner should happen on the server from authenticated identity.
```

Query parameters are useful for search and status filters. They should not be used to decide which user's private data is visible.

Reader Exercise

Create tasks for two users.

Log in as each user and call:

```
GET /api/tasks
```

Confirm each user sees only their own tasks.

Then verify that `isCompleted` and `search` still work inside the authenticated user's own task list.

Next Step

The next step is to continue applying ownership rules to more task endpoints, especially reading a single task by id and blocking access to another user's task.

Day 65 - Restrict Get Task By Id

Lesson Goal

Return a task only when it belongs to the logged-in user.

Day 65 changes only the `GET /api/tasks/{id}` endpoint. The endpoint now checks both the task id and the authenticated user id before returning task data.

Why This Matters

Before this change, the service could search for a task by id only:

```
await _dbContext.Tasks.FindAsync(id);
```

That is not enough in a multi-user API.

If the API only checks the task id, a logged-in user could change the id in the URL and potentially read another user's task.

The safer rule is:

```
task id must match  
owner user id must also match
```

This prevents users from reading private task data that does not belong to them.

What Changed

The `Get By Id` flow now uses authenticated ownership:

```
controller reads current user id from JWT claims  
controller passes task id and user id to the service  
service queries by both task id and user id  
API returns the task only when both values match
```

If the task is missing or belongs to another user, the API returns the same result:

```
404 Not Found
```

Service Contract Change

The task service method changed from:

```
Task<TaskResponse?> GetByIdAsync(int id);
```

To:

```
Task<TaskResponse?> GetByIdAsync(int id, int userId);
```

The authenticated user id is now required because the service must verify ownership before returning the task.

Query Change

The task lookup should use both task id and user id:

```
var task = await _dbContext.Tasks
    .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);
```

This replaces id-only lookup logic.

The important difference is:

```
FindAsync(id) checks only the task id.
FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId) checks both task id and
owner.
```

Controller Flow

The controller reads the authenticated user id:

```
var currentUserId = GetCurrentUserid();
```

If no valid user id exists, the API returns:

```
401 Unauthorized
```

If a user id exists, the controller passes it into the service:

```
var response = await _taskService.GetByIdAsync(id, currentUserId.Value);
```

If no matching task is found, the controller returns:

```
404 Not Found
```

If the task exists and belongs to the logged-in user, the controller returns:

```
200 OK
```

Why Return 404 For Another User's Task

The API should not reveal whether another user's task exists.

That is why both cases use the same response:

```
missing task -> 404 Not Found
another user's task -> 404 Not Found
```

This is safer than returning a different response that confirms the task exists but is owned by someone else.

Test Flow

Create two users and get a JWT token for each user.

Create a task while authenticated as user one.

Then test the owner request:

```
GET /api/tasks/1
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

Now test another user requesting the same task:

```
GET /api/tasks/1
Authorization: Bearer USER_TWO_TOKEN
```

Expected:

```
404 Not Found
```

Then test a missing task:

```
GET /api/tasks/99999
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

Finally, test without a token:

```
GET /api/tasks/1
```

Expected:

```
401 Unauthorized
```

What Did Not Change

Day 65 does not change ownership rules for:

```
update task
delete task
complete task
reopen task
```

Those endpoints still need their own authorization checks.

What This Day Teaches

This lesson teaches a core multi-user backend rule:

```
Every private resource lookup must include the authenticated owner condition.
```

For user-owned data, checking the resource id alone is not enough.

Reader Exercise

Find every endpoint that reads or changes a task by id.

Ask this question for each endpoint:

```
Does this query check both task id and authenticated user id?
```

If the answer is no, that endpoint still needs an ownership restriction.

Next Step

The next step is to continue applying ownership checks to task mutation endpoints such as update, delete, complete, and reopen.

Day 66 - Restrict Update Task

Lesson Goal

Allow a task to be updated only by the authenticated user who owns it.

Day 66 changes only the PUT `/api/tasks/{id}` endpoint. The update operation now checks both the task id and the authenticated user id before changing task data.

Why This Matters

An update endpoint is more sensitive than a read endpoint because it changes data.

Before this change, the task service could find a task by id only:

```
await _dbContext.Tasks.FindAsync(id);
```

That is not enough for user-owned data.

If a logged-in user can guess or change another task id in the URL, they could update another user's task unless the backend checks ownership.

The correct rule is:

```
task id must match  
authenticated user id must also match
```

What Changed

The update flow now works like this:

```
client sends PUT /api/tasks/{id}  
controller reads the current user id from JWT claims  
controller passes id, request, and user id to the service  
service searches by both task id and owner user id  
service updates the task only when ownership matches
```

If the task is missing or belongs to another user, the API returns:

```
404 Not Found
```

Service Contract Change

The task service update method changed from:

```
Task<TaskResponse> UpdateTaskAsync(int id, UpdateTaskRequest request);
```

To:

```
Task<TaskResponse> UpdateTaskAsync(int id, UpdateTaskRequest request, int userId);
```

The service needs `userId` because ownership must be checked before applying updates.

Query Change

The service should not use id-only lookup for updates.

Instead of:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Use:

```
var task = await _dbContext.Tasks  
    .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);
```

This makes the update operation ownership-aware.

Validation Still Applies

The endpoint should still validate update data.

Examples:

```
title is required
priority is required
```

Invalid request data should return:

```
400 Bad Request
```

Ownership checks and validation are both needed. Ownership decides whether the user can update the task. Validation decides whether the submitted task data is acceptable.

Controller Flow

The controller gets the current user id:

```
var currentUserId = GetCurrentUserId();
```

If the JWT token is missing or invalid, the API returns:

```
401 Unauthorized
```

Then the controller calls the service:

```
var response = await _taskService.UpdateTaskAsync(
    id,
    request,
    currentUserId.Value);
```

If the service returns null, the controller returns:

```
404 Not Found
```

If the task belongs to the logged-in user and the request is valid, the API returns:

```
200 OK
```

Why Return 404 For Another User's Task

The API should not reveal whether another user's task exists.

These two cases should look the same to the caller:

```
task does not exist
task exists but belongs to another user
```

Both return:

```
404 Not Found
```

This avoids leaking private resource information.

Test Flow

Create two users and obtain a JWT token for each user.

Create a task while authenticated as user one.

Then update it as the owner:

```
PUT /api/tasks/1
Authorization: Bearer USER_ONE_TOKEN
Content-Type: application/json
```

Body:

```
{
  "title": "Updated owned task",
  "description": "Updated by the owner",
  "dueDateUtc": null,
  "priority": "High"
}
```

Expected:

```
200 OK
```

Now try the same task id with user two's token:

```
PUT /api/tasks/1
Authorization: Bearer USER_TWO_TOKEN
Content-Type: application/json
```

Expected:

```
404 Not Found
```

The task should remain unchanged.

Anonymous Request Test

Request:

```
PUT /api/tasks/1
```

Expected:

```
401 Unauthorized
```

This confirms update requires authentication.

What Did Not Change

Day 66 does not change ownership rules for:

```
delete task
complete task
reopen task
```

Those mutation endpoints still need separate ownership checks.

What This Day Teaches

This lesson teaches a practical authorization rule:

```
Every write operation on user-owned data must verify ownership before changing data.
```

Reading private data needs ownership checks. Updating private data needs them even more.

Reader Exercise

Try to update a task with the owner's token and confirm the response is 200 OK.

Then try to update the same task with another user's token and confirm the response is 404 Not Found.

Finally, verify that invalid update data still returns 400 Bad Request.

Next Step

The next step is to apply the same ownership pattern to delete, complete, and reopen task endpoints.

Day 67 - Restrict Delete Task

Lesson Goal

Allow a task to be deleted only by the authenticated user who owns it.

Day 67 changes only the DELETE /api/tasks/{id} endpoint. The delete operation now checks both the task id and authenticated user id before removing task data.

Why This Matters

Delete endpoints are high risk because they permanently remove data.

Before this change, the task service could find a task by id only:

```
await _dbContext.Tasks.FindAsync(id);
```

That is unsafe for user-owned data.

If a logged-in user can change the id in the URL and the backend does not check ownership, that user could delete another user's task.

The correct rule is:

```
task id must match  
authenticated user id must also match
```

What Changed

The delete flow now works like this:

```
client sends DELETE /api/tasks/{id}  
controller reads the current user id from JWT claims  
controller passes id and user id to the service  
service searches by both task id and owner user id  
service deletes the task only when ownership matches
```

If the task is missing or belongs to another user, the API returns:

```
404 Not Found
```

If the owner deletes the task successfully, the API returns:

```
204 No Content
```

Service Contract Change

The task service delete method changed from:

```
Task<bool> DeleteTaskAsync(int id);
```

To:

```
Task<bool> DeleteTaskAsync(int id, int userId);
```

The service needs userId because deletion must verify ownership before removing the row.

Query Change

The service should not use id-only lookup for deletion.

Instead of:

```
var task = await _dbContext.Tasks.FindAsync(id);
```

Use:

```
var task = await _dbContext.Tasks
    .FirstOrDefaultAsync(task => task.Id == id && task.UserId == userId);
```

If this query returns null, the task either does not exist or does not belong to the logged-in user.

Both cases return false from the service, and the controller converts that into:

```
404 Not Found
```

Controller Flow

The controller reads the current user id:

```
var currentUserId = GetCurrentUserId();
```

If the JWT token is missing or invalid, the API returns:

```
401 Unauthorized
```

Then the controller calls the service:

```
var isDeleted = await _taskService.DeleteTaskAsync(
    id,
    currentUserId.Value);
```

If the service returns false, the controller returns:

```
404 Not Found
```

If the delete succeeds, the controller returns:

```
204 No Content
```

Why 204 No Content

For a successful delete, the API does not need to return the deleted task.

204 No Content means:

```
the request succeeded
there is no response body
```

That is a standard response for successful delete operations.

Why Return 404 For Another User's Task

The API should not reveal whether another user's task exists.

These two cases should look the same:

```
task does not exist
task exists but belongs to another user
```

Both return:

```
404 Not Found
```

This avoids leaking private resource information.

Test Flow

Create two users and obtain a JWT token for each user.

Create a task while authenticated as user one.

First, try deleting it as another user:

```
DELETE /api/tasks/1
Authorization: Bearer USER_TWO_TOKEN
```

Expected:

```
404 Not Found
```

The task should still exist for user one.

Then delete it as the owner:

```
DELETE /api/tasks/1
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
204 No Content
```

The response should not contain a body.

After deletion, requesting the same task should return:

```
404 Not Found
```

Anonymous Request Test

Request:

```
DELETE /api/tasks/1
```

Expected:

```
401 Unauthorized
```

This confirms deletion requires authentication.

What Did Not Change

Day 67 does not change ownership rules for:

```
complete task
reopen task
```

Those endpoints still need separate ownership checks.

What This Day Teaches

This lesson teaches a direct authorization rule:

```
Before deleting user-owned data, verify the authenticated user owns the resource.
```

For delete operations, ownership checks are not optional. They protect user data from accidental or malicious deletion.

Reader Exercise

Delete a task with the owner's token and confirm the response is 204 No Content.

Then try to delete another user's task and confirm the response is 404 Not Found.

Finally, confirm the task still exists for the correct owner after the failed delete attempt.

Next Step

The next step is to apply the same ownership pattern to complete and reopen task endpoints.

Day 68 - Review User Ownership

Lesson Goal

Review task ownership with two different users and confirm that the protections added during Days 63-67 work together.

Day 68 is mainly a testing and review day. The goal is not to add a new architecture or feature. The goal is to verify that the user ownership rules behave correctly across the main task endpoints.

What Is Reviewed

The review checks that authenticated users can:

```
create tasks only for themselves
list only their own tasks
retrieve only their own task by id
update only their own task
delete only their own task
```

This confirms that the API is moving from simple authentication toward practical authorization.

Important Current Limitation

The complete and reopen endpoints are protected by JWT authentication, but they do not yet check task ownership.

These endpoints are not part of the Day 68 ownership confirmation:

```
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen
```

Until those endpoints also check the authenticated user id, the project should not claim that every task endpoint is owner-restricted.

Ownership Test Setup

The review uses two users.

Example user one:

```
{
  "fullName": "Ownership User One",
  "email": "owner.one@example.com",
  "password": "Password123"
}
```

Example user two:

```
{
  "fullName": "Ownership User Two",
  "email": "owner.two@example.com",
  "password": "Password123"
}
```

After registering both users, log in as each user and keep the returned JWT tokens:

```
USER_ONE_TOKEN
USER_TWO_TOKEN
```

Swagger can authorize one token at a time, so the token must be switched when testing each user's behavior.

Create Task Ownership

Create one task while authenticated as user one:

```
{
  "title": "User one private task",
  "description": "Owned by user one",
  "dueDateUtc": null,
  "priority": "Medium"
}
```

Then create one task while authenticated as user two:

```
{
  "title": "User two private task",
  "description": "Owned by user two",
  "dueDateUtc": null,
  "priority": "High"
}
```

The request body should not accept `userId`.

The response should include a `userId` that matches the logged-in user.

This confirms task ownership is assigned by the backend from JWT identity, not from client-submitted data.

Get All Ownership

When user one calls:

```
GET /api/tasks
Authorization: Bearer USER_ONE_TOKEN
```

The response should include only user one's tasks.

When user two calls the same endpoint with `USER_TWO_TOKEN`, the response should include only user two's tasks.

The filters should still work inside the logged-in user's task list:

```
GET /api/tasks?isCompleted=false
GET /api/tasks?search=private
```

This confirms filtering works after ownership is applied.

Get By Id Ownership

When user one requests their own task:

```
GET /api/tasks/USER_ONE_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
200 OK
```

When user one requests user two's task:

```
GET /api/tasks/USER_TWO_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

This confirms task id lookup is restricted by owner.

Update Ownership

When user one tries to update user two's task:

```
PUT /api/tasks/USER_TWO_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

The task should remain unchanged.

When user two updates the same task with their own token:

```
200 OK
```

This confirms update operations are owner-restricted.

Delete Ownership

When user one tries to delete user two's task:

```
DELETE /api/tasks/USER_TWO_TASK_ID
Authorization: Bearer USER_ONE_TOKEN
```

Expected:

```
404 Not Found
```

The task should still exist for user two.

When user two deletes their own task:

```
204 No Content
```

This confirms delete operations are owner-restricted.

Ownership Checklist

Use this checklist during the review:

```
Create Task uses the authenticated user id
Get All returns only the authenticated user's tasks
Completion and search filters stay inside the user's task list
Get By Id returns 404 for another user's task
Update returns 404 for another user's task
An unsuccessful update does not change the task
Delete returns 404 for another user's task
An unsuccessful delete does not remove the task
Owner delete returns 204 No Content
Requests without a token return 401 Unauthorized
```

README Accuracy

The project README should describe the current API behavior accurately.

The README should mention:

```
JWT token generation on login
protected task endpoints with JWT authentication
created tasks belong to the authenticated user
task listing, retrieval, update, and deletion are restricted to the task owner
```

The README should not show old examples that use `userId` as a public task list query parameter.

For create task behavior, the README should explain:

```
POST /api/tasks uses the authenticated user id from the JWT token and returns 201 Created.
```

For login behavior, the README should explain:

```
Successful login returns a JWT token for authenticating protected task endpoints.
```

What This Day Teaches

This review teaches that authorization is not one endpoint.

For user-owned data, each endpoint needs to answer a direct question:

```
Is the authenticated user allowed to access or change this resource?
```

Days 63-67 added this pattern step by step for task creation, listing, retrieval, update, and deletion.

Reader Exercise

Run the full two-user ownership test.

Use separate tokens for user one and user two. Create one task per user, then test list, get by id, update, and delete behavior from both identities.

Write down every endpoint that returns 404 Not Found for another user's task.

Next Step

The next step is to apply ownership protection to the complete and reopen endpoints so task status changes also respect authenticated ownership.

Day 69 - Add Basic Unit Test Project

Lesson Goal

Create the first automated test project and add one simple service test.

Day 69 starts the testing phase of JobTrackr. The goal is to add an xUnit test project, connect it to the solution, and write the first small unit test for password hashing.

Why This Matters

Manual testing through Swagger is useful while building early features, but it does not scale well.

As the backend grows, the project needs automated tests that can be run repeatedly:

```
before commits  
before refactoring  
before adding new features  
before publishing code
```

Automated tests help catch regressions earlier.

What Is A Unit Test

A unit test checks a small piece of code automatically.

For this first test, the project does not test database behavior. It tests PasswordHasherService because password hashing is isolated and does not require EF Core or SQL Server.

The test verifies two things:

```
the password hash is not the original password
```

the correct password verifies successfully against the hash

Project Structure

The test project is placed under `tests`, outside the production `src` folder:

```
JobTrackr
|-- src
|   |-- JobTrackr.Api
|   |-- JobTrackr.Application
|   |-- JobTrackr.Domain
|   |-- JobTrackr.Infrastructure
|-- tests
|   |-- JobTrackr.Tests
|       |-- Auth
|           |-- PasswordHasherServiceTests.cs
```

This keeps test code separate from the running API code.

Test Project Setup

The xUnit project is created with:

```
dotnet new xunit -n JobTrackr.Tests -o tests\JobTrackr.Tests --framework net8.0
```

Then the test project is added to the solution:

```
dotnet sln JobTrackr.slnx add tests\JobTrackr.Tests\JobTrackr.Tests.csproj
```

The test project references the Application layer:

```
dotnet add tests\JobTrackr.Tests\JobTrackr.Tests.csproj reference src\JobTrackr.Application\
JobTrackr.Application.csproj
```

That allows the test project to use application services such as `PasswordHasherService`.

First Test File

The generated example test is removed, and a real auth test is added:

```
tests\JobTrackr.Tests\Auth\PasswordHasherServiceTests.cs
```

The test class focuses on password hashing behavior.

First Test

The first test is:

```
using JobTrackr.Application.Auth;

namespace JobTrackr.Tests.Auth
{
    public class PasswordHasherServiceTests
    {
        [Fact]
        public void HashAndVerifyPassword_WithCorrectPassword_ReturnsTrue()
        {
            var passwordHasher = new PasswordHasherService();
            const string password = "Password123";

            var passwordHash = passwordHasher.HashPassword(password);
            var isValid = passwordHasher.VerifyPassword(password, passwordHash);

            Assert.NotEqual(password, passwordHash);
            Assert.True(isValid);
        }
    }
}
```

This test creates the password hasher, hashes a password, verifies the password, and checks the result.

What The Assertions Mean

This assertion confirms the stored value is not the plain password:

```
Assert.NotEqual(password, passwordHash);
```

This assertion confirms the correct password can be verified:

```
Assert.True(isValid);
```

Together, these checks confirm a basic but important authentication behavior.

Test Naming

The test name follows this pattern:

```
MethodOrBehavior_Scenario_ExpectedResult
```

The test name is:

```
HashAndVerifyPassword_WithCorrectPassword_ReturnsTrue
```

That makes the purpose of the test readable without opening the implementation.

Commands To Verify

Run all tests:

```
dotnet test
```

Expected result:

```
Passed: 1  
Failed: 0
```

Then build the solution:

```
dotnet build
```

Expected result:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

What Did Not Change

Day 69 intentionally avoids extra testing complexity.

No changes are made to:

```
production authentication code  
EF Core test setup  
TaskService tests  
mocking libraries  
database behavior
```

The first test stays simple so the testing foundation is easy to understand.

What This Day Teaches

This lesson teaches that testing should start with a small, isolated behavior.

Password hashing is a good first unit test because it has clear input and output:

```
input: plain password  
output: hash and verification result
```

It does not need controllers, HTTP requests, Swagger, a database, or authentication middleware.

Reader Exercise

Run the test project and confirm that one test passes.

Then add a second test that verifies the wrong password does not validate against the generated hash.

Expected idea:

```
HashAndVerifyPassword_WithWrongPassword_ReturnsFalse
```

Next Step

The next step is to begin testing task service behavior, where the project will need to think carefully about database setup and test isolation.

Day 70 - Test Task Creation

Lesson Goal

Add automated tests for creating a task through TaskService.

Day 70 continues the testing phase started on Day 69. The goal is to test task creation with a temporary EF Core in-memory database, without connecting to SQL Server and without changing production code.

What Is Tested

The tests focus on two behaviors:

```
valid task creation  
empty task title validation
```

This keeps the lesson focused on the create task flow.

Why Use An In-Memory Database

TaskService depends on AppDbContext, so the service needs a database context during testing.

The EF Core in-memory provider creates a temporary database inside the test process.

That means the tests can run without:

```
SQL Server  
real application data  
manual database setup  
Swagger  
HTTP requests
```

These are service-level tests using the real TaskService and a temporary AppDbContext.

Test Project Updates

The test project now references the Infrastructure project:

```
dotnet add tests\JobTracker.Tests\JobTracker.Tests.csproj reference src\JobTracker.Infrastructure\JobTracker.Infrastructure.csproj
```

This gives the test project access to:

```
TaskService  
AppDbContext  
Infrastructure dependencies
```

The test project also uses EF Core InMemory:

```
dotnet add tests\JobTrackr.Tests\JobTrackr.Tests.csproj package Microsoft.EntityFrameworkCore.InMemory --version 8.0.27
```

The version should match the EF Core version used by the project.

Test File

The task creation tests are placed in:

```
tests\JobTrackr.Tests\Tasks\TaskServiceCreateTests.cs
```

The file contains tests for valid creation and title validation.

Fresh Database Per Test

Each test creates a fresh in-memory database:

```
private static AppDbContext CreateDbContext()
{
    var options = new DbContextOptionsBuilder<AppDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;

    return new AppDbContext(options);
}
```

Using a unique database name matters:

```
data from one test cannot affect another test
test order does not matter
each test starts clean
```

Seeding A User

Task creation requires a valid user id.

The test adds a user first:

```
private static async Task<User> AddUserAsync(AppDbContext dbContext)
{
    var user = new User
    {
        FullName = "Task Test User",
        Email = "task.test@example.com",
        PasswordHash = "test-password-hash"
    };

    dbContext.Users.Add(user);
    await dbContext.SaveChangesAsync();

    return user;
}
```

This gives the test a real user row for the task ownership check.

Valid Task Creation Test

The valid test creates:

```
database context
test user
real TaskService
CreateTaskRequest
```

Then it calls:

```
var response = await taskService.CreateTaskAsync(request, user.Id);
```

The important assertions are:

```
Assert.Equal(request.Title, response.Title);
Assert.Equal(request.Description, response.Description);
Assert.Equal(request.Priority, response.Priority);
Assert.Equal(user.Id, response.UserId);
Assert.False(response.IsCompleted);
Assert.Equal(1, await dbContext.Tasks.CountAsync());
```

These checks confirm that the task was created, assigned to the correct user, starts incomplete, and was saved to the database context.

Empty Title Test

The second test sends an invalid request with an empty title.

It expects:

```
Assert.ThrowsAsync<ArgumentException>
```

Then it confirms the exact validation message:

```
Assert.Equal(ErrorMessages.TaskTitleRequired, exception.Message);
```

Finally, it confirms the invalid task was not saved:

```
Assert.Empty(dbContext.Tasks);
```

This is important because validation should stop bad data before it reaches storage.

Expected Test Result

After Day 70, running:

```
dotnet test
```

Should show three passing tests:

```
password hashing test
valid task creation test
empty title validation test
```

Expected summary:

```
Passed: 3
Failed: 0
```

What Did Not Change

Day 70 intentionally avoids extra complexity.

No changes are made to:

```
production service code
SQL Server test setup
mocking libraries
controller tests
TaskService update/delete tests
```

The tests stay focused on create task behavior.

What This Day Teaches

This lesson teaches how to test a service that depends on EF Core.

The key idea is:

```
use the real service
use a temporary database
```

```
seed only the data required for the test
assert both the response and saved data
```

That gives useful confidence without requiring a full API or SQL Server setup.

Reader Exercise

Add one more create task test:

```
CreateTaskAsync_WithEmptyPriority_ThrowsArgumentException
```

It should verify that an empty priority throws the expected error and does not save a task.

Next Step

The next step is to expand task service tests beyond creation, such as testing task listing, filtering, or ownership behavior.

Day 71 - Test Task Get By Id

Lesson Goal

Add automated tests for retrieving one task through `TaskService.GetByIdAsync`.

Day 71 continues the service testing work from Day 70. The goal is to test the Get By Id behavior with an EF Core in-memory database while keeping the tests focused and simple.

What Is Tested

The tests cover two cases:

```
an existing task owned by the user is returned
a missing task returns null
```

These are core behaviors for the GET `/api/tasks/{id}` API flow.

No New Packages

Day 70 already added the required test setup:

```
Infrastructure project reference
Microsoft.EntityFrameworkCore.InMemory 8.0.27
```

Day 71 does not install new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Tasks\TaskServiceGetByIdTests.cs
```

It tests `TaskService.GetByIdAsync` using a temporary in-memory database.

Existing Owned Task Test

The first test creates a user and a task:

```
var user = new User
{
    FullName = "Get Task Test User",
    Email = "get.task@example.com",
```



```
        PasswordHash = "test-password-hash"
    };
```

Then it creates a task owned by that user:

```
var task = new JobTask
{
    Title = "Review Get By Id",
    Description = "Test existing task behavior",
    Priority = "High",
    User = user
};
```

Using `User = user` creates the relationship between the task and the user. EF Core assigns the ids when the data is saved.

Calling The Service

The test creates the real service:

```
var taskService = new TaskService(dbContext);
```

Then it calls:

```
var response = await taskService.GetByIdAsync(task.Id, user.Id);
```

The method receives both:

```
task id
authenticated user id
```

That matches the ownership-aware behavior added earlier.

Assertions

The test first confirms the response exists:

```
Assert.NotNull(response);
```

Then it confirms the returned data is mapped correctly:

```
Assert.Equal(task.Id, response.Id);
Assert.Equal(task.Title, response.Title);
Assert.Equal(task.Description, response.Description);
Assert.Equal(task.Priority, response.Priority);
Assert.Equal(user.Id, response.UserId);
```

This proves the service can retrieve an owned task and return the expected response model.

Missing Task Test

The second test uses a fresh empty database and calls:

```
var response = await taskService.GetByIdAsync(999, 1);
```

Because no matching task exists, the service should return:

```
null
```

The test confirms that with:

```
Assert.Null(response);
```

In the real API, the controller converts this `null` result into:

```
404 Not Found
```

Isolated Database Per Test

Each test uses a unique in-memory database:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

That prevents one test from leaking data into another test.

This matters because the missing-task test must not accidentally see the task created by the existing-task test.

Expected Test Result

After Day 71, running:

```
dotnet test
```

Should show five passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
```

Expected summary:

```
Passed: 5
Failed: 0
```

What Did Not Change

Day 71 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production TaskService code
new packages
controller tests
update/delete tests
```

The lesson stays focused on retrieving a task by id.

What This Day Teaches

This lesson teaches how to test a service method that returns either data or null.

That behavior matters because service methods often separate business logic from HTTP status codes:

```
service returns null
controller returns 404 Not Found
```

Keeping that boundary clear makes the code easier to test.

Reader Exercise

Add a third test:

```
GetByIdAsync_WithTaskOwnedByAnotherUser_ReturnsNull
```

Create two users, create a task for user one, then call GetByIdAsync with user two's id.

Expected result:

```
null
```

This verifies ownership protection directly in the service test.

Next Step

The next step is to continue expanding TaskService tests for list, update, delete, complete, or reopen behavior.

Day 72 - Test Task Update

Lesson Goal

Add automated tests for updating a task through TaskService.UpdateTaskAsync.

Day 72 continues the TaskService testing work. The goal is to verify successful updates, missing task behavior, and validation failure behavior using an EF Core in-memory database.

What Is Tested

The tests cover three cases:

```
a valid owned task update succeeds
a missing task returns null
an empty title throws the expected validation error
```

The tests also confirm that invalid update data does not change the saved task.

No New Packages

The test project already has the required setup:

```
Infrastructure project reference
Microsoft.EntityFrameworkCore.InMemory 8.0.27
```

Day 72 does not add new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Tasks\TaskServiceUpdateTests.cs
```

It tests TaskService.UpdateTaskAsync using an isolated in-memory database for each test.

Valid Update Test

The first test creates and saves an original task:

```
var task = await AddTaskAsync(dbContext);
```

Then it creates an update request:

```
var request = new UpdateTaskRequest
{
    Title = "Updated task title",
    Description = "Updated task description",
    DueDateUtc = DateTime.UtcNow.AddDays(7),
    Priority = "High"
};
```

The service is called with the task id, update request, and owner user id:

```
var response = await taskService.UpdateTaskAsync(
    task.Id,
    request,
    task.UserId);
```

This matches the ownership-aware update method added earlier.

Response Assertions

The test checks that the response contains the updated values:

```
Assert.NotNull(response);
Assert.Equal(request.Title, response.Title);
Assert.Equal(request.Description, response.Description);
Assert.Equal(request.DueDateUtc, response.DueDateUtc);
Assert.Equal(request.Priority, response.Priority);
Assert.Equal(task.UserId, response.UserId);
```

These assertions confirm that the service returned the expected updated task response.

Database Assertions

The test also reads the saved task from the in-memory database:

```
var savedTask = await dbContext.Tasks.SingleAsync();
```

Then it confirms the values were actually persisted:

```
Assert.Equal(request.Title, savedTask.Title);
Assert.Equal(request.Description, savedTask.Description);
Assert.Equal(request.DueDateUtc, savedTask.DueDateUtc);
Assert.Equal(request.Priority, savedTask.Priority);
```

This is important because the response alone is not enough. A useful service test should confirm the database state changed as expected.

Missing Task Test

The missing task test calls:

```
var response = await taskService.UpdateTaskAsync(999, request, 1);
```

Because no matching task exists, the service returns:

```
null
```

The test confirms:

```
Assert.Null(response);
```

In the real API, the controller converts this `null` result into:

```
404 Not Found
```

Empty Title Test

The empty title test sends:

```
Title = ""
```

The service should reject this invalid update by throwing:

```
ArgumentException
```

The test confirms the exact validation message:

```
Assert.Equal(ErrorMessages.TaskTitleRequired, exception.Message);
```

Then it reloads the saved task and confirms the original title is still present:

```
Assert.Equal(originalTitle, savedTask.Title);
```

This proves that invalid data was not saved.

Isolated Database Per Test

Each test uses:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

That gives every test a clean database.

This prevents a task created in one test from affecting another test.

Expected Test Result

After Day 72, running:

```
dotnet test
```

Should show eight passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
3 update task tests
```

Expected summary:

```
Passed: 8
Failed: 0
```

What Did Not Change

Day 72 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production TaskService code
new packages
controller tests
delete tests
complete or reopen tests
```

The lesson stays focused on update behavior.

What This Day Teaches

This lesson teaches that update tests should verify both:

```
the response returned by the service
the actual saved database state
```

It also teaches that validation tests should confirm bad data does not partially update the stored entity.

Reader Exercise

Add one more update test:

```
UpdateTaskAsync_WithTaskOwnedByAnotherUser_ReturnsNull
```

Create a task for user one, then call `UpdateTaskAsync` with user two's id.

Expected result:

```
null
```

Also confirm the task data was not changed.

Next Step

The next step is to continue expanding service tests for delete, list, complete, or reopen task behavior.

Day 73 - Test Task Delete

Lesson Goal

Add automated tests for deleting a task through `TaskService.DeleteTaskAsync`.

Day 73 continues the `TaskService` testing phase. The goal is to verify successful task deletion and missing-task behavior using an EF Core in-memory database.

What Is Tested

The tests cover two cases:

```
an owned task can be deleted successfully
deleting a missing task returns false
```

The successful delete test also confirms that the task is actually removed from the database.

No New Packages

The test project already has everything required:

```
Infrastructure project reference
EF Core InMemory package
xUnit
```

Day 73 does not install new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTracker.Tests\Tasks\TaskServiceDeleteTests.cs
```

It tests `TaskService.DeleteTaskAsync` with an isolated in-memory database.

Successful Delete Test

The first test creates one user and one owned task:

```
var task = await AddTaskAsync(dbContext);
```

Then it calls the real service:

```
var isDeleted = await taskService.DeleteTaskAsync(
    task.Id,
    task.UserId);
```

The method receives both:

```
task id
authenticated owner user id
```

That matches the ownership-aware delete behavior added earlier.

Delete Assertions

The first assertion checks the service result:

```
Assert.True(isDeleted);
```

This confirms the service reported a successful deletion.

The next assertion checks the database state:

```
Assert.Equal(0, await dbContext.Tasks.CountAsync());
```

This confirms the task was actually removed.

The final assertion confirms the owner was not deleted:

```
Assert.Equal(1, await dbContext.Users.CountAsync());
```

Deleting a task should remove the task only. It should not remove the user who owned it.

Missing Task Test

The second test uses an empty database and calls:

```
var isDeleted = await taskService.DeleteTaskAsync(999, 1);
```

Because no matching task exists, the service returns:

```
false
```

The test confirms:

```
Assert.False(isDeleted);
```

In the real API, the controller converts this result into:

```
404 Not Found
```

Why Check The Database

Checking only the boolean result is not enough.

This assertion confirms actual persistence behavior:

```
Assert.Equal(0, await dbContext.Tasks.CountAsync());
```

Good service tests verify both:

```
the returned value  
the saved database state
```

Isolated Database Per Test

Each test uses a fresh in-memory database:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

This prevents data from one test from affecting another test.

The missing-task test must remain empty, regardless of what the successful delete test created.

Expected Test Result

After Day 73, running:

```
dotnet test
```

Should show ten passing tests:

```
1 password hashing test  
2 task creation tests  
2 Get By Id tests  
3 update task tests  
2 delete task tests
```

Expected summary:

```
Passed: 10
Failed: 0
```

What Did Not Change

Day 73 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production TaskService code
new packages
controller tests
complete or reopen tests
```

The lesson stays focused on delete behavior.

What This Day Teaches

This lesson teaches that delete tests should verify more than the returned status.

The service should:

```
return true when an owned task is deleted
remove the task from storage
leave the owning user intact
return false when no matching task exists
```

This gives confidence that the delete behavior is correct before testing more complex task flows.

Reader Exercise

Add one more delete test:

```
DeleteTaskAsync_WithTaskOwnedByAnotherUser_ReturnsFalse
```

Create a task for user one, then call `DeleteTaskAsync` with user two's id.

Expected result:

```
false
```

Also confirm the task still exists in the database.

Next Step

The next step is to continue expanding service tests for task listing, complete, reopen, or ownership behavior.

Day 74 - Test User Service

Lesson Goal

Add automated tests for basic `UserService` behavior.

Day 74 expands the automated test suite beyond tasks. The goal is to test user creation and user retrieval by id through the real `UserService` using an EF Core in-memory database.

What Is Tested

The tests cover two cases:


```
a valid user is created and saved
an existing user can be retrieved by id
```

The tests also confirm that user entities are mapped correctly to `UserResponse`.

No Duplicate Email Test Today

`UserService.CreateUserAsync` validates that the email is not empty, but it does not currently check whether another user already has the same email.

That means `UserService` currently allows duplicate emails.

Day 74 does not add a duplicate-email rejection test because that behavior is not implemented in `UserService`.

Duplicate-email behavior exists separately in authentication registration and should be tested later in the auth service test area.

No New Packages

The test project already has everything required:

```
Infrastructure project reference
EF Core InMemory
xUnit
```

Day 74 does not install new packages and does not change production code.

Test File

The new test file is:

```
tests\JobTracker.Tests\Users\UserServiceTests.cs
```

It tests `UserService` with an isolated in-memory database.

Create User Test

The create test starts with a valid request:

```
var request = new CreateUserRequest
{
    FullName = "Create User Test",
    Email = "create.user@example.com"
};
```

Then it calls the real service:

```
var response = await userService.CreateUserAsync(request);
```

The response assertions confirm that a `UserResponse` is returned with the expected values:

```
Assert.True(response.Id > 0);
Assert.Equal(request.FullName, response.FullName);
Assert.Equal(request.Email, response.Email);
```

The database assertions confirm that the user was actually saved:

```
Assert.Equal(1, await dbContext.Users.CountAsync());
```

Then the saved entity is checked directly:

```
var savedUser = await dbContext.Users.SingleAsync();

Assert.Equal(request.FullName, savedUser.FullName);
Assert.Equal(request.Email, savedUser.Email);
```

This verifies both the returned DTO and the persisted entity.

Get By Id Test

The Get By Id test first creates a user through UserService:

```
var createdUser = await userService.CreateUserAsync(createRequest);
```

Then it retrieves the same user by id:

```
var response = await userService.GetByIdAsync(createdUser.Id);
```

The assertions confirm:

```
the response is not null
the id matches
the full name matches
the email matches
the created date matches
```

This proves the service can retrieve and map an existing user correctly.

Why Use UserService To Seed The Test

The Get By Id test uses CreateUserAsync to create the user first.

This keeps the setup simple and uses the same real service behavior to generate a saved user and id.

The main behavior under test is still the retrieval step:

```
GetByIdAsync returns the existing user as UserResponse
```

Isolated Database Per Test

Each test uses a fresh in-memory database:

```
.UseInMemoryDatabase(Guid.NewGuid().ToString())
```

That keeps the tests independent.

The create test and get-by-id test do not share data.

Expected Test Result

After Day 74, running:

```
dotnet test
```

Should show twelve passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
3 update task tests
2 delete task tests
2 user service tests
```

Expected summary:

```
Passed: 12
Failed: 0
```

What Did Not Change

Day 74 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production UserService code
new packages
```

```
duplicate email behavior
auth registration tests
controller tests
```

The lesson stays focused on basic user service behavior.

What This Day Teaches

This lesson teaches that service tests should match the behavior that exists today.

A test should not assert duplicate-email rejection in UserService if UserService does not implement duplicate-email rejection.

Good tests document real behavior clearly:

```
valid users can be created
saved users can be retrieved by id
responses are mapped correctly
```

Reader Exercise

Add a missing-user test:

```
GetByIdAsync_WithMissingUser_ReturnsNull
```

Use an empty in-memory database, call GetByIdAsync with an id that does not exist, and confirm the response is null.

Next Step

The next step is to continue expanding tests carefully, especially around authentication registration behavior and task ownership edge cases.

Day 75 - Test Auth Service

Lesson Goal

Add automated tests for registration and login through AuthService.

Day 75 expands the test suite into authentication behavior. The goal is to test registration, valid login, and invalid-password login without connecting to SQL Server and without changing production authentication code.

What Is Tested

The tests cover three cases:

```
successful registration
login with valid credentials
login with an invalid password
```

These are core authentication behaviors for the backend.

Why Use A Fake JWT Service

AuthService depends on IJwtTokenService.

The real JWT token service needs signing-key configuration. These tests are not trying to verify JWT signing, token expiration, or cryptographic configuration.

The tests use a small fake implementation that returns a predictable token:

```
test-token-USER_ID
```

This keeps the test focused on authentication flow:

```
register user
hash password
verify password
call token service after successful login
return auth response
```

No mocking library is needed.

No New Packages

The test project already has everything required:

```
Application reference
Infrastructure reference
EF Core InMemory
xUnit
```

Day 75 does not install new packages.

Test File

The new test file is:

```
tests\JobTrackr.Tests\Auth\AuthServiceTests.cs
```

It tests AuthService with an isolated in-memory database.

Registration Test

The registration test uses the real password hasher:

```
var passwordHasher = new PasswordHasherService();
```

Then it creates AuthService with:

```
AppDbContext
PasswordHasherService
FakeJwtTokenService
```

The test registers a valid user:

```
var response = await authService.RegisterAsync(request);
```

The response assertions confirm:

```
user id is generated
full name matches
email matches
registration response token is empty
```

Registration currently creates the user. Login is what returns the JWT token.

Password Hash Assertions

After registration, the test reads the saved user from the database.

It confirms the plain password was not stored:

```
Assert.NotEqual(request.Password, savedUser.PasswordHash);
```

Then it verifies the stored hash belongs to the supplied password:

```
Assert.True(passwordHasher.VerifyPassword(
    request.Password,
```

```
savedUser.PasswordHash));
```

These checks prove the registration flow stores a hash instead of storing the original password.

Valid Login Test

The valid login test first registers a user.

That gives the database a user with a properly hashed password.

Then the test logs in with the same email and password:

```
var response = await authService.LoginAsync(loginRequest);
```

The assertions confirm:

```
user id matches the registered user
full name matches
email matches
token matches the fake token format
```

The token assertion is:

```
Assert.Equal($"test-token-{{registeredUser.UserId}}", response.Token);
```

This proves AuthService called IJwtTokenService.GenerateToken after successful credential validation.

Invalid Password Test

The invalid login test registers a user first.

Then it tries to log in with:

```
WrongPassword
```

The test expects:

```
ArgumentException
```

And confirms the safe message:

```
Invalid email or password.
```

This message does not reveal whether the email exists or only the password was wrong.

FakeJwtTokenService

The fake token service is private inside the test class:

```
private class FakeJwtTokenService : IJwtTokenService
{
    public string GenerateToken(User user)
    {
        return $"test-token-{{user.Id}}";
    }
}
```

It follows the same interface as the real JWT service but returns a predictable test value.

It is used only by tests and never by the running API.

Expected Test Result

After Day 75, running:

```
dotnet test
```

Should show fifteen passing tests:

```
1 password hashing test
2 task creation tests
2 Get By Id tests
3 update task tests
2 delete task tests
2 user service tests
3 auth service tests
```

Expected summary:

```
Passed: 15
Failed: 0
```

What Did Not Change

Day 75 intentionally avoids extra scope.

No changes are made to:

```
SQL Server
production authentication code
JWT configuration
mocking libraries
controller tests
real token signing behavior
```

The lesson stays focused on authentication service behavior.

What This Day Teaches

This lesson teaches how to test a service with a dependency.

The real password hasher is useful because password hashing is part of the authentication behavior being tested.

The fake JWT service is useful because token signing configuration is not the behavior being tested today.

The key idea is:

```
use real dependencies when they are part of the behavior
use a small test double when a dependency would distract from the behavior
```

Reader Exercise

Add a fourth auth test:

```
RegisterAsync_WithDuplicateEmail_ThrowsArgumentException
```

Only add this if duplicate-email rejection exists in AuthService.

The test should register one user, try registering the same email again, and confirm the expected error message.

Next Step

The next step is to continue expanding authentication and ownership tests, especially for edge cases that protect real user data.

Day 76 - Test Build And CI Readiness

Lesson Goal

Verify that the complete JobTrackr solution is ready for Continuous Integration before adding a GitHub Actions workflow.

This is a review and verification lesson. No feature or test is added. The focus is proving that the same restore, build, and test commands planned for CI already work from the solution root.

What CI Readiness Means

Continuous Integration automatically checks the project whenever code is pushed or a pull request is opened.

Before asking GitHub to run those checks, the commands should work locally:

```
restore dependencies
build the complete solution
run the complete test suite
leave the repository clean
```

A CI workflow cannot repair an unreliable build. It only repeats the project's commands in a consistent environment.

Verify The Starting State

Start from the repository root and check Git:

```
git status
```

The expected result is:

```
nothing to commit, working tree clean
```

This matters because generated files under bin and obj should be ignored. A build or test run should not introduce tracked changes.

Confirm The Test Project Is Discoverable

The solution file must include the test project:

```
<Folder Name="/tests/">
  <Project Path="tests/JobTrackr.Tests/JobTrackr.Tests.csproj" />
</Folder>
```

This allows solution-level commands to build and test every required project without targeting the test project manually.

Restore Dependencies

Run:

```
dotnet restore JobTrackr.slnx
```

Restore resolves the NuGet dependencies for all projects in the solution.

If the dependencies are already available, the command can report that all projects are up to date. That is also a successful result.

Build In Release Configuration

Run:

```
dotnet build JobTrackr.slnx --configuration Release --no-restore
```

The separate `--no-restore` build proves that the restore stage completed successfully.

Using Release configuration is useful because it is closer to the configuration normally used by CI and deployment than a local Debug build.

The verified Day 76 result is:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

The complete solution builds, including:

```
JobTrackr.Domain  
JobTrackr.Application  
JobTrackr.Infrastructure  
JobTrackr.Api  
JobTrackr.Tests
```

Run The Complete Test Suite

Run:

```
dotnet test JobTrackr.slnx --configuration Release --no-build
```

The `--no-build` option uses the Release output from the previous step. This keeps each stage clear:

```
restore once  
build once  
test the completed build
```

The verified result is:

```
Passed: 15  
Failed: 0  
Skipped: 0
```

Current Test Coverage

The fifteen tests currently cover:

Authentication

- Password hashing and verification
- User registration
- Login with valid credentials
- Login with an invalid password

Tasks

- Creating a valid task
- Rejecting an empty task title
- Getting an existing task by ID
- Handling a missing task
- Updating a valid task
- Handling an update for a missing task

- Rejecting an empty title during update
- Deleting an owned task
- Handling deletion of a missing task

Users

- Creating a valid user
- Getting an existing user by ID

Review Test Names

Clear test names make failures easier to understand in local output and in CI logs.

The test suite follows this structure:

```
MethodOrBehavior_Scenario_ExpectedResult
```

Examples include:

```
CreateTaskAsync_WithValidRequest_CreatesTask  
GetByIdAsync_WithMissingTask_ReturnsNull  
UpdateTaskAsync_WithEmptyTitle_ThrowsArgumentException  
DeleteTaskAsync_WithExistingOwnedTask_DeletesTask  
LoginAsync_WithInvalidPassword_ThrowsArgumentException
```

Each name explains the behavior, condition, and expected outcome without requiring the reader to open the test first.

Check The Test Project

The test project should identify itself correctly:

```
<TargetFramework>net8.0</TargetFramework>  
<IsTestProject>true</IsTestProject>
```

Its dependencies include:

```
xUnit  
Microsoft.NET.Test.Sdk  
EF Core InMemory 8.0.27  
JobTrackr.Application project reference  
JobTrackr.Infrastructure project reference
```

Working package versions should not be changed during a verification day without a specific reason.

Verify The Final Repository State

Run Git status again after the build and tests:

```
git status
```

The repository remained clean during the verified Day 76 run.

This confirms that generated build and test files are ignored correctly.

Why There Is No Day 76 Commit

No source code, tests, or configuration required a change.

Creating an empty commit would make the project history noisier without documenting a real code change.

The previous commit remains the latest verified source state:

```
3eca56c Day 75: Test auth service
```

The Day 76 result is still valuable: it establishes a known-good baseline before CI automation is introduced.

What This Day Teaches

CI readiness is not mainly about writing a workflow file. It is about having reliable commands and a repository that behaves predictably.

The key sequence is:

```
clean repository
-> successful restore
-> successful Release build
-> complete passing test suite
-> clean repository
```

Once this sequence works locally, it can be transferred into GitHub Actions with much greater confidence.

Reader Exercise

Run the verification sequence from the repository root:

```
dotnet restore JobTrackr.slnx
dotnet build JobTrackr.slnx --configuration Release --no-restore
dotnet test JobTrackr.slnx --configuration Release --no-build
git status
```

Record:

```
the number of warnings
the number of errors
the number of passing tests
whether Git remained clean
```

Do not change code if everything passes.

Next Step

The next lesson can add a GitHub Actions workflow that runs the same restore, Release build, and test commands automatically.

Day 77 - Add GitHub Actions Build

Lesson Goal

Add the first GitHub Actions workflow to JobTrackr so every push to main, and every pull request targeting main, automatically verifies that the production projects build.

Day 76 proved that the project builds locally. Day 77 moves that check onto a clean computer managed by GitHub.

This first workflow intentionally performs build automation only. Automated tests will be added separately.

What GitHub Actions Provides

GitHub Actions can run repository commands when an event occurs.

For this workflow, the sequence is:

```
push or pull request
-> create a temporary Ubuntu runner
-> download the repository
-> install .NET 8
```

```
-> restore NuGet dependencies
-> build the API in Release configuration
```

This catches compilation failures without relying on a developer to remember to run the build locally.

Workflow Location

GitHub discovers workflow files inside:

```
.github/workflows/
```

The new file is:

```
.github/workflows/build.yml
```

No production C# file is changed in this lesson.

The Build Workflow

The completed workflow is:

```
name: Build

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v7

      - name: Set up .NET
        uses: actions/setup-dotnet@v6
        with:
          dotnet-version: 8.0.x

      - name: Restore dependencies
        run: dotnet restore src/JobTrackr.Api/JobTrackr.Api.csproj

      - name: Build
        run: dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-restore
```

YAML uses indentation to represent structure. Spaces must be used consistently because incorrect indentation can make the workflow invalid or change its meaning.

Workflow Triggers

The workflow responds to two events:

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

The push trigger checks code after it reaches main.

The pull request trigger checks proposed changes before they are merged into `main`.

Together, these triggers support both direct pushes and a future pull-request workflow.

Minimal Permissions

The workflow declares:

```
permissions:
  contents: read
```

The build only needs to read the repository. It does not need permission to modify source code, create releases, or write packages.

Giving automation only the permissions it requires is a useful security habit.

The GitHub Runner

The build job uses:

```
runs-on: ubuntu-latest
```

GitHub creates a temporary Linux environment for the job and removes it afterward.

Building successfully on Ubuntu demonstrates that the production projects compile outside the local Windows development environment.

Checkout And .NET Setup

The first action downloads the repository:

```
- name: Checkout repository
  uses: actions/checkout@v7
```

The next action installs the .NET 8 SDK:

```
- name: Set up .NET
  uses: actions/setup-dotnet@v6
  with:
    dotnet-version: 8.0.x
```

The `8.0.x` value allows the runner to use a current servicing release of the .NET 8 SDK.

Why The Action Versions Were Updated

The workflow was initially added in commit:

```
8f21e1a Day 77: Add GitHub Actions build
```

The action versions were then updated in:

```
7d8e8eb Day 77: Update GitHub Actions runtime
```

The update moved the workflow away from older action versions that produced a Node.js runtime deprecation warning on GitHub's runners.

This does not change JobTrackr's target framework. The application still uses .NET 8; the Node.js runtime is an internal implementation detail of the reusable GitHub actions.

Why Build The API Project

The workflow targets:

```
src/JobTrackr.Api/JobTrackr.Api.csproj
```

The repository currently uses the newer `JobTrackr.slnx` solution format. Building the API project directly avoids depending on solution-format support across different .NET 8 SDK versions.

The API project references the other production layers:

```
JobTrackr.Api
-> JobTrackr.Infrastructure
-> JobTrackr.Application
-> JobTrackr.Domain
```

Therefore, building the API also compiles all four production projects.

The test project is not part of this dependency chain and is deliberately left for the next workflow improvement.

Separate Restore And Build Stages

The restore step is:

```
- name: Restore dependencies
  run: dotnet restore src/JobTrackr.Api/JobTrackr.Api.csproj
```

The build step is:

```
- name: Build
  run: dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-restore
```

Using `--no-restore` makes the stages explicit. If dependency restoration fails, the workflow stops at restore instead of hiding the problem inside the build command.

Verified Local Result

The same commands were run locally from the repository root:

```
dotnet restore src\JobTrackr.Api\JobTrackr.Api.csproj
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release --no-restore
```

The verified result was:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The build compiled:

```
JobTrackr.Domain
JobTrackr.Application
JobTrackr.Infrastructure
JobTrackr.Api
```

Git remained clean after the verification.

Why No Secrets Or Database Are Required

This workflow compiles the code. It does not launch the API or execute database operations.

Therefore, the build does not need:

```
SQL Server
a production connection string
a JWT signing key
GitHub repository secrets
```

Runtime configuration should only be added when a workflow step actually needs to run the application or connect to an external service.

Checking The Workflow On GitHub

After the workflow is pushed, open the repository's Actions tab and inspect the newest Build run.

The job should contain these steps:

```
Checkout repository
Set up .NET
Restore dependencies
Build
```

A successful run confirms the production code builds on a clean GitHub runner.

When a workflow fails, open the failed step first. Common causes include:

```
incorrect YAML indentation
an incorrect project path
an invalid action version
a misspelled action name
an actual compilation error
```

Fix the reported problem rather than adding unrelated workflow complexity.

What This Day Teaches

Continuous Integration starts with a small, dependable check.

The first workflow answers one important question:

```
Can the production application compile from a clean checkout?
```

Keeping the workflow focused makes failures easier to understand and provides a stable foundation for adding tests next.

Reader Exercise

Create a branch, make a harmless documentation change, and open a pull request targeting main.

Confirm that the Build workflow starts automatically and completes successfully. Then inspect each step to see how GitHub presents logs and execution time.

Do not add deployment credentials or services to this exercise.

Next Step

The next lesson can extend Continuous Integration by running the complete automated test suite after the Release build succeeds.

Day 78 - Add GitHub Actions Tests

Lesson Goal

Extend the existing GitHub Actions build workflow so every push to main, and every pull request targeting main, runs the complete JobTrackr test suite.

Day 77 automated the production build. Day 78 adds behavioral verification without creating another workflow or changing production code.

From Build Automation To Continuous Integration

The existing workflow answers:

```
Does the production application compile?
```

The test step adds another question:

```
Do the tested backend behaviors still work?
```

The complete CI sequence is now:

```
push or pull request
-> restore API dependencies
-> restore test dependencies
-> build production projects
-> build the test project
-> run all automated tests
-> report success or failure
```

Update The Existing Workflow

The only changed file is:

```
.github/workflows/build.yml
```

A second workflow is unnecessary because building and testing are parts of the same validation process.

Keeping them in one job also means the Test step only runs after the earlier checkout, setup, restore, and build steps succeed.

Completed Workflow

The workflow now contains:

```
name: Build

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v7

      - name: Set up .NET
        uses: actions/setup-dotnet@v6
        with:
          dotnet-version: 8.0.x

      - name: Restore API dependencies
        run: dotnet restore src/JobTrackr.Api/JobTrackr.Api.csproj

      - name: Restore test dependencies
        run: dotnet restore tests/JobTrackr.Tests/JobTrackr.Tests.csproj

      - name: Build
        run: dotnet build src/JobTrackr.Api/JobTrackr.Api.csproj --configuration Release --no-restore

      - name: Test
        run: dotnet test tests/JobTrackr.Tests/JobTrackr.Tests.csproj --configuration Release --
```

```
no-restore
```

The existing triggers, permissions, Ubuntu runner, and .NET setup remain unchanged.

Clear Restore Step Names

The original step was named:

```
- name: Restore dependencies
```

It is renamed to:

```
- name: Restore API dependencies
```

The test restore step is named separately:

```
- name: Restore test dependencies
```

These labels make GitHub's workflow log easier to scan. A failed restore immediately reveals whether the production or test project caused the problem.

Why Restore The Test Project Separately

The API project does not depend on the test project.

The test project has its own packages, including:

```
xUnit
Microsoft.NET.Test.Sdk
Microsoft.EntityFrameworkCore.InMemory
```

Restoring only the API project does not guarantee that these test-specific dependencies are available.

The workflow therefore runs:

```
run: dotnet restore tests/JobTrackr.Tests/JobTrackr.Tests.csproj
```

This creates a clear dependency stage before testing.

Run The Test Project Directly

The Test step is:

```
- name: Test
  run: dotnet test tests/JobTrackr.Tests/JobTrackr.Tests.csproj --configuration Release --no-restore
```

The repository uses the newer JobTrackr.slnx solution format. Targeting the test project directly avoids depending on .slnx support across different .NET 8 SDK servicing versions.

The test project references the Application and Infrastructure projects. Their dependencies, including Domain, are compiled automatically when the test project is built.

Understanding The Test Command

The command has three important parts:

```
dotnet test
```

Builds the test project when needed and runs all discovered tests.

```
--configuration Release
```

Uses the same Release configuration as the production build.

```
--no-restore
```


Proves that the earlier test dependency restore completed successfully.

The command does not use `--no-build` because the earlier Build step targets the API and its production dependencies, not `JobTrackr.Tests`.

Verified Result

The same workflow commands were run locally from the repository root.

The production build completed with:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

The test command completed with:

```
Passed: 15  
Failed: 0  
Skipped: 0  
Total: 15
```

The passing tests cover task operations, user operations, password hashing, registration, and login behavior.

How A Failed Test Protects The Project

`dotnet test` returns a nonzero exit code when any test fails.

GitHub Actions responds by:

```
marking the Test step as failed  
marking the complete workflow run as failed  
showing the failed test name  
displaying the assertion or exception details
```

This gives the developer feedback before a regression is accepted as healthy code.

When branch protection is added later, a successful workflow can also become a requirement before merging a pull request.

Why CI Does Not Need SQL Server

The JobTrackr service tests use:

```
Microsoft.EntityFrameworkCore.InMemory
```

Each test creates an isolated in-memory database. The workflow does not need to install or connect to SQL Server.

It also does not need:

```
a database connection string  
database migrations  
a JWT signing key  
GitHub secrets
```

The fake JWT service used by authentication tests avoids requiring token-signing configuration during these service-level tests.

Workflow Steps On GitHub

The Build job now presents these main steps:

```
Checkout repository  
Set up .NET  
Restore API dependencies  
Restore test dependencies
```

Build
Test

Opening the Test step reveals the test runner output and the number of passed, failed, and skipped tests.

If the step fails, start with the failed test name and its error message. Avoid changing unrelated workflow configuration when the failure comes from application behavior.

Scope Control

Day 78 deliberately does not add:

```
a second workflow file
SQL Server services
repository secrets
deployment
code coverage reporting
test result artifacts
changes to production C# code
changes to test code
```

The goal is a small and understandable improvement: automatically run the tests that already exist.

What This Day Teaches

A successful build proves that code can compile. A passing test suite provides evidence that important behavior remains correct.

The CI pipeline now checks both:

```
structural correctness through compilation
behavioral correctness through automated tests
```

This creates a stronger safety net for future backend changes.

Reader Exercise

Open a successful workflow run in GitHub Actions and inspect the Test step.

Find:

```
the test assembly
the total number of tests
the number passed
the number failed
the number skipped
```

Then locate one test locally and connect its method name to the name shown in the CI output.

Do not intentionally break the main branch to demonstrate a failed test.

Next Step

The next lesson can improve the CI workflow with another practical quality check, or begin the next tested backend feature using the automated build-and-test baseline.

Day 79 - Review README For Testing

Lesson Goal

Update the JobTrackr repository README so it accurately describes the backend as it exists today.

This is a documentation-only lesson. No C# source code, test code, or GitHub Actions workflow is changed.

Why Documentation Must Evolve

A README is often the first part of a repository that another developer, interviewer, client, or contributor reads.

Before Day 79, the README still implied that login did not generate a JWT, task requests accepted a public user ID, task queries could expose every user's data, and testing was future work.

Those statements no longer matched JobTrackr.

Technical documentation should answer:

```
What does the project do?
How is it organized?
Which endpoints exist?
How does authentication work?
Which security behavior is complete?
Which limitations remain?
How can the project be built and tested?
What does CI verify?
```

Current Project Summary

The README now introduces JobTrackr as a learning-focused .NET 8 Web API built with production-style backend practices.

It identifies the current capabilities:

```
ASP.NET Core Web API
Clean Architecture-style project boundaries
SQL Server and Entity Framework Core
user and task endpoints
password hashing
registration and JWT login
authenticated, user-owned tasks
xUnit service tests
GitHub Actions continuous integration
```

Architecture Documentation

The five projects are documented by responsibility:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
JobTrackr.Tests
```

JobTrackr.Api contains controllers, middleware, authentication configuration, dependency registration, and startup.

JobTrackr.Application contains DTOs, interfaces, shared messages, and application contracts.

JobTrackr.Domain contains the core User and JobTask entities.

JobTrackr.Infrastructure contains EF Core access, migrations, service implementations, authentication logic, and JWT generation.

JobTrackr.Tests contains xUnit service tests using isolated in-memory databases.

Current Endpoint Documentation

The README groups endpoints into Authentication, Tasks, and Users.

Authentication endpoints are:

```
POST /api/auth/register
POST /api/auth/login
```

Task documentation covers listing, completion filtering, title searching, creation, retrieval, updating, deletion, completion, and reopening.

The outdated public `userId` task filter is no longer advertised.

Accurate Authentication Behavior

The README distinguishes registration from login:

```
registration creates the user and currently returns an empty token
successful login returns a JWT token
```

It also documents that duplicate emails are rejected, passwords are hashed, authentication responses do not expose password hashes, and protected task endpoints require a Bearer token.

Accurate Task Ownership

Task ownership now comes from the authenticated identity:

```
task creation does not accept UserId
new tasks belong to the authenticated user
task listing returns only that user's tasks
filters operate within that user's tasks
Get By Id checks ownership
Update checks ownership
Delete checks ownership
```

For ownership-protected operations, missing tasks and tasks belonging to another user both return 404 Not Found.

This avoids revealing whether another user's task exists.

Document Limitations Honestly

Good documentation does not present planned security work as completed behavior.

The README explicitly records:

```
Complete and Reopen require authentication but do not yet check ownership
User CRUD endpoints are not yet protected by authorization
```

Publishing these limitations prevents readers from assuming protections that do not exist and creates an accurate roadmap for future work.

Database Documentation

The current persistence summary includes SQL Server, Entity Framework Core, migrations, Users and Tasks, password hashes, due dates, priorities, the task-to-user foreign key, and cascade deletion.

This provides useful context without turning the README into a full database specification.

Automated Tests

The README now records the current fifteen xUnit service tests.

Coverage includes:

```
password hashing and verification
registration
valid and invalid login
```

```
valid and invalid task creation
existing and missing task retrieval
valid and invalid task updates
successful and missing task deletion
valid user creation
existing user retrieval
```

The service tests use `Microsoft.EntityFrameworkCore.InMemory`, so they do not connect to SQL Server.

Reproducible Test Commands

Readers can run:

```
dotnet restore tests\JobTracker.Tests\JobTracker.Tests.csproj
dotnet test tests\JobTracker.Tests\JobTracker.Tests.csproj --configuration Release --no-restore
```

The verified result is:

```
Passed: 15
Failed: 0
Skipped: 0
```

GitHub Actions Documentation

The README explains that `.github/workflows/build.yml` runs on pushes and pull requests targeting `main`.

The workflow checks out the repository, installs .NET 8, restores API and test dependencies, builds the production projects in Release configuration, and runs all automated tests.

Because the tests use an in-memory database, CI does not need SQL Server or repository secrets.

Verified Build

The documented production build was verified:

```
dotnet restore src\JobTracker.Api\JobTracker.Api.csproj
dotnet build src\JobTracker.Api\JobTracker.Api.csproj --configuration Release --no-restore
```

Result:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

All fifteen tests also passed.

Scope Of The Commit

Commit 772fb66 changes only `README.md`:

```
162 lines added
102 lines removed
```

No production code, test code, or workflow configuration was modified.

What This Day Teaches

Documentation quality depends on accuracy, not only presentation.

The important habit is:

```
compare documentation with current behavior
remove outdated claims
describe verified features precisely
publish known limitations
test every command given to readers
```

A trustworthy README helps other people evaluate the project and helps the developer explain it clearly during interviews.

Reader Exercise

Review one of your own repository READMEs.

For every feature claim, identify the controller, service, test, or workflow that proves it exists. Mark unsupported claims as outdated, planned, partially complete, or verified.

Then update the README and run every documented command before publishing it.

Next Step

The next lesson can address one of the authorization limitations now documented publicly, or continue improving reliability from the verified build, test, and CI baseline.

Day 80 - Add Basic Request Logging

Lesson Goal

Add simple request logging to JobTrackr using the logging system built into ASP.NET Core.

Each completed request records:

```
HTTP method
request path
response status code
elapsed time in milliseconds
```

The implementation deliberately avoids request bodies, passwords, JWT tokens, authorization headers, signing keys, and connection strings.

Why Request Logging Matters

When an API is running, developers need a quick way to understand which routes are being called, whether they succeed, and how long they take.

A useful request log can answer:

```
Which endpoint received traffic?
Did it return success, a client error, or a server error?
How long did processing take?
```

Request logging improves visibility without changing controller or service behavior.

The Middleware

The new file is:

```
src/JobTrackr.Api/Middleware/RequestLoggingMiddleware.cs
```

It uses a stopwatch to measure the full request:

```
var stopwatch = Stopwatch.StartNew();

try
{
    await _next(context);
}
finally
{
    stopwatch.Stop();
    // Log the request details
}
```

```
stopwatch.Stop();

_logger.LogInformation(
    "HTTP {Method} {Path} responded {StatusCode} in {ElapsedMilliseconds} ms",
    context.Request.Method,
    context.Request.Path,
    context.Response.StatusCode,
    stopwatch.ElapsedMilliseconds);
}
```

The middleware starts timing before passing control to the rest of the pipeline. It writes the log after downstream processing completes.

Why Use A Finally Block

The logging code belongs in a finally block because finally runs whether downstream processing succeeds or throws an exception.

Without it, failed requests could disappear from the request log precisely when diagnostic information is most useful.

The pattern is:

```
start timer
-> execute remaining middleware and endpoint
-> stop timer and log result in finally
```

Structured Logging

The message template uses named placeholders:

```
HTTP {Method} {Path} responded {StatusCode} in {ElapsedMilliseconds} ms
```

This is structured logging. Method, Path, StatusCode, and ElapsedMilliseconds are supplied as separate values rather than being joined manually into one string.

Logging systems can later filter or group records using those fields. Examples include finding all 500 responses or comparing elapsed time by route.

Built-In ILogger

The middleware receives:

```
ILogger<RequestLoggingMiddleware>
```

No third-party package or custom service registration is required. ASP.NET Core supplies both the logger and the next request delegate when it creates the middleware.

The logger category is associated with RequestLoggingMiddleware, which makes its records identifiable among other application logs.

Middleware Registration

Program.cs adds one line to the request pipeline:

```
app.UseHttpsRedirection();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseMiddleware<ExceptionHandlerMiddleware>();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

The position of request logging is intentional.

Why Middleware Order Matters

Request logging is registered before exception handling.

When a downstream component throws an unexpected exception:

```
RequestLoggingMiddleware starts timing
-> ExceptionHandlingMiddleware calls downstream code
-> downstream code throws
-> ExceptionHandlingMiddleware creates a 500 response
-> control returns to RequestLoggingMiddleware
-> request logger records status 500
```

This order allows the request log to capture the final response status produced by the exception handler.

If logging were placed inside exception handling, the behavior and final recorded status could differ.

Safe Request Metadata

An example log is:

```
HTTP GET /api/tasks responded 200 in 24 ms
```

This contains enough information for basic operational visibility without exposing user content or credentials.

Only the request path is logged. The query string is not included, reducing the chance that sensitive query values enter the logs.

Information That Must Not Be Logged

The middleware does not record:

```
request or response bodies
passwords
JWT tokens
Authorization headers
JWT signing keys
database connection strings
```

Logs often remain available longer than individual requests and may be accessed by more people than production data. A secret written to a log should be treated as exposed.

The safe default is to log metadata, not payloads.

Example Outcomes

An unauthenticated request to a protected endpoint can produce:

```
HTTP GET /api/tasks responded 401 in 12 ms
```

A successful login can produce:

```
HTTP POST /api/auth/login responded 200 in 35 ms
```

The password and returned token must not appear.

An unknown route can produce:

```
HTTP GET /api/unknown responded 404 in 1 ms
```

Elapsed time naturally varies between requests and environments.

Scope Of The Change

Commit 32fef89 changes two files:

```
RequestLoggingMiddleware.cs: new middleware
```



```
Program.cs: middleware pipeline registration
```

The commit adds 40 lines and does not change controllers, application services, domain entities, persistence behavior, or the existing exception handler.

Verified Result

The API was built in Release configuration:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

The existing test suite also passed:

```
Passed: 15  
Failed: 0  
Skipped: 0
```

This confirms that adding request logging did not break the tested backend behavior.

Current Limitation

The existing tests focus on application and infrastructure services. They do not directly test the new middleware.

A future middleware or integration test could verify:

```
the next delegate is called  
the log is written after completion  
the method, path, status, and elapsed fields are present  
exceptions still result in a request log
```

That broader test is outside Day 80's intentionally small scope.

What This Day Teaches

Request logging is a cross-cutting concern, so middleware is a natural implementation point.

The main lessons are:

```
measure the whole request pipeline  
log in finally so failures are visible  
use structured placeholders  
place middleware deliberately  
record useful metadata without recording secrets
```

The result is a small observability improvement that applies consistently across the API.

Reader Exercise

Run the API and make three requests:

```
a successful public request  
an unauthorized protected request  
an unknown route
```

For each request, confirm that the terminal shows method, path, final status, and elapsed time.

Also confirm that request bodies, passwords, tokens, authorization headers, and query strings do not appear.

Next Step

The next lesson can add focused middleware testing, improve log context with a safe request identifier, or address one of the remaining authorization gaps.

Day 81 - Review Structured Error Responses

Lesson Goal

Review how JobTrackr responds to invalid requests, authentication failures, missing resources, and unexpected server errors.

This is a review and testing lesson. No controller, service, DTO, middleware, or workflow file is changed.

Why Review Errors Separately

Successful responses demonstrate the normal path. Error responses define how clients understand and recover from problems.

An API error contract should answer:

- What happened?
- Which HTTP status applies?
- Is the problem safe to show to the client?
- Can the client parse the response consistently?

Day 81 finds that JobTrackr uses appropriate status codes and safe messages, but does not yet use one consistent response-body format.

Current Error Summary

Situation	Status	Current body
Invalid DTO validation	400 Bad Request	Structured validation JSON
Duplicate registration email	400 Bad Request	Plain-text message
Invalid login credentials	400 Bad Request	Plain-text message
Missing or invalid JWT	401 Unauthorized	Usually empty
Task not found	404 Not Found	Task not found.
User not found	404 Not Found	User not found.
Unexpected exception	500 Internal Server Error	An unexpected error occurred.

The status codes communicate the broad categories correctly. The inconsistency lies in whether the body is JSON, plain text, or absent.

Automatic DTO Validation

The API controllers use the `ApiController` attribute.

Request models use data annotations such as:

- `Required`
- `EmailAddress`
- `MinLength`
- `MaxLength`

ASP.NET Core checks these annotations before the controller action executes.

An invalid registration request can produce:

```
{
  "fullName": "",
  "email": "not-an-email",
  "password": "123"
}
```

The response is 400 Bad Request and includes field-level errors for FullName, Email, and Password.

A typical validation body includes:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "FullName": ["Full name is required."],
    "Email": ["Email is not valid."],
    "Password": ["Password must be at least 6 characters."]
  }
}
```

ASP.NET Core may also include fields such as type and traceId.

Why Automatic Validation Helps

Automatic validation creates several useful guarantees:

```
invalid DTOs do not reach application logic
field names are associated with their messages
multiple validation failures can be returned together
controllers need less repeated guard code
```

This is currently the most structured error response in JobTrackr.

Invalid Task Creation

Creating a task with an empty title also fails DTO validation:

```
{
  "title": "",
  "description": "Validation test",
  "priority": "Medium"
}
```

The API returns 400 Bad Request with a Title validation error before the task action performs normal creation work.

Because task endpoints are protected, the caller must still provide a valid JWT before reaching DTO validation for that endpoint.

Missing Authentication

TasksController uses the Authorize attribute.

Calling a protected route without a valid Bearer token returns:

```
401 Unauthorized
```

Authentication rejects the request before the task action runs. The response body is usually empty.

An empty 401 body is valid HTTP behavior, but it differs from the structured validation response and the plain-text application errors.

Invalid Login

Login with a registered email and incorrect password returns:

```
400 Bad Request
Invalid email or password.
```

The message deliberately does not reveal whether the email exists or whether only the password was incorrect.

This is an important security property. Detailed credential failure messages can help attackers discover registered accounts.

The message is safe, but the body is plain text rather than structured JSON.

Not-Found Responses

Requesting a missing task returns:

```
404 Not Found
Task not found.
```

Requesting a missing user returns:

```
404 Not Found
User not found.
```

These shared messages come from the Application project's `ErrorMessages` class.

Ownership-protected task operations also use 404 when the requested task belongs to another user. This avoids confirming the existence of another user's resource.

Unexpected Exceptions

`ExceptionHandlerMiddleware` catches exceptions that escape downstream code and returns:

```
500 Internal Server Error
An unexpected error occurred.
```

The middleware sets the content type to plain text.

The client does not receive:

```
stack traces
database details
connection strings
JWT signing keys
internal exception messages
```

The generic message prevents accidental disclosure of technical or sensitive information.

Request Logging And Errors

Day 80 request logging records the final status code for these requests.

Examples include:

```
HTTP POST /api/auth/register responded 400 in ... ms
HTTP GET /api/tasks responded 401 in ... ms
HTTP GET /api/tasks/999999 responded 404 in ... ms
```

The logger records request metadata without recording passwords, JWT tokens, authorization headers, or request bodies.

This gives the server operational visibility while preserving a safer client response.

What Is Already Correct

The review confirms several good behaviors:

```
invalid request data returns 400
missing authentication returns 401
missing or non-owned resources return 404
```

```
unexpected failures return 500
login errors do not reveal which credential failed
unexpected errors do not expose internal details
request logging records status without recording secrets
```

These behaviors should be preserved during any future response-format improvement.

The Main Inconsistency

Clients currently need to handle three response-body styles:

```
structured JSON for validation
plain text for many application and server errors
an empty body for some authentication failures
```

This makes client code more complicated because it cannot parse every error through one predictable contract.

The problem is not primarily the status codes. It is the representation of error details.

Why Not Fix It During The Review

Day 81 separates diagnosis from implementation.

Changing error responses can affect:

```
controllers
exception middleware
authentication behavior
frontend or API clients
tests
public documentation
```

The current behavior should be understood before selecting a standard format.

ASP.NET Core ProblemDetails is a candidate for Day 82 because it provides a recognized JSON structure without requiring a custom error model for every case.

Verified Baseline

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

Git remains clean, and no commit is needed for this review-only day.

Testing Gap

The current fifteen tests focus mainly on application and infrastructure services.

They do not yet prove the HTTP shape of:

```
automatic validation responses
401 authentication responses
controller 404 responses
exception middleware 500 responses
```

Future integration tests can lock down those contracts after the project chooses a consistent format.

What This Day Teaches

Error handling has two related but separate concerns:

- HTTP semantics: selecting the correct status code
- response contract: returning predictable, parseable details

JobTrackr's status-code semantics are mostly sound. Its next opportunity is a consistent response contract that retains safe messages and useful validation detail.

Reader Exercise

Create an error-response table for another API.

For each endpoint failure, record:

- status code
- content type
- response body shape
- whether sensitive details are exposed
- whether clients can parse it consistently

Separate genuine status-code problems from formatting inconsistencies before proposing changes.

Next Step

Evaluate ASP.NET Core ProblemDetails as a standard JSON format for validation, application, authentication, not-found, and unexpected-error responses without adding unnecessary complexity.

Day 82 - Add ProblemDetails Error Responses

Lesson Goal

Use ASP.NET Core ProblemDetails to make JobTrackr error responses more consistent.

Day 81 established that the API already used appropriate status codes but mixed structured JSON, plain text, and empty response bodies. Day 82 standardizes known controller errors and unexpected server failures without changing business logic.

What ProblemDetails Provides

ProblemDetails is a standard JSON representation for API errors.

A missing task can now return:

```
{
  "title": "Not Found",
  "status": 404,
  "detail": "Task not found."
}
```

The main fields are:

- title: broad error category
- status: HTTP status code
- detail: safe explanation for the client

ASP.NET Core can also include standard fields such as type and instance when appropriate.

Scope Of The Change

Day 82 updates:

```
AuthController
TasksController
UsersController
ExceptionHandlerMiddleware
```

It does not:

```
install a package
create another error layer
change service interfaces
change successful responses
replace automatic DTO validation
customize 401 authentication challenges
expose exception messages or stack traces
```

The work stays focused on response consistency.

Known Bad Requests

Controllers previously returned exception messages as plain text:

```
return BadRequest(ex.Message);
```

They now return:

```
return Problem(
    statusCode: StatusCodes.Status400BadRequest,
    title: "Bad Request",
    detail: ex.Message);
```

The existing safe service message is preserved, but it is placed in a predictable JSON field.

Authentication Errors

Registration and login business errors now use ProblemDetails.

An invalid login can return:

```
{
  "title": "Bad Request",
  "status": 400,
  "detail": "Invalid email or password."
}
```

The message remains intentionally vague. It does not reveal whether the email exists or whether only the password was incorrect.

Duplicate registration and other known argument errors follow the same 400 response shape.

Task Not-Found Errors

TaskController now uses ProblemDetails wherever the shared task-not-found message is returned.

The pattern is:

```
return Problem(
    statusCode: StatusCodes.Status404NotFound,
    title: "Not Found",
    detail: ErrorMessages.TaskNotFound);
```

This applies to task retrieval, update, deletion, completion, and reopening paths that return the shared missing-task result.

A missing or non-owned task therefore uses:

```
{
  "title": "Not Found",
  "status": 404,
  "detail": "Task not found."
}
```

The ownership behavior remains unchanged. Another user's task is still treated as not found.

User Not-Found Errors

UserController applies the same pattern:

```
return Problem(
    statusCode: StatusCodes.Status404NotFound,
    title: "Not Found",
    detail: ErrorMessages.UserNotFound);
```

The client receives:

```
{
  "title": "Not Found",
  "status": 404,
  "detail": "User not found."
}
```

User validation and business-rule errors also use the standard 400 structure.

Automatic Validation Remains Intact

Controllers use the ApiController attribute, and DTOs use data annotations.

Invalid DTOs already produce ValidationProblemDetails with an errors object:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Email": ["Email is not valid."],
    "Password": ["Password must be at least 6 characters."]
  }
}
```

This behavior is more detailed than ordinary ProblemDetails because validation must associate messages with specific fields.

Day 82 does not replace or duplicate it.

Unexpected Server Errors

ExceptionHandlerMiddleware previously returned a plain-text 500 response.

It now creates:

```
var problemDetails = new ProblemDetails
{
    Status = StatusCodes.Status500InternalServerError,
    Title = "Internal Server Error",
    Detail = "An unexpected error occurred."
};
```

The response content type is:

```
application/problem+json
```

The client receives safe JSON:

```
{
  "title": "Internal Server Error",
  "status": 500,
  "detail": "An unexpected error occurred."
}
```


Log The Real Exception On The Server

The middleware now receives ILogger for its own category and records the exception:

```
_logger.LogError(  
    ex,  
    "An unexpected error occurred while processing the request.");
```

This creates an important separation:

```
server log: exception object and technical diagnostic details  
client response: generic, safe ProblemDetails message
```

Developers retain the information needed to investigate failures without exposing internals to API consumers.

Information Kept Out Of Responses

ProblemDetails responses must not contain:

```
password values  
password hashes  
JWT tokens  
JWT signing keys  
authorization headers  
connection strings  
stack traces  
database exception details  
internal exception messages
```

A structured response is not automatically safe. The values assigned to its fields still require careful judgment.

Why 401 Remains Unchanged

Missing or invalid authentication continues to return the standard:

```
401 Unauthorized
```

The body may be empty.

Customizing authentication challenge responses is possible, but it touches a different part of the ASP.NET Core pipeline. Day 82 avoids expanding scope simply to make every error look identical immediately.

Consistency is valuable, but narrow, understandable changes are valuable too.

Error Formats After Day 82

Error category	Response format
DTO validation	ValidationProblemDetails JSON
Known bad request	ProblemDetails JSON
Missing resource	ProblemDetails JSON
Unexpected exception	ProblemDetails JSON
Missing or invalid JWT	Standard 401 response

The API now has a substantially more predictable error contract while preserving specialized validation details.

Request Logging Still Works

RequestLoggingMiddleware remains outside exception handling in the pipeline.

When an unexpected exception occurs:

```
exception middleware logs technical details
exception middleware writes safe ProblemDetails
request logging records the final 500 status and elapsed time
```

These two logging responsibilities complement each other:

```
request log: what route failed and how long it took
exception log: why the server failed
```

Neither log should record credentials or tokens.

Scope Of The Commit

Commit 7dc3769 changes four files:

```
AuthController.cs
TasksController.cs
UsersController.cs
ExceptionHandlerMiddleware.cs
```

The commit contains 35 additions and 21 removals.

No service, domain, persistence, DTO, or workflow file changes.

Verified Result

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The existing automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The test suite verifies that application behavior still works, although it does not yet assert the HTTP ProblemDetails shapes directly.

Testing Gap

Future integration tests should verify:

```
400 business errors use application/problem+json
404 responses include the expected title, status, and detail
500 responses hide exception details
automatic validation still includes its errors object
401 behavior remains intentional
```

These tests would protect the public error contract from accidental regression.

What This Day Teaches

Standardizing API errors improves predictability for clients.

The important principles are:

```
preserve correct status codes
use a recognized error representation
keep field-level validation detail
log exceptions on the server
return generic server errors to clients
avoid broad architectural changes for a focused improvement
```

ProblemDetails creates consistency without requiring a custom error DTO for every controller.

Reader Exercise

Call four failing endpoints:

```
invalid login
missing task
missing user
invalid registration DTO
```

Compare the content type and JSON shape.

Confirm that ordinary errors contain title, status, and detail, while validation adds an errors object. Also confirm that no password, token, stack trace, or database detail appears.

Next Step

Add focused integration tests for the ProblemDetails contract, or review whether the remaining standard 401 response needs safe JSON customization.

Day 83 - Add A Health Check Endpoint

Lesson Goal

Add a public health endpoint using ASP.NET Core's built-in health-check support.

The endpoint is:

```
GET /health
```

When the API process is running and can respond, it returns:

```
200 OK
Healthy
```

Why Health Checks Matter

Deployment platforms, load balancers, monitoring systems, and developers need a simple way to determine whether an application process is available.

Without a dedicated route, a monitoring tool may call a business endpoint that requires authentication, database data, or a specific request payload.

A health route provides a stable operational signal with a small response.

Scope Of Day 83

This lesson adds only a basic application health check.

It does:

```
use built-in ASP.NET Core health checks
add one public route
return a minimal response
change only Program.cs
```

It does not:

```
install a NuGet package
create a health controller
check SQL Server
require a JWT
expose configuration or secrets
redesign application startup
```

Register Health-Check Services

Program.cs now contains:

```
builder.Services.AddAuthorization();
builder.Services.AddHealthChecks();
```

AddHealthChecks registers the built-in health-check services with dependency injection.

Basic support is included with ASP.NET Core, so no external package is needed.

Map The Endpoint

The request pipeline now includes:

```
app.UseHttpsRedirection();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseMiddleware<ExceptionHandlerMiddleware>();
app.UseAuthentication();
app.UseAuthorization();
app.MapHealthChecks("/health");
app.MapControllers();
```

MapHealthChecks creates the endpoint directly. A controller and action method are unnecessary.

Why The Endpoint Is Public

Monitoring systems must often check application availability before they can obtain or refresh an application token.

The health endpoint therefore works without a JWT.

Public access is safe here because the response reveals only:

```
Healthy
```

It does not reveal:

```
database names
connection strings
server names
JWT configuration
environment variables
application data
exception details
```

Health endpoints should provide enough information for operations without becoming an information-disclosure endpoint.

What This Basic Check Proves

The current health check confirms:

```
the JobTrackr API process has started
ASP.NET Core can accept an HTTP request
the routing pipeline can return a response
```

This is sometimes called a liveness-style signal.

It answers a narrow question:

```
Is the web application running and responsive?
```

What It Does Not Prove

The endpoint does not confirm:

```
SQL Server is available
```

```
database credentials are valid
migrations are current
external services are available
every controller action works
authentication can issue tokens
```

A healthy response should not be interpreted as proof that every dependency and feature is operational.

Dependency checks can be added later when deployment requirements justify them.

Live Verification

The API was started locally on a temporary HTTP port, and the health endpoint was called without an authorization token.

The verified response was:

```
StatusCode : 200
Content    : Healthy
ContentType : text/plain
```

This confirms that the route is mapped, public, and returns the expected built-in response.

The temporary server was stopped after verification.

Request Logging Integration

Because RequestLoggingMiddleware wraps mapped endpoints, a health request also appears in the server logs:

```
HTTP GET /health responded 200 in ... ms
```

This can help operators see when monitoring systems are checking the API.

Elapsed time varies between machines and requests.

Swagger Behavior

The health endpoint may not appear in Swagger.

Swagger primarily discovers API actions described through controllers and endpoint metadata. The health route is mapped directly through middleware-oriented endpoint configuration.

Its absence from Swagger does not mean the endpoint is unavailable.

It can be tested with:

```
a web browser
PowerShell
curl
an uptime monitor
a deployment platform health probe
```

PowerShell Test

With the API running, a local check can use:

```
Invoke-WebRequest -Uri "https://localhost:PORT/health" -SkipCertificateCheck
```

The important values are:

```
StatusCode : 200
Content    : Healthy
```

The actual port comes from the API startup output.

Why No Database Check Yet

Database health checks add a stronger readiness signal, but they also add concerns:

```
connection timeouts
database load from frequent probes
credentials and configuration
whether temporary database failure should remove the API from service
separate liveness and readiness semantics
```

Day 83 avoids pretending those operational decisions are trivial.

The basic check is useful on its own and creates a clean foundation for later readiness checks.

Scope Of The Commit

Commit 1d6f8d9 changes only Program.cs.

It adds two lines:

```
service registration through AddHealthChecks
route mapping through MapHealthChecks
```

No controller, service, domain, persistence, authentication, or workflow file changes.

Verified Build And Tests

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The existing tests do not directly test the health endpoint, so the live HTTP verification provides evidence for the new route.

Future Test Opportunity

An integration test could start the application in memory and call:

```
GET /health
```

It could assert:

```
status is 200
body is Healthy
authentication is not required
no sensitive data is returned
```

This would protect the operational contract automatically.

What This Day Teaches

A useful health check should have a clear meaning.

The important principles are:

```
keep the first health signal simple
use framework support when it fits
avoid exposing internal details
make public probes safe
distinguish process health from dependency health
```

verify the route through a real HTTP request

The endpoint is small, but it is a practical step toward deployment and monitoring.

Reader Exercise

Run the API and call the health endpoint without a JWT.

Confirm:

```
HTTP status is 200
response body is Healthy
content type is text/plain
request logging records the call
the response contains no configuration or private data
```

Then explain in one sentence what the endpoint proves and what it does not prove.

Next Step

Add an automated integration test for the health endpoint, or design separate liveness and readiness checks before introducing database dependency checks.

Day 84 - Review Security Basics

Lesson Goal

Review JobTrackr's password, JWT, authentication, and authorization security, then fix one concrete configuration problem.

The review found that the development JWT signing key was tracked in appsettings.json.

Day 84 rotates that key, stores the replacement in .NET User Secrets, and removes the key from tracked configuration.

Security Review Summary

Area	Result	Notes
Password hashing	Pass	Passwords are hashed before storage
Password verification	Pass	Login verifies the submitted password against the stored hash
Password exposure	Pass	Response DTOs do not return passwords or hashes
JWT validation	Pass	Issuer, audience, lifetime, signature, and signing key are validated
JWT key storage	Fixed	The local key was removed from tracked configuration
Task authentication	Pass	TasksController requires authentication
Task ownership	Partial	List, get, create, update, and delete use the authenticated user
Complete/Reopen ownership	Not complete	Authentication is required, but ownership is not checked
User endpoint authorization	Not complete	UsersController remains public
Health endpoint	Intentional	The public route returns only Healthy

An honest security review distinguishes verified protection from partial or planned protection.

Why A JWT Signing Key Is Sensitive

The signing key creates and validates JWT signatures.

Anyone who obtains it may be able to create tokens that the API trusts. A signing key must therefore never be committed to a public repository.

Once a key has been tracked, removing it from the latest file is not enough. Earlier Git history may still contain it.

The safe response is:

```
treat the old value as compromised
generate a new value
store the replacement outside Git
stop using tokens signed with the old key
```

Initialize User Secrets

The API project is initialized for .NET User Secrets.

The project file contains a UserSecretsId:

```
<UserSecretsId>generated-identifier</UserSecretsId>
```

The identifier is safe to commit. It is a lookup identifier, not the signing key.

The actual secret value is stored outside the repository in the current developer's user profile.

Generate And Store A New Key

A new random local key is generated and stored under:


```
Jwt:Key
```

The value must not be copied into:

```
source code
appsettings files
documentation
commit messages
screenshots
videos
chat messages
```

After storage, temporary shell variables containing the value should be removed.

Remove The Tracked Key

Tracked appsettings now keeps only non-secret JWT configuration:

```
{
  "Jwt": {
    "Issuer": "JobTrackr",
    "Audience": "JobTrackr"
  }
}
```

The Key property is absent.

Verification confirmed:

```
appsettings contains only Issuer and Audience under Jwt
the project has a UserSecretsId
the local Jwt:Key secret name exists
the old development key is absent from current tracked files
```

The secret value itself was not printed or copied during verification.

Configuration Precedence

During local development, ASP.NET Core loads User Secrets and combines them with the other configuration sources.

The application still reads:

```
var key = configuration["Jwt:Key"];
```

The authentication and token services do not need a different interface. Only the configuration source changes.

This is a useful design property: application code requests a configuration key without needing to know whether its value came from JSON, User Secrets, an environment variable, or a production secret manager.

Old Tokens Become Invalid

Tokens signed with the removed key cannot be validated with the new key.

After rotation, users must log in again to receive newly signed tokens.

This is expected security behavior:

```
old signing key removed
new signing key loaded
old token signature no longer matches
new login produces a valid token
```

Key rotation can invalidate active sessions, so production rotations should be planned deliberately.

Password Hashing Review

JobTrackr uses ASP.NET Core's PasswordHasher.

Registration calls HashPassword before saving a user:

```
plain password
-> PasswordHasher
-> PasswordHash stored on User
```

The User entity stores PasswordHash rather than Password.

Login calls VerifyPassword with the submitted password and stored hash.

The password-hashing implementation is appropriate for the current project and does not need replacement during this review.

Password Exposure Review

Authentication and user response DTOs do not include:

```
Password
PasswordHash
```

Request logging records method, path, status, and elapsed time rather than request bodies.

This keeps passwords out of:

```
API response JSON
request logs
normal application output
```

Hashing protects stored passwords, while careful DTO and logging design prevents accidental disclosure through other channels.

JWT Creation And Validation

JwtTokenService reads issuer, audience, and key from configuration.

It creates identity claims for:

```
user identifier
email
full name
```

Tokens are signed with HMAC SHA-256 and currently expire after one hour.

The API validates:

```
issuer
audience
lifetime
signature
signing key
```

If the key is missing, token generation fails rather than silently creating an unsigned token.

Task Authentication

TasksController uses the Authorize attribute.

Anonymous access to task routes returns:

```
401 Unauthorized
```

For list, get, create, update, and delete operations, the authenticated user identifier is extracted from the token and passed into the application service.

These operations are designed around the current user's ownership boundary.

Known Task Ownership Gap

Complete and Reopen require authentication because the controller is protected.

However, those actions currently call service methods without the authenticated user identifier.

Therefore:

```
authentication: complete
ownership verification: not complete
```

The distinction matters. Requiring any valid user token is not the same as proving that the caller owns the requested task.

This gap should be fixed through separate focused implementation and tests.

Public User Endpoints

UserController is currently not protected by authorization.

That means its CRUD routes should not be described as secured simply because task routes use JWT authentication.

Future work must define who is allowed to:

```
list users
retrieve user details
create users outside registration
update a user
delete a user
view a user's task route
```

Authorization policy should be designed before adding attributes mechanically.

Public Health Endpoint

The health route remains intentionally public:

```
GET /health
```

It returns only:

```
Healthy
```

This allows deployment and monitoring systems to check process health without a user token while revealing no database, configuration, token, or user information.

Local And Production Secrets

.NET User Secrets are intended for local development.

A deployed environment should supply the signing key through a secure platform setting or secret manager.

ASP.NET Core maps:

```
Jwt__Key
```

to:

```
Jwt:Key
```

The double underscore allows hierarchical configuration keys to be represented in environment variables.

No deployment configuration is added during Day 84.

Scope Of The Commit

Commit cc23ece changes two tracked files:

```
JobTrackr.Api.csproj
appsettings.json
```

The project file adds the User Secrets identifier. The appsettings file removes the tracked key while preserving issuer and audience.

The local secret storage is outside the repository and is never committed.

No controller, service, entity, middleware, database, or workflow file changes.

Verified Result

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The local secret name exists, and the old tracked development key is absent from current repository files.

Remaining Test Gaps

The current tests verify password hashing and authentication service behavior, but future integration coverage could verify:

```
API startup fails clearly when the JWT key is absent
login creates a token using runtime configuration
anonymous task requests return 401
newly issued tokens can access protected routes
old tokens fail after key rotation
```

Secrets used by tests should also come from isolated test configuration, not a developer's personal User Secrets.

What This Day Teaches

Security reviews should produce specific evidence and scoped fixes.

The main lessons are:

```
hash passwords before storage
do not expose hashes in responses
validate JWT signatures and lifetime
never commit signing keys
rotate a key after exposure
separate local and production secret storage
distinguish authentication from ownership authorization
state known security limitations honestly
```

Moving a secret out of Git is a small code change with meaningful security value.

Reader Exercise

Audit another project for secret-bearing configuration.

For every sensitive setting, identify:

```
where the application reads it
where local development stores it
where production should store it
whether it has ever entered Git history
how it would be rotated
```

Do not print or copy actual secret values while performing the audit.

Next Step

Add ownership enforcement to Complete and Reopen, or design authorization rules for the currently public user endpoints with focused automated tests.

Day 85 - Clean Up Application Settings

Lesson Goal

Organize JobTrackr configuration by purpose, environment, and sensitivity.

After Day 85:

```
appsettings.json contains shared non-secret settings
appsettings.Development.json contains local development database configuration
.NET User Secrets contains the local JWT signing key
launchSettings.json starts with the HTTPS profile
tracked JSON contains no signing key or database password
```

No C# code needs to change because ASP.NET Core already combines these configuration sources through IConfiguration.

Why Configuration Organization Matters

Configuration often starts in one file because that is convenient during early development.

As an application grows, settings begin to differ in important ways:

```
some settings are shared across environments
some belong only to local development
some are secrets and must stay outside Git
some control local tooling rather than application behavior
```

Treating every value the same eventually creates security and deployment problems.

The Configuration Categories

JobTrackr now separates settings into four categories.

Shared Settings

Shared, non-secret values belong in appsettings.json.

Examples:

```
JWT issuer
JWT audience
logging levels
allowed hosts
```

Development Settings

Values specific to the local Development environment belong in appsettings.Development.json.

The local SQL Server connection string fits this category because it targets the developer's machine.

Local Secrets

Sensitive local values belong in .NET User Secrets.

The JWT signing key remains outside the repository under:

```
Jwt:Key
```

Launch Tooling

Local launch-profile behavior belongs in launchSettings.json.

This file controls which profile and URLs are convenient during local development. It is not production hosting configuration.

Shared appsettings.json

The shared settings file now contains:

```
{
  "Jwt": {
    "Issuer": "JobTrackr",
    "Audience": "JobTrackr"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

It intentionally does not contain:

```
JWT signing key
database connection string
database password
user credentials
issued JWT token
```

This file can be shared by Development and future deployed environments without assuming a local database location.

Development appsettings

appsettings.Development.json now contains the local SQL Server connection:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=JobTrackrDb;Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

The connection uses Windows authentication and does not contain a username or password.

That makes it acceptable for this tracked development file.

If a future connection string includes a password, the entire connection string should move to User Secrets or another secure provider.

Why Move The Connection String

The local connection string is not a universal application setting.

It assumes:

```
SQL Server is available on localhost
```

```
the database is named JobTrackrDb
Windows authentication is available
the local certificate should be trusted
```

These assumptions belong to the Development environment, not every environment where JobTrackr may run.

Moving the value makes future deployment configuration clearer.

Keep The JWT Key In User Secrets

Day 84 removed the tracked signing key and created a local replacement in .NET User Secrets.

Day 85 preserves that boundary.

Verification confirms:

```
the local Jwt:Key secret name exists
no tracked JSON file contains a JWT key
the real secret value was not printed
```

The key does not move into appsettings.Development.json merely because that file is development-specific. It remains a tracked file, so it is not a secret store.

HTTPS As The Default Profile

The launch profiles now appear in this order:

```
https
http
IIS Express
```

The HTTPS profile listens on:

```
https://localhost:7024
http://localhost:5123
```

Program.cs uses HTTPS redirection.

When only the HTTP profile is selected, ASP.NET Core may not have an HTTPS address to use and can log:

```
Failed to determine the https port for redirect.
```

Making the existing HTTPS profile first gives local startup a valid redirect target while preserving the optional HTTP profile.

What Launch Settings Do Not Control

launchSettings.json is used by local development tooling.

It does not define:

```
production reverse proxy configuration
container port mappings
cloud platform health probes
production TLS certificates
deployment environment variables
```

Reordering a local profile improves development startup without changing production authentication or middleware behavior.

Configuration Loading Order

In Development, ASP.NET Core combines several sources.

For JobTrackr, the important progression is:

```
appsettings.json
appsettings.Development.json
User Secrets
environment variables
command-line values
```

Later sources can override values from earlier sources.

The resulting configuration includes:

```
Jwt:Issuer from appsettings.json
Jwt:Audience from appsettings.json
ConnectionStrings:DefaultConnection from appsettings.Development.json
Jwt:Key from User Secrets
```

Program.cs continues reading these values through `builder.Configuration`.

Why Program.cs Does Not Change

The application asks configuration for logical keys:

```
builder.Configuration.GetConnectionString("DefaultConnection")
builder.Configuration["Jwt:Issuer"]
builder.Configuration["Jwt:Audience"]
builder.Configuration["Jwt:Key"]
```

The code does not need to know which provider supplied each value.

This separates application behavior from environment-specific storage.

The same code can later use environment variables or a managed secret store without rewriting authentication or persistence services.

JSON Validation

All three edited JSON files were parsed successfully:

```
appsettings.json
appsettings.Development.json
launchSettings.json
```

Parsing configuration during review catches missing commas, braces, or quotation marks before runtime startup.

Secret Scan

Current tracked JSON was searched for:

```
JWT key properties
database Password values
database Pwd values
```

The result:

```
no JWT key found in tracked JSON
no database password found in tracked JSON
```

The development connection string is present, but it uses password-free Windows authentication.

Development Runtime Behavior

When the API runs with the HTTPS launch profile and the Development environment:

```
shared JWT metadata loads from appsettings.json
the local database connection loads from appsettings.Development.json
the signing key loads from User Secrets
HTTPS and HTTP addresses are available
```


This configuration supports:

- health checks
- SQL Server-backed login
- JWT token creation
- authenticated task requests
- HTTPS redirection

Old JWT tokens from before the Day 84 key rotation remain invalid and users must log in again.

Production Configuration

appsettings.Development.json is loaded only when the environment is Development.

A future deployment should provide its own values through environment variables or a secret manager.

ASP.NET Core maps:

```
ConnectionStrings__DefaultConnection
Jwt__Key
```

to:

```
ConnectionStrings:DefaultConnection
Jwt:Key
```

The double underscore represents a configuration hierarchy in environment-variable form.

Production configuration is deliberately outside Day 85.

Scope Of The Commit

Commit cd2bde5 changes three files:

```
appsettings.json
appsettings.Development.json
Properties/launchSettings.json
```

The commit contains 6 additions and 12 removals.

No changes are made to:

```
Program.cs
controllers
services
entities
migrations
packages
GitHub Actions
```

This is a configuration organization change rather than an application behavior change.

Verified Result

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

The automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

The repository working tree is clean after commit cd2bde5.

Operational Boundaries

This cleanup improves organization, but it does not create production configuration.

Before deployment, the project still needs decisions about:

- production SQL Server location
- production database credentials
- JWT key storage and rotation
- allowed hosts
- logging destinations and retention
- HTTPS termination
- environment-specific health checks

Keeping those concerns out of local files is the first useful step.

What This Day Teaches

Configuration should be grouped by meaning rather than convenience.

The main lessons are:

- shared non-secret settings belong in shared configuration
- local environment values belong in environment configuration
- secrets belong outside Git
- launch profiles are local tooling
- configuration providers let code remain unchanged
- tracked files should be scanned for accidental secrets

Good configuration organization makes both security and deployment easier to reason about.

Reader Exercise

Audit the configuration sources in another ASP.NET Core project.

Classify each setting as:

- shared and non-secret
- environment-specific and non-secret
- local secret
- production secret
- local launch tooling

Move only one category at a time, validate every JSON file, scan tracked files for secrets, and run the build and tests afterward.

Next Step

Add configuration validation with clear startup errors, or begin designing production-ready environment variables and secret-provider requirements without committing real deployment credentials.

Day 86 - Document The Local Environment

Lesson Goal

Document everything another developer needs to configure and run JobTrackr locally.

The repository README now covers:

- required development software
- dependency restoration
- local SQL Server configuration
- EF Core migrations
- JWT signing-key setup

```
development HTTPS trust
build and test commands
API startup
health verification
registration and login
Bearer-token requests
local secret safety
```

Day 86 changes documentation only. It does not change application behavior.

Why Local Setup Documentation Matters

A repository is not fully usable when only its original developer knows how to run it.

Source code may compile, but a new developer can still become blocked by:

```
missing SDKs
missing command-line tools
database configuration
unapplied migrations
missing secret values
untrusted HTTPS certificates
unknown ports
authentication requirements
```

A good setup guide turns those hidden assumptions into explicit, repeatable steps.

The Configuration Model

JobTrackr now loads local configuration from several places:

Purpose	Source
Shared non-secret settings	appsettings.json
Local SQL Server settings	appsettings.Development.json
Local JWT signing key	.NET User Secrets
Development URLs and profiles	launchSettings.json

The README explains each source without publishing the real JWT key.

Prerequisites

The local guide lists:

```
.NET SDK capable of building net8.0
SQL Server
SQL Server Management Studio or another SQL client
EF Core command-line tools 8.x
Git
```

Developers can verify their tools:

```
dotnet --version
dotnet ef --version
```

If the EF Core tool is absent, the documented installation command pins the existing project-compatible version:

```
dotnet tool install --global dotnet-ef --version 8.0.27
```

Matching the major EF Core version reduces tooling and migration compatibility surprises.

Restore Dependencies

From the repository root, restore the production and test projects:

```
dotnet restore src\JobTrackr.Api\JobTrackr.Api.csproj
dotnet restore tests\JobTrackr.Tests\JobTrackr.Tests.csproj
```

The guide uses a placeholder repository path rather than the original developer's local path.

This keeps the instructions portable between machines.

Configure SQL Server

The Development environment reads its connection string from:

```
src/JobTrackr.Api/appsettings.Development.json
```

The default local connection targets:

```
Server: localhost
Database: JobTrackrDb
Authentication: Windows authentication
```

The tracked development connection string contains no database username or password.

The guide explains that SQL Server must be running and the current Windows user must be allowed to create or update the database.

Apply EF Core Migrations

The documented migration command is:

```
dotnet ef database update --project src\JobTrackr.Infrastructure --startup-project src\
JobTrackr.Api
```

The two project arguments have different responsibilities:

```
Infrastructure project: contains AppDbContext and migrations
API startup project: supplies runtime configuration and dependency setup
```

Applying migrations creates or updates JobTrackrDb using the migration history already stored in the repository.

Configure A Local JWT Signing Key

The repository intentionally contains no JWT signing key.

Each developer generates a unique local value:

```
$jwtKey = [Guid]::NewGuid().ToString("N") + [Guid]::NewGuid().ToString("N")
dotnet user-secrets set "Jwt:Key" "$jwtKey" --project src\JobTrackr.Api
Remove-Variable jwtKey
```

The command generates the value on the developer's machine instead of distributing a shared development secret through documentation.

The UserSecretsId is tracked, but the key value is stored outside the repository in the current user's profile.

Why The README Does Not Include A Sample Key

A copyable signing key in documentation tends to become a shared secret across multiple machines and environments.

Generating a fresh value provides better habits:

```
each developer has a different key
```

```
the key never enters Git
the key is not exposed in screenshots
the key is not copied through chat
tokens are tied to the developer's local environment
```

The setup guide documents the process, not the secret.

Trust Development HTTPS

The README includes:

```
dotnet dev-certs https --trust
```

This trusts the local ASP.NET Core development certificate after the user accepts the operating-system prompt.

The step prevents avoidable browser and PowerShell certificate errors while testing the HTTPS launch profile.

Development certificates are for local use and are not production TLS certificates.

Build And Test

The documented commands are:

```
dotnet build src\JobTrackr.Api\JobTrackr.Api.csproj --configuration Release
dotnet test tests\JobTrackr.Tests\JobTrackr.Tests.csproj --configuration Release
```

The verified result is:

```
Build succeeded.
0 Warning(s)
0 Error(s)

Passed: 15
Failed: 0
Skipped: 0
```

The service tests use EF Core InMemory and do not connect to SQL Server.

This means developers can verify the existing automated behavior independently from local database availability.

Run The API

The README starts JobTrackr through the HTTPS profile:

```
dotnet run --project src\JobTrackr.Api --launch-profile https
```

The default local addresses are:

```
https://localhost:7024
http://localhost:5123
```

The HTTPS profile supplies a valid redirect target for UseHttpsRedirection.

Swagger And Health URLs

Swagger is available in the Development environment at:

```
https://localhost:7024/swagger
```

The public health endpoint is:

```
https://localhost:7024/health
```

Its expected response is:

```
Healthy
```

These URLs give a new developer two quick startup checks:

Swagger proves the development API documentation is available
health proves the application process can answer a request

Registration And Login

The setup guide describes the authentication sequence:

```
POST /api/auth/register
-> create a local user

POST /api/auth/login
-> authenticate with email and password
-> receive a JWT
```

The token is then sent as:

```
Authorization: Bearer TOKEN
```

Without a valid token, protected task routes return 401 Unauthorized.

Accurate Swagger Limitation

The README does not claim that Swagger currently has a Bearer-token Authorize button.

The existing Swagger registration has not added a Bearer security definition.

Swagger can still be used for registration and login, while an authenticated task request can be sent through PowerShell:

```
$token = "PASTE_LOGIN_TOKEN_HERE"
Invoke-RestMethod -Uri "https://localhost:7024/api/tasks" -Headers @{ Authorization = "Bearer $token" }
Remove-Variable token
```

The placeholder is intentionally not a real token.

Why Remove The Token Variable

The example removes the temporary shell variable after use.

This does not erase every possible command-history or process-memory trace, but it avoids leaving the token available in the active PowerShell session.

The larger safety rule remains:

```
do not commit tokens
do not paste tokens into documentation
do not publish terminal output containing tokens
```

Local Configuration Safety

The README explicitly prohibits committing:

```
JWT signing keys
database passwords
access tokens
user passwords
production connection strings
```

Local secrets belong in User Secrets.

Future deployment secrets should come from environment variables or a secure secret manager.

Technical Accuracy Checks

The documentation was checked against the repository:

```
API target framework: net8.0
EF Core command-line tools: 8.0.27
database: JobTrackrDb
local database authentication: Windows authentication
HTTPS port: 7024
HTTP port: 5123
Swagger route: /swagger
health route: /health
current test count: 15
task endpoints: Bearer authentication required
Swagger Bearer button: not configured
```

This prevents the guide from promising features or paths that do not exist.

Secret Scan

Tracked JSON and Markdown were checked for:

```
the removed development JWT key
database Password values
database Pwd values
```

No old key or database password was found.

The README contains only a command that generates a new random JWT key on each developer's machine.

Scope Of The Commit

Commit f60f13c changes only README.md:

```
174 lines added
6 lines removed
```

No changes are made to:

```
C# source code
configuration files
migrations
packages
database model
GitHub Actions
```

The narrow scope makes the documentation update easy to review.

What Was Verified

The local environment section was compared with:

```
project target framework
installed EF Core tool version
development connection settings
launch profile URLs
health route
Swagger configuration
authentication design
current test count
tracked secret scan
```

The documented Release build and test commands completed successfully.

The database-update command is intentionally an environment-changing operation. Developers should run it against their own intended local SQL Server instance after reviewing the connection configuration.

What This Day Teaches

Environment documentation is part of software delivery.

The main lessons are:

- document prerequisites explicitly
- separate restore, database, secret, certificate, and startup steps
- use placeholders instead of personal machine paths
- generate secrets locally instead of publishing samples
- state tooling limitations accurately
- verify documentation against the repository
- run safe documented commands before publishing

A reliable setup guide reduces onboarding time and proves that the project is understandable beyond the machine where it was created.

Reader Exercise

Follow the README from a fresh PowerShell session.

Record every step that depends on knowledge not written in the guide.

For each gap, add:

- the required prerequisite
- the exact command
- the expected result
- a safe troubleshooting note

Never add a real credential merely to make setup appear easier.

Next Step

Add Swagger Bearer-token support so authenticated API testing can be completed directly in Swagger, or create an automated integration test that verifies startup and the health route using isolated configuration.

Day 87 - Manual API Regression Test

Lesson Goal

Manually test the current JobTrackr API from startup through its main success, validation, authentication, ownership, and error paths.

Day 87 verifies:

- API health
- registration and login
- JWT authentication
- request validation
- task create, read, update, delete, complete, and reopen
- task search and completion filters
- two-user task ownership
- user CRUD
- structured 400 and 404 responses
- anonymous 401 responses
- safe request logging
- the complete automated test suite

This is a testing day. No repository file changes when the API behaves as expected.

Why Manual Regression Testing Still Matters

Automated tests are repeatable and fast, but the current suite focuses on application and infrastructure services.

It does not yet exercise the complete HTTP pipeline:

```
routing
DTO model binding
automatic validation
JWT authentication
controller responses
ProblemDetails serialization
request logging
SQL Server persistence
HTTPS hosting
```

A focused manual regression run connects those pieces and verifies that the API behaves as a complete system.

Keep Known Limitations Visible

Task ownership currently protects:

```
Create
Get All
Get By Id
Update
Delete
```

Complete and Reopen require authentication but do not pass the current user identifier to their service methods.

UserController also remains public.

The regression test records those facts instead of presenting the API as fully authorized.

Start From A Known Baseline

Before making requests, build and run the API through the HTTPS profile:

```
dotnet build src\JobTracker.Api\JobTracker.Api.csproj --configuration Release
dotnet run --project src\JobTracker.Api --launch-profile https
```

Expected addresses:

```
https://localhost:7024
http://localhost:5123
```

The test uses the HTTPS address.

Use Unique Test Data

Two test users are created with timestamp-based email addresses:

```
Regression User A
Regression User B
```

Unique addresses prevent duplicate-email failures from earlier regression runs.

The data belongs to the local development database and should never contain a real person's credentials.

Health Check

The first request is:

```
GET /health
```

Expected result:

```
200 OK
Healthy
```

Testing health first confirms the API process is reachable before authentication and persistence behavior are tested.

Request logging should record the call without exposing configuration.

Register Two Users

The regression creates User A and User B through:

```
POST /api/auth/register
```

Each response should include safe identity information:

```
user id
full name
email
```

It must not include:

```
plain password
password hash
JWT signing key
```

Creating two accounts provides the identities needed to test ownership isolation.

Registration Validation

An invalid registration uses:

```
{
  "fullName": "",
  "email": "invalid-email",
  "password": "123"
}
```

Expected result:

```
400 Bad Request
ValidationProblemDetails with field errors
no user created
```

This checks ApiController validation before ordinary controller logic executes.

Login And Token Handling

Both users log in through:

```
POST /api/auth/login
```

Each successful response returns a non-empty JWT.

The tokens are assigned to temporary PowerShell variables and used only through Authorization headers:

```
Authorization: Bearer TOKEN
```

Token values should not be printed, included in screenshots, committed, or copied into documentation.

Anonymous Access

Calling the task collection without a token must return:

```
401 Unauthorized
```

This proves the TasksController authorization boundary is active at the HTTP layer.

An authenticated route that accidentally succeeds anonymously is a high-priority regression.

Create A User-Owned Task

User A creates a task through:

```
POST /api/tasks
```

The request includes:

```
title
description
due date
priority
```

It intentionally does not include UserId.

The API reads User A's identifier from the JWT claim and assigns ownership internally.

Expected result:

```
201 Created
submitted values returned
task belongs to User A
task begins incomplete
```

Invalid Task Validation

Creating a task with an empty title should return:

```
400 Bad Request
field-level Title validation error
```

The request still includes User A's valid token because authentication and DTO validation are separate concerns.

Task Query Regression

User A verifies:

```
Get All returns the new task
search finds the task by title
isCompleted=false includes the task
combined incomplete and search filters include it
Get By Id returns the correct task
```

This checks both query behavior and the authenticated-user scope.

Two-User Ownership Isolation

User B then attempts to observe User A's task.

Expected behavior:

```
User B's list does not include User A's task
User B Get By Id returns 404
ProblemDetails says Task not found
```

Returning 404 instead of 403 avoids confirming that another user's resource exists.

This is an important test because successful authentication alone does not prove correct authorization.

Update Ownership

User A updates:

```
title
description
due date
priority
```

Expected result:

```
200 OK
updated values returned
```

User B then attempts the same update and receives 404.

This verifies that TaskService receives the authenticated user identifier for updates.

Delete Ownership

User B attempts to delete User A's task and receives:

```
404 Not Found
```

User A can still retrieve the task afterward.

This confirms the rejected request did not mutate the resource.

Testing the final state is as important as checking the status code.

Complete And Reopen

User A completes the task:

```
PATCH /api/tasks/{id}/complete
```

The response should show:

```
isCompleted = true
```

The completed filter should include it.

User A then reopens the task:

```
PATCH /api/tasks/{id}/reopen
```

The response should show:

```
isCompleted = false
```

Why Cross-User Complete/Reopen Is Not Run

User B is deliberately not used to call Complete or Reopen on User A's task.

Those endpoints currently lack ownership checks and could mutate User A's data.

Running a known unsafe request merely to prove the vulnerability would create unwanted development data changes.

The correct regression note is:

```
authentication is required
ownership is not enforced
focused implementation and tests are still needed
```

User CRUD Regression

A separate temporary user is used for public user CRUD testing.

The flow verifies:

```
POST creates a user with 201
```

```
GET collection includes the user
GET by id returns the user
GET user tasks returns an empty collection
PUT updates the user
DELETE returns 204
GET after deletion returns structured 404
```

Separate temporary data prevents this portion of the test from deleting either authenticated regression account.

Public User Endpoint Limitation

These requests currently work without a JWT.

That is existing behavior, not a security success.

The regression test confirms functional CRUD while keeping the authorization limitation explicit:

```
user CRUD behavior works
user CRUD authorization remains incomplete
```

Future work must define who may list, update, and delete users.

Delete The Task

User A deletes their own task:

```
DELETE /api/tasks/{id}
```

Expected result:

```
204 No Content
```

A later Get By Id should return:

```
404 Not Found
ProblemDetails detail: Task not found.
```

This completes the task lifecycle and confirms deletion persists.

Review Request Logs

The API terminal should contain a mix of successful and failed status codes:

```
GET /health -> 200
POST /api/auth/register -> 200
GET /api/tasks without token -> 401
POST /api/tasks -> 201
cross-user task operation -> 404
DELETE /api/tasks/{id} -> 204
```

Logs should not contain:

```
passwords
JWT tokens
Authorization headers
request bodies
connection strings
JWT signing keys
```

The regression verifies both observability and confidentiality.

Clear Sensitive Shell Variables

After testing, temporary variables containing tokens, headers, passwords, and request bodies are removed from the active PowerShell session.

Examples include:

```
tokenA
tokenB
headersA
headersB
password
login request bodies
registration request bodies
```

Removing variables is a useful cleanup step, though developers should also avoid publishing command history or terminal output containing secrets.

Automated Test Result

After the manual flow, the complete test project is run in Release configuration.

The verified result is:

```
Passed: 15
Failed: 0
Skipped: 0
```

Manual and automated testing provide different evidence:

```
manual regression: complete HTTP and SQL Server workflow
automated tests: repeatable service-level behavior
```

Both are valuable until broader integration tests are added.

Regression Checklist

The Day 87 checklist covers:

```
health returns Healthy
two users register and log in
invalid registration returns 400
anonymous task access returns 401
task creation derives ownership from JWT
invalid task returns 400
task list and filters work
User B cannot list, get, update, or delete User A's task
User A can complete and reopen
temporary user CRUD works
missing resources return structured 404
task deletion works
logs contain metadata but no secrets
all automated tests pass
```

A written checklist makes the manual run reproducible rather than relying on memory.

No Repository Change

Day 87 does not change source code or tracked documentation in the JobTrackr repository.

The latest source commit remains:

```
f60f13c Day 86: Document local environment setup
```

Git remains clean, so no empty commit is created.

The manual test does create temporary rows in the local development database. Those rows are runtime test data, not repository changes.

What This Day Teaches

Regression testing should verify both success and failure behavior.

The main lessons are:

```
start with a known running baseline
use unique and disposable test data
test unauthenticated and authenticated paths
use two identities for ownership checks
verify state after rejected mutations
do not perform knowingly unsafe mutations
keep known security gaps explicit
review logs for both status coverage and secret safety
combine manual HTTP testing with automated tests
avoid empty commits on review-only days
```

The result is an evidence-based picture of what JobTrackr currently protects and what still needs work.

Reader Exercise

Turn the manual checklist into a reusable test worksheet.

For each request, record:

```
acting user
HTTP method and route
expected status
expected response shape
expected database state
whether ownership is involved
whether sensitive data could appear
```

Then identify the highest-value cases to convert into automated integration tests.

Next Step

Fix ownership checks for Complete and Reopen with two-user automated tests, or begin protecting UsersController through explicit authorization requirements.

Day 88 - Cleanup Before The Month Review

Lesson Goal

Perform a small, behavior-preserving cleanup before the month review.

The repository audit found no dead starter code that needed deletion. Day 88 therefore focuses on formatting consistency, whitespace cleanup, and verification.

The key principle is:

```
do not delete valid code merely to make a cleanup commit look substantial
```

Repository Audit

The source and test folders were searched while excluding generated bin and obj directories.

The audit found:

```
no WeatherForecast starter code
no Class1 placeholder files
no empty C# source files
no TODO placeholders
no HACK placeholders
no NotImplementedException
no untracked changes
no obvious dead application classes
```

This is a successful cleanup result. A repository does not require deletion when the files are already meaningful.

Why Generated Folders Are Excluded

The bin and obj directories contain compiler and build-tool output.

Searching them can produce misleading matches because generated files may contain:

```
copied source metadata
generated assembly information
compiler caches
intermediate files
restored package artifacts
```

Those folders are ignored by Git and are not part of the hand-maintained source audit.

Cleanup Scope

Day 88 changes only:

```
ProblemDetails call formatting
one unnecessary blank line
```

It does not change:

```
API routes
method parameters
status codes
error messages
authorization attributes
service calls
DTO properties
database entities
migrations
settings
tests
```

This narrow scope makes behavioral review straightforward.

Consistent ProblemDetails Formatting

Several controller responses were written on one long line:

```
return Problem(statusCode: StatusCodes.Status400BadRequest, title: "Bad Request", detail: ex.
Message);
```

They now use a consistent multi-line style:

```
return Problem(
    statusCode: StatusCodes.Status400BadRequest,
    title: "Bad Request",
    detail: ex.Message);
```

The same pattern is used for not-found errors:

```
return Problem(
    statusCode: StatusCodes.Status404NotFound,
    title: "Not Found",
    detail: ErrorMessages.TaskNotFound);
```

Only presentation changes. The named arguments and their values remain identical.

AuthController Cleanup

The Register and Login actions both catch known argument errors.

Their 400 ProblemDetails calls are now formatted over multiple lines.

The following behavior remains unchanged:

```
status: 400 Bad Request
title: Bad Request
```



```
detail: existing safe service message
```

Registration and login logic are untouched.

TasksController Cleanup

ProblemDetails formatting is standardized in:

```
Create
Update
Delete
Complete
Reopen
```

The existing Get By Id response already used the desired layout.

The cleanup does not change:

```
authenticated user extraction
ownership filtering
task service calls
404 behavior
400 behavior
successful response types
```

Complete and Reopen also retain their current ownership limitation. Formatting does not improve authorization.

UsersController Cleanup

ProblemDetails calls are formatted consistently for:

```
Get By Id
Get Tasks By User Id
Create
Update
Delete
```

Both bad-request and user-not-found responses now follow the same visual structure.

UsersController remains publicly accessible. Day 88 does not quietly add authorization during a formatting cleanup.

DTO Whitespace Cleanup

CreateTaskRequest contained an extra blank line before the closing class brace.

The blank line was removed.

The Priority property remains:

```
[Required]
[MaxLength(20)]
public string Priority { get; set; } = "Medium";
```

Its type, validation attributes, and default value are unchanged.

Namespace And Using Review

The edited files retain their existing namespaces:

```
JobTrackr.Api.Controllers
JobTrackr.Application.Tasks
```

No duplicate or stale using directive was introduced.

The review avoids removing imports based on guesswork. A using directive should only be removed when tooling or the compiler confirms it is unused.

Preserve EF Core Migrations

Migration files are generated history used by Entity Framework Core.

Files under the migrations directory may not be referenced directly by a controller, but they remain essential for:

- creating a database from scratch
- upgrading an existing database
- tracking schema history
- generating migration scripts
- understanding model changes

They are not dead code simply because normal request handling does not call them directly.

Preserve Architectural Files

The cleanup also keeps:

- DTOs referenced by interfaces
- service interfaces used by dependency injection
- middleware registered in Program.cs
- entities referenced by ApplicationDbContext
- GitHub Actions workflow files
- test infrastructure

Repository cleanup requires understanding indirect usage, configuration-based usage, and framework conventions.

Deleting a valid file can create a delayed failure that does not appear in the edited location.

Review The Diff, Not Only The Intent

The commit diff was inspected to confirm every change was visual.

The review checked that the diff did not modify:

- route templates
- HTTP methods
- status constants
- message constants
- authentication attributes
- service method names
- method arguments
- DTO members

An author may intend a formatting-only change, but the diff is the evidence.

Whitespace Verification

Git's whitespace checker reported no errors.

This detects issues such as:

- trailing whitespace
- space-before-tab problems
- invalid blank-line whitespace
- conflict markers in some workflows

Clean formatting should not introduce a new category of formatting defect.

Scope Of The Commit

Commit e81e388 changes four files:

```
AuthController.cs
TasksController.cs
UsersController.cs
CreateTaskRequest.cs
```

The commit contains:

```
56 inserted formatting lines
15 removed lines
```

Most added lines come from wrapping long method calls. The semantic content remains the same.

Verified Build

The API builds successfully in Release configuration:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

A formatting cleanup should never leave the project with compiler warnings or errors.

Verified Tests

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

Tests provide additional evidence that the cleanup did not alter the covered behavior.

They do not replace diff review, because the test suite does not cover every controller response format or route.

Final Audit Result

After the change:

```
no starter or placeholder patterns are present
no empty C# source files are present
no whitespace errors are present
Release build succeeds
all tests pass
Git working tree is clean
```

The repository is ready for the upcoming month review without unnecessary deletion or refactoring.

Why Small Cleanup Commits Are Valuable

A small cleanup commit can improve:

```
readability
review speed
future diff clarity
consistency between neighboring files
confidence before a larger review
```

Its value does not depend on changing many files.

Keeping formatting separate from behavior also makes future debugging easier because a reader can see that this commit was not intended to alter application logic.

What This Day Teaches

Cleanup is a technical judgment exercise, not a file-deletion contest.

The main lessons are:

- audit before editing
- exclude generated output from source searches
- remove only genuinely unnecessary content
- preserve framework and architecture files
- keep formatting changes behavior-free
- inspect the actual diff
- run whitespace checks
- build and test afterward
- avoid unrelated refactoring

A disciplined cleanup leaves the project easier to read without making its behavior harder to trust.

Reader Exercise

Audit another repository for:

- starter templates
- empty source files
- TODO and HACK placeholders
- NotImplementedException
- inconsistent formatting
- stale namespaces
- generated files mistaken for dead code

For every proposed deletion, identify how the file is referenced directly, through dependency injection, by configuration, by migrations, or by framework convention.

Delete nothing until its role is understood.

Next Step

Use the clean repository baseline for the month review, summarizing architecture, authentication, ownership, testing, CI, logging, error handling, configuration, documentation, and remaining security work.

Day 89 - Update README And Progress

Lesson Goal

Update JobTrackr's public documentation so it accurately reflects the API after Days 80 through 88.

The README now documents:

- request logging
- ProblemDetails error responses
- the public health endpoint
- User Secrets
- environment-specific configuration
- local environment setup
- manual regression status
- the remaining authorization work

The separate daily progress record also receives a Day 89 entry.

Why Documentation Needs Regular Reviews

Documentation can become inaccurate even when every individual lesson is documented somewhere.

Before Day 89, the health endpoint appeared only in local setup instructions. A reader scanning the primary feature and endpoint lists could miss it.

Request logging, health checks, and environment documentation were complete, but some still appeared as future work.

This creates two common documentation failures:

```
completed capabilities are difficult to discover
the roadmap lists work that is no longer unfinished
```

A periodic documentation review reconnects the README with the current product.

Opening Summary

The repository introduction now identifies the major current capabilities:

```
users
user-owned tasks
SQL Server persistence
JWT authentication
structured error handling
request logging
health checks
automated service tests
GitHub Actions CI
```

This gives a reader a useful first impression without requiring them to inspect the complete endpoint list.

Current Features

The feature list now includes the reliability and configuration work completed during the recent lessons:

```
safe global ProblemDetails exception responses
structured ProblemDetails for known API errors
field-level DTO validation
request logging for method, path, status, and elapsed time
public GET /health endpoint
environment-specific local configuration
JWT signing key stored through User Secrets
documented local database, migration, HTTPS, and authentication setup
xUnit service tests
GitHub Actions build and test execution
```

Only completed behavior belongs in this section.

Make Health Discoverable

The primary endpoint list now begins with:

```
GET /health
```

The README explains that it:

```
is public
returns 200 OK
returns Healthy
confirms the API process is responding
may not appear in Swagger because it is mapped directly
```

The guide does not claim that this basic endpoint checks SQL Server or every dependency.

Complete Endpoint Inventory

The README lists the current routes by category.

Health

GET /health

Authentication

POST /api/auth/register
POST /api/auth/login

Tasks

GET /api/tasks
POST /api/tasks
GET /api/tasks/{id}
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
PATCH /api/tasks/{id}/complete
PATCH /api/tasks/{id}/reopen

The list also retains completion and title-search examples.

Users

GET /api/users
POST /api/users
GET /api/users/{id}
GET /api/users/{userId}/tasks
PUT /api/users/{id}
DELETE /api/users/{id}

Reviewing documentation against controllers and Program.cs prevents route drift.

API Reliability Behavior

A new section explains the API's operational and error behavior:

invalid DTOs return 400 with field-level errors
known bad requests use ProblemDetails
missing resources use ProblemDetails
unexpected exceptions return a safe 500
request logs record safe request metadata
request logs exclude bodies and credentials
GET /health provides basic process health

This brings several cross-cutting concerns together in one discoverable place.

Safe Error Documentation

The README explains what clients receive without documenting internal exception details.

Known errors use a structured response, while unexpected failures return a generic message.

It does not expose:

stack traces
database exception details
connection strings
signing keys
passwords
tokens

Public documentation should describe the contract, not reveal private diagnostics.

Request Logging Documentation

Request logging is now a current feature rather than a planned item.

The README records the fields:

HTTP method
request path

```
status code
elapsed time
```

It also records what is deliberately excluded:

```
request bodies
passwords
JWT tokens
Authorization headers
signing keys
```

This communicates both usefulness and privacy boundaries.

Configuration Section

The new Configuration section summarizes where settings belong:

```
shared non-secret settings: appsettings.json
local SQL connection: appsettings.Development.json
local JWT signing key: .NET User Secrets
default local launch: HTTPS profile
future production secrets: environment variables or secret manager
```

This short overview complements the longer local environment setup instructions.

Why Configuration Belongs In The README

Configuration location affects whether another developer can run the application safely.

A reader should not need to infer:

```
which settings are committed
which settings are environment-specific
which value must be generated locally
where production secrets will eventually come from
```

The README makes those boundaries explicit without containing real credentials.

Manual Regression Status

The Automated Tests section still reports fifteen xUnit service tests.

It now separately records that the API passed a manual regression flow covering:

```
health
registration and login
validation
task CRUD and filters
two-user ownership isolation
user CRUD
structured errors
request logging
```

The distinction matters:

```
automated suite: repeatable service tests
manual regression: broader HTTP and SQL Server verification
```

The README does not inflate the automated test count or imply that the manual flow runs in CI.

Keep Authorization Limitations Explicit

The documentation still states:

```
Complete and Reopen require authentication but do not check ownership
User CRUD endpoints are not protected by authorization
```

These limitations are not hidden by the larger feature list.

A project can have JWT authentication and still contain incomplete resource authorization.

Accurate documentation helps readers understand that difference.

Correct The Roadmap

Completed work is removed from Next Steps.

The roadmap now includes:

- owner protection for Complete and Reopen
- stronger authorization for user endpoints
- Swagger Bearer-token configuration
- broader controller and integration tests
- database-aware health checks when useful
- deployment fundamentals

Request logging, the basic health endpoint, and local environment documentation no longer appear as future work.

Why Roadmap Accuracy Matters

A stale roadmap makes project planning less useful.

It can cause:

- duplicate implementation work
- confusing project evaluations
- incorrect interview explanations
- unclear priorities
- loss of confidence in documentation

Moving completed items into current capabilities keeps the next steps actionable.

Security Review

Tracked JSON and Markdown were searched for:

- the removed development JWT key
- database Password values
- database Pwd values

No old key or database password was found.

The README includes secret-generation instructions but no actual JWT signing key, access token, user password, or production connection string.

Separate Progress Record

The daily learning record receives a Day 89 entry outside the application repository.

It records:

- the README sections updated
- the health endpoint becoming discoverable
- the successful secret check
- the healthy build and test result
- the Day 90 month review as the next task

Keeping the learning journal separate from the repository avoids mixing personal daily notes with public project documentation.

Scope Of The Repository Commit

Commit fed948d changes only README.md:


```
41 lines added
6 lines removed
```

PROGRESS.md is stored outside the JobTrackr repository and is not part of the application commit.

No changes are made to:

```
C# code
configuration
migrations
packages
database model
tests
GitHub Actions
```

Verified Build

Documentation is published from a healthy source baseline.

The Release build result is:

```
Build succeeded.
0 Warning(s)
0 Error(s)
```

Verified Tests

The complete automated suite passes:

```
Passed: 15
Failed: 0
Skipped: 0
```

Git remains clean after the committed documentation update.

What This Day Teaches

Documentation maintenance is part of engineering work.

The main lessons are:

```
make important endpoints easy to discover
move completed work into current features
remove completed work from future plans
separate automated and manual test claims
keep security limitations visible
describe configuration without publishing secrets
compare endpoint documentation with actual routes
verify build and tests before publishing
keep personal progress records separate from public project files
```

A README should describe the project that exists today, not a mixture of old behavior and future intentions.

Reader Exercise

Audit another project's README using three columns:

```
documented as current
verified in code
still planned
```

For every item, move it into the correct category.

Then check:

```
endpoint discoverability
security limitations
test counts
configuration sources
secret exposure
```

roadmap accuracy

Run the documented build and test commands before publishing the update.

Next Step

Complete the Month 3 review by summarizing the architecture, authentication, ownership, testing, CI, reliability, configuration, documentation, and highest-priority remaining work.

Day 90 - Month 3 Review

Review Goal

Close the third 30-day backend phase by verifying that authentication, task ownership, testing, continuous integration, API reliability, configuration, security, and documentation form a stable foundation.

Day 90 is a review day.

It deliberately avoids:

- adding a feature
- redesigning architecture
- expanding test scope
- starting pagination early
- creating an empty commit

The purpose is to establish what is genuinely complete, what evidence supports it, and what remains unfinished.

Month 3 Scope

Days 61 through 90 focused on:

- authenticated user identity
- user-owned tasks
- automated service tests
- GitHub Actions
- request logging
- structured error responses
- health checks
- secret handling
- environment-specific configuration
- local setup documentation
- manual API regression testing
- repository cleanup
- project documentation

This phase moved JobTrackr from basic authentication toward a more dependable backend foundation.

Days 61-68: Authentication And Ownership

The first part of Month 3 connected JWT identity to task behavior.

Completed work includes:

- reviewing registration and login
- reviewing JWT generation and validation
- protecting task endpoints
- reading the authenticated user id from JWT claims
- removing UserId from public task creation requests
- assigning new tasks to the authenticated user
- restricting task lists to the authenticated user's tasks
- restricting Get By Id to the owner
- restricting Update to the owner
- restricting Delete to the owner

```
testing isolation with two authenticated users
```

This establishes an important rule:

```
task ownership comes from authenticated identity, not client-supplied ownership data
```

Current Ownership Matrix

Operation	Current behavior
Create	Uses authenticated user id
Get All	Returns only the authenticated user's tasks
Get By Id	Returns only an owned task
Update	Updates only an owned task
Delete	Deletes only an owned task
Complete	Requires authentication; ownership not checked
Reopen	Requires authentication; ownership not checked

For ownership-protected operations, another user's task appears as:

```
404 Not Found
```

This avoids confirming that another user's resource exists.

Known Ownership Gap

Complete and Reopen still call service methods without the authenticated user identifier.

Therefore:

```
authentication is complete for these routes
ownership authorization is not complete
```

The month review does not hide this distinction.

It is one of the highest-priority items for the next phase.

Days 69-76: Automated Service Tests

The second part of the month introduced repeatable automated testing.

Completed work includes:

```
xUnit test project
password hashing test
task creation tests
task Get By Id tests
task update tests
task deletion tests
user service tests
authentication service tests
Release build readiness review
```

The project reached fifteen automated tests.

Current Automated Coverage

Authentication

- password hashing and verification
- registration
- valid login
- invalid-password login

Tasks

- valid creation
- empty-title creation
- existing task retrieval
- missing task retrieval
- valid update
- missing-task update
- empty-title update
- successful deletion
- missing-task deletion

Users

- valid user creation
- existing user retrieval

The service tests use EF Core InMemory and do not require SQL Server.

Verified Test Result

The complete Release test run reports:

```
Passed: 15
Failed: 0
Skipped: 0
```

This is the verified Day 90 result, not only an expected value copied from documentation.

Current Testing Gaps

The suite does not yet provide broad HTTP integration coverage.

Future automated tests should cover:

- controller routes and response contracts
- JWT authentication through the HTTP pipeline
- ProblemDetails serialization
- health endpoint behavior
- two-user ownership through real API requests
- Complete and Reopen ownership
- user endpoint authorization
- SQL Server integration where useful

The manual regression flow currently supplies evidence for several of these areas, but automation would make the evidence repeatable in CI.

Days 77-79: Continuous Integration

GitHub Actions was added to run on pushes and pull requests targeting main.

The workflow:

- checks out the repository
- installs .NET 8
- restores API dependencies
- restores test dependencies
- builds production projects in Release configuration
- runs all fifteen tests

It uses read-only repository permissions.

The workflow does not require SQL Server because the existing tests use EF Core InMemory.

Why CI Matters

Local success can depend on cached packages, installed tooling, or machine state.

GitHub Actions repeats the build on a clean runner and answers:

```
can the repository restore from a clean checkout?  
does production code compile?  
does the automated suite pass?
```

This provides a shared baseline for future changes.

README After CI

The repository README was updated to describe:

```
authentication behavior  
task ownership  
the fifteen tests  
local test commands  
GitHub Actions behavior  
known authorization limitations
```

Documentation was treated as part of the deliverable rather than an end-of-project task.

Days 80-89: API Reliability

The final third of Month 3 focused on operational behavior and maintainability.

Completed reliability work includes:

```
safe request logging  
error-response review  
ProblemDetails  
global exception logging  
health endpoint  
security review  
configuration cleanup  
local setup guide  
manual regression flow  
formatting cleanup  
README and progress updates
```

Request Logging

RequestLoggingMiddleware records:

```
HTTP method  
request path  
response status  
elapsed milliseconds
```

It deliberately avoids:

```
request bodies  
passwords  
JWT tokens  
Authorization headers  
connection strings  
signing keys
```

The middleware uses structured placeholders so logging systems can later query individual fields.

Structured Error Responses

Known controller errors and unexpected failures now use ASP.NET Core ProblemDetails.

Current behavior includes:

```
invalid DTO -> 400 ValidationProblemDetails
known bad request -> 400 ProblemDetails
missing resource -> 404 ProblemDetails
unexpected exception -> safe 500 ProblemDetails
missing authentication -> standard 401 response
```

Unexpected exception details are logged on the server, while clients receive a generic message.

This preserves diagnostic value without exposing implementation details.

Health Endpoint

JobTrackr provides:

```
GET /health
```

The basic public endpoint returns:

```
200 OK
Healthy
```

It confirms:

```
the API process is running
ASP.NET Core can answer a request
```

It does not currently confirm:

```
SQL Server availability
migration state
external service health
every endpoint's behavior
```

Database-aware checks remain optional future work when operational requirements justify them.

Secret Handling

The development JWT signing key was removed from tracked appsettings.

A replacement key is stored through .NET User Secrets.

Current configuration sources are:

Setting	Source
JWT issuer and audience	appsettings.json
Local SQL Server connection	appsettings.Development.json
Local JWT signing key	.NET User Secrets

Tracked configuration was checked for the removed development key and database password patterns.

The review found:

```
old JWT key not present
database password not present in tracked JSON or Markdown
local Jwt:Key secret is configured
```

Secret values were not printed during verification.

Environment Configuration

The local SQL Server connection is separated from shared settings.

The HTTPS launch profile is the default development profile.

The local setup documentation explains:

```
.NET prerequisites
EF Core tools
dependency restoration
SQL Server requirements
migration command
User Secrets setup
HTTPS certificate trust
build and tests
API startup
health verification
registration and login
Bearer-token requests
```

Another developer can follow the setup without receiving a committed JWT key.

Manual Regression Testing

Day 87 exercised the application from start to finish.

The flow covered:

```
health
two-user registration
validation errors
two-user login
anonymous 401 behavior
task creation
task filters and search
task ownership isolation
task update and deletion
complete and reopen for the owner
public user CRUD
structured 404 responses
request log safety
```

The test deliberately avoided using User B to mutate User A's task through Complete or Reopen because the known ownership gap could change another user's data.

This is careful testing rather than pretending a known unsafe case is complete.

Repository Cleanup

The source audit found:

```
no WeatherForecast starter code
no Class1 placeholders
no empty C# files
no TODO or HACK placeholders
no NotImplementedException
no obvious dead application classes
```

Only formatting consistency and one unnecessary blank line required changes.

Valid migrations, interfaces, middleware, DTOs, and infrastructure files were preserved.

Architecture Review

The project retains five clear areas:

```
JobTrackr.Api
JobTrackr.Application
JobTrackr.Domain
JobTrackr.Infrastructure
JobTrackr.Tests
```

API

Contains controllers, middleware, authentication configuration, dependency registration, health routing, and startup.

Application

Contains DTOs, service contracts, shared messages, and authentication abstractions.

Domain

Contains the User and JobTask entities.

Infrastructure

Contains EF Core persistence, SQL Server services, authentication implementation, JWT generation, and migrations.

Tests

Contains xUnit service tests using isolated in-memory databases.

Controllers do not access AppDbContext directly.

That preserves the intended application and infrastructure boundary.

Verified Release Build

The final Month 3 Release build reports:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

All projects restore and compile from the current repository state.

Repository State

The latest source commit is:

```
fed948d Day 89: Update project documentation
```

The working tree is clean.

Day 90 does not create an empty commit because no source or repository documentation changed during the successful review.

Documentation Review

The README now includes:

```
current features  
health, authentication, task, and user endpoints  
authentication behavior  
task ownership behavior  
authorization limitations  
reliability behavior  
SQL Server and migration setup  
User Secrets setup  
HTTPS startup  
automated tests  
manual regression status  
GitHub Actions behavior  
accurate next steps
```

GET /health appears in the main endpoint list rather than only in setup instructions.

Known Security Limitations

Month 3 ends with three explicit gaps:

- Complete does not verify task ownership
- Reopen does not verify task ownership
- UserController is not protected by authorization

Swagger also does not yet include Bearer-token authorization configuration.

These are planned improvements, not hidden failures.

Month 3 Result

JobTrackr now has a stable backend foundation containing:

- SQL Server persistence
- JWT authentication
- authenticated task ownership
- structured API errors
- safe request logging
- public process health
- secure local secret handling
- environment-specific configuration
- automated service tests
- continuous integration
- manual regression evidence
- reproducible local setup documentation

The project is ready to continue, but it is not presented as finished.

Evidence Summary

Area	Result
Release build	Passed with 0 warnings and 0 errors
Automated tests	15 passed, 0 failed, 0 skipped
Git status	Clean
Latest commit	fed948d
Architecture boundary	Controllers do not use ApplicationDbContext
CI configuration	Release build and tests configured
Tracked old JWT key	Not found
Tracked database password	Not found
Local JWT secret	Configured
README	Current and complete for Month 3

Next Phase: Days 91-120

The next phase will focus on:

- task pagination
- sorting by created and due dates
- combining filters, sorting, and pagination
- authenticated profile endpoints
- change-password behavior
- authorization tests
- Swagger Bearer-token configuration
- API integration tests
- database reset and migration documentation

Day 91 Direction

Day 91 begins by planning query improvements.

Before implementing pagination, the project should define predictable parameters for:

```
page number
page size
sort field
sort direction
existing search filter
existing completion filter
```

Planning the contract first reduces accidental breaking changes.

What This Review Teaches

A phase review should be evidence-based.

The important habits are:

```
verify the repository state
run the exact Release build
run the complete test suite
inspect architecture boundaries
review configuration and secret storage
compare documentation with code
state known limitations plainly
avoid adding unrelated work during review
avoid empty milestone commits
define the next phase without starting it early
```

Stable software grows through verified increments, not through feature count alone.

Reader Exercise

Create a phase-review table for another backend.

Include:

```
features completed
behavioral evidence
test coverage
CI status
security boundaries
configuration sources
runtime health
documentation accuracy
known gaps
next-phase priorities
```

For every completion claim, attach a build result, test, route, configuration check, or code reference.

Next Step

Start Day 91 by planning pagination, sorting, and predictable task-query behavior before changing the API contract.